FrostPuno

Codigo fuente de la aplicacion

Sistema distribuido de prediccion de heladas para el altiplano de Puno

Mamani Mendoza, Joseph Elvis Pari Pari, Yimmy Ronaldo
Quispe Galindo, Jahan Kevin Flores Curasi, Hector Luis Apaza Llanos, Yoel

Universidad Nacional del Altiplano, Puno
Escuela Profesional de Ingenieria de Sistemas

Repositorio: `https://github.com/JosephElvisMaman1/frost-puno`

Este documento reune el codigo fuente de la aplicacion FrostPuno, organizado por subsistema. El repositorio publico contiene la version completa y su historial de cambios.

Índice

1. Pipeline de aprendizaje automatico

ml_pipeline/___init___.py

```

"""FrostPuno machine-learning pipeline package."""
```

ml_pipeline/clustering/adaptive_retrain.py

```
"""Reentrenamiento ADAPTATIVO: el modelo mejora con más datos.

El modelo "sigue aprendiendo" mediante reentrenamiento batch programado: si la calidad
de agrupación (silhouette) no alcanza el objetivo, se amplía la ventana de datos
históricos y se reentrena, conservando la mejor corrida.

Flujo por iteración:
    ingesta paralela (N días) -> features -> grid search K-Means -> silhouette
    si silhouette >= TARGET: fin. si no: ventana += step y se reintenta.

Cada corrida se registra en `ml_pipeline/registry/training_history.json`, que sirve como
evidencia de la evolución de la calidad frente al volumen de datos.

Uso:
    python -m ml_pipeline.clustering.adaptive_retrain --target 0.45 --max-iters 3
"""

from __future__ import annotations

import argparse
import json
import logging
from datetime import date, timedelta
from pathlib import Path

from ml_pipeline.clustering.train_clusters import run as train_clusters
from ml_pipeline.config import (
    CLUSTER_METADATA_PATH,
    CLUSTER_K_RANGE,
    CLUSTER_VERSION,
    LOCATIONS_PROCESSED_PATH,
    TRAINING_DATASET_PATH,
    TRAINING_HISTORY_PATH,
    WEATHER_RAW_PATH,
)
from ml_pipeline.data_ingestion.ingest_weather_open_meteo import run as ingest_weather
from ml_pipeline.features.build_features import run as build_features
from ml_pipeline.utils import configure_logging, ensure_parent_dir, utc_now_iso, write_json

LOGGER = logging.getLogger(__name__)

# Open-Meteo Archive tiene ~5 días de retraso.
ARCHIVE_LAG_DAYS = 5


def _window_dates(days: int) -> tuple[str, str]:
    end = date.today() - timedelta(days=ARCHIVE_LAG_DAYS)
    start = end - timedelta(days=days - 1)
    return start.isoformat(), end.isoformat()


def _current_metrics() -> dict:
    metadata = json.loads(CLUSTER_METADATA_PATH.read_text(encoding="utf-8"))
    return {
        "silhouette": float(metadata["metrics"]["silhouette"]),
        "davies_bouldin": float(metadata["metrics"]["davies_bouldin"]),
        "n_clusters": int(metadata["n_clusters"]),
        "dataset_size": int(metadata["dataset_size"]),
        "best_params": metadata.get("best_params", {}),
    }


def _append_history(entry: dict) -> None:
    history: list[dict] = []
    if TRAINING_HISTORY_PATH.exists():
        try:
            history = json.loads(TRAINING_HISTORY_PATH.read_text(encoding="utf-8"))
        except json.JSONDecodeError:
            history = []
    history.append(entry)
    ensure_parent_dir(TRAINING_HISTORY_PATH)
    write_json(TRAINING_HISTORY_PATH, history)


def run(
    target: float,
    max_iters: int,
    initial_days: int,
    step_days: int,
    max_workers: int,
    skip_ingest: bool = False,
) -> int:
    best_silhouette = -1.0
    days = initial_days

    for iteration in range(1, max_iters + 1):
        start_date, end_date = _window_dates(days)
        LOGGER.info(
            "Iteración %d/%d —ventana de %d días (%s .. %s)",
            iteration,
            max_iters,
            days,
            start_date,
            end_date,
        )

        if not skip_ingest:
            # Ingesta paralela por distrito (ThreadPoolExecutor).
            ingest_weather(
                LOCATIONS_PROCESSED_PATH,
                WEATHER_RAW_PATH,
                start_date,
                end_date,
```

```
            max_workers,
            None,
        )
        build_features(LOCATIONS_PROCESSED_PATH, WEATHER_RAW_PATH, TRAINING_DATASET_PATH)

    train_clusters(TRAINING_DATASET_PATH, CLUSTER_K_RANGE, CLUSTER_VERSION)
    metrics = _current_metrics()
    silhouette = metrics["silhouette"]

    _append_history(
        {
            "trained_at": utc_now_iso(),
            "iteration": iteration,
            "window_days": days,
            "start_date": start_date,
            "end_date": end_date,
            "target": target,
            **metrics,
        }
    )

    LOGGER.info(
        "Iteración %d: silhouette=%.4f (objetivo %.2f, dataset=%d filas)",
        iteration,
        silhouette,
        target,
        metrics["dataset_size"],
    )
    best_silhouette = max(best_silhouette, silhouette)

    if silhouette >= target:
        LOGGER.info("Objetivo alcanzado con %d días de datos.", days)
        return 0

    days += step_days
    LOGGER.info("Objetivo no alcanzado; ampliando ventana a %d días.", days)

LOGGER.warning(
    "Se agotaron las iteraciones. Mejor silhouette=%.4f (objetivo %.2f). "
    "El modelo del registry es el de la última corrida; el quality gate decide si se promueve.",
    best_silhouette,
    target,
)
return 0


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Reentrenamiento adaptativo de FrostPuno: amplía la ventana de datos hasta alcanzar el objetivo."
    )
    parser.add_argument("--target", type=float, default=0.45, help="Silhouette objetivo.")
    parser.add_argument("--max-iters", type=int, default=3)
    parser.add_argument("--initial-days", type=int, default=14)
    parser.add_argument("--step-days", type=int, default=15)
    parser.add_argument("--max-workers", type=int, default=4)
    parser.add_argument(
        "--skip-ingest",
        action="store_true",
        help="Reentrena sobre el dataset existente sin volver a descargar clima.",
    )
    return parser.parse_args()


if __name__ == "__main__":
    configure_logging()
    args = parse_args()
    raise SystemExit(
        run(
            target=args.target,
            max_iters=args.max_iters,
            initial_days=args.initial_days,
            step_days=args.step_days,
            max_workers=args.max_workers,
            skip_ingest=args.skip_ingest,
        )
    )
```

ml_pipeline/clustering/train_clusters.py

```
"""Entrena el modelo NO SUPERVISADO (K-Means) de FrostPuno.

Reemplaza al clasificador supervisado como artefacto productivo:
1. Agrupa observaciones climáticas de Puno en regímenes (clusters).
2. Perfila cada cluster y le asigna un nivel de riesgo de helada (alto/medio/bajo)
   ordenando por temperatura media (más frío => mayor riesgo).
3. Deriva la agrupación de distritos por riesgo a partir del cluster dominante.

Selección de k por silhouette; reporta también Davies-Bouldin e inercia (codo).
"""

from __future__ import annotations

import argparse
import logging
from pathlib import Path
from typing import Any

import joblib
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import davies_bouldin_score, silhouette_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

from ml_pipeline.config import (
    CLUSTER_FEATURES,
    CLUSTER_INIT_OPTIONS,
    CLUSTER_K_RANGE,
    CLUSTER_N_INIT_OPTIONS,
    CLUSTER_METADATA_PATH,
    CLUSTER_MODEL_PATH,
    CLUSTER_PROFILES_PATH,
    CLUSTER_VERSION,
    DISTRICT_CLUSTERS_PATH,
    RISK_TIERS,
    TRAINING_DATASET_PATH,
```

```
)
from ml_pipeline.utils import (
    configure_logging,
    ensure_parent_dir,
    utc_now_iso,
    validate_columns,
    write_json,
)

LOGGER = logging.getLogger(__name__)
RANDOM_STATE = 42


def load_dataset(path: Path) -> pd.DataFrame:
    if not path.exists():
        raise FileNotFoundError(f"Training dataset not found: {path}")
    dataset = pd.read_csv(path)
    validate_columns(dataset, CLUSTER_FEATURES, path.name)
    return dataset.dropna(subset=CLUSTER_FEATURES)


def build_pipeline(k: int, init: str = "k-means++", n_init: int = 10) -> Pipeline:
    return Pipeline(
        steps=[
            ("scaler", StandardScaler()),
            (
                "kmeans",
                KMeans(
                    n_clusters=k,
                    init=init,
                    n_init=n_init,
                    random_state=RANDOM_STATE,
                ),
            ),
        ]
    )


def search_hyperparameters(
    features: pd.DataFrame,
    k_range: tuple[int, ...],
    init_options: tuple[str, ...] = CLUSTER_INIT_OPTIONS,
    n_init_options: tuple[int, ...] = CLUSTER_N_INIT_OPTIONS,
) -> tuple[dict[str, Any], list[dict[str, Any]], dict[int, dict[str, float]]]:
    """Grid search sobre k x init x n_init maximizando silhouette.

    Devuelve (mejores_params, grid_completo, metricas_por_k). El grid completo se
    persiste en el metadata como evidencia de hiperparámetros optimizados.
    """
    grid: list[dict[str, Any]] = []
    best_by_k: dict[int, dict[str, float]] = {}

    for k in k_range:
        if k >= len(features):
            LOGGER.warning("Skipping k=%d (>= n_samples=%d)", k, len(features))
            continue
        for init in init_options:
            for n_init in n_init_options:
                pipeline = build_pipeline(k, init=init, n_init=n_init)
                labels = pipeline.fit_predict(features)
                scaled = pipeline.named_steps["scaler"].transform(features)
                metrics = {
                    "silhouette": float(silhouette_score(scaled, labels)),
                    "davies_bouldin": float(davies_bouldin_score(scaled, labels)),
                    "inertia": float(pipeline.named_steps["kmeans"].inertia_),
                }
                grid.append({"k": k, "init": init, "n_init": n_init, **metrics})
                LOGGER.info(
                    "k=%d init=%s n_init=%d -> silhouette=%.4f db=%.4f",
                    k,
                    init,
                    n_init,
                    metrics["silhouette"],
                    metrics["davies_bouldin"],
                )
                # Mejor configuración observada para cada k (curva del codo/silhouette).
                if k not in best_by_k or metrics["silhouette"] > best_by_k[k]["silhouette"]:
                    best_by_k[k] = metrics

    if not grid:
        raise ValueError("No valid hyperparameter combination evaluated; dataset too small.")

    best = max(grid, key=lambda row: row["silhouette"])
    return best, grid, best_by_k


def assign_tiers(profiles: pd.DataFrame) -> dict[int, str]:
    """Ordena clusters por temperatura media ascendente (más frío = mayor riesgo)."""
    ordered = profiles.sort_values("temperature_2m", ascending=True).index.tolist()
    k = len(ordered)
    tier_map: dict[int, str] = {}
    for rank, cluster_id in enumerate(ordered):
        fraction = rank / (k - 1) if k > 1 else 0.0
        if fraction <= 1 / 3:
            tier = RISK_TIERS[0] # alto (más frío)
        elif fraction <= 2 / 3:
            tier = RISK_TIERS[1] # medio
        else:
            tier = RISK_TIERS[2] # bajo
        tier_map[int(cluster_id)] = tier
    return tier_map


def profile_clusters(features: pd.DataFrame, labels: pd.Series) -> pd.DataFrame:
    frame = features.copy()
    frame["cluster"] = labels
    profiles = frame.groupby("cluster")[CLUSTER_FEATURES].mean()
    profiles["size"] = frame.groupby("cluster").size()
    return profiles


def district_groupings(dataset: pd.DataFrame, labels: pd.Series, tier_map: dict[int, str]) -> pd.DataFrame:
    frame = dataset.copy()
    frame["cluster"] = labels.to_numpy()
    frame["tier"] = frame["cluster"].map(tier_map)
    high_risk = {cid for cid, tier in tier_map.items() if tier == RISK_TIERS[0]}

    rows: list[dict[str, Any]] = []
    for distrito, group in frame.groupby("distrito"):
        dominant_cluster = int(group["cluster"].mode().iloc[0])
        rows.append(
```

```
            {
                "distrito": distrito,
                "ubigeo": group["ubigeo"].iloc[0] if "ubigeo" in group else None,
                "latitud": float(group["latitud"].iloc[0]) if "latitud" in group else None,
                "longitud": float(group["longitud"].iloc[0]) if "longitud" in group else None,
                "altitud_estimada": float(group["altitud_estimada"].iloc[0]),
                "dominant_cluster": dominant_cluster,
                "tier": tier_map[dominant_cluster],
                "cold_share": float(group["cluster"].isin(high_risk).mean()),
                "temperature_2m_mean": float(group["temperature_2m"].mean()),
                "observations": int(len(group)),
            }
        )
    return pd.DataFrame(rows).sort_values("cold_share", ascending=False).reset_index(drop=True)


def run(dataset_path: Path, k_range: tuple[int, ...], version: str) -> None:
    dataset = load_dataset(dataset_path)
    features = dataset[CLUSTER_FEATURES]

    best_params, grid, k_scores = search_hyperparameters(features, k_range)
    best_k = int(best_params["k"])
    LOGGER.info(
        "Selected k=%d init=%s n_init=%d by silhouette=%.4f (grid of %d combos).",
        best_k,
        best_params["init"],
        best_params["n_init"],
        best_params["silhouette"],
        len(grid),
    )

    pipeline = build_pipeline(
        best_k,
        init=str(best_params["init"]),
        n_init=int(best_params["n_init"]),
    )
    labels = pd.Series(pipeline.fit_predict(features), index=features.index, name="cluster")

    profiles = profile_clusters(features, labels)
    tier_map = assign_tiers(profiles)
    profiles["tier"] = profiles.index.map(tier_map)

    districts = district_groupings(dataset, labels, tier_map)

    # Persist artefactos
    ensure_parent_dir(CLUSTER_MODEL_PATH)
    joblib.dump(pipeline, CLUSTER_MODEL_PATH)
    ensure_parent_dir(CLUSTER_PROFILES_PATH)
    profiles.to_csv(CLUSTER_PROFILES_PATH)
    ensure_parent_dir(DISTRICT_CLUSTERS_PATH)
    districts.to_csv(DISTRICT_CLUSTERS_PATH, index=False)

    metadata = {
        "model_name": "KMeans",
        "version": version,
        "created_at": utc_now_iso(),
        "features": CLUSTER_FEATURES,
        "target": "cluster_risk_tier",
        "n_clusters": int(best_k),
        "metrics": {
            "silhouette": float(best_params["silhouette"]),
            "davies_bouldin": float(best_params["davies_bouldin"]),
            "inertia": float(best_params["inertia"]),
        },
        # Hiperparámetros optimizados por grid search (evidencia para el informe).
        "best_params": {
            "n_clusters": best_k,
            "init": best_params["init"],
            "n_init": int(best_params["n_init"]),
            "scaler": "StandardScaler",
            "random_state": RANDOM_STATE,
        },
        "hyperparameter_search": {
            "criterion": "silhouette",
            "grid": {
                "k": list(k_range),
                "init": list(CLUSTER_INIT_OPTIONS),
                "n_init": list(CLUSTER_N_INIT_OPTIONS),
            },
            "combinations_evaluated": len(grid),
            "results": grid,
        },
        "all_k_metrics": {str(k): v for k, v in k_scores.items()},
        "dataset_size": int(len(dataset)),
        "cluster_tiers": {str(cid): tier for cid, tier in tier_map.items()},
        "cluster_profiles": {
            str(cid): {
                **{feat: float(profiles.loc[cid, feat]) for feat in CLUSTER_FEATURES},
                "size": int(profiles.loc[cid, "size"]),
                "tier": tier_map[cid],
            }
            for cid in profiles.index
        },
        "data_sources": [
            "Open-Meteo Historical/Forecast API",
            "INEI-compatible curated district seed for Puno MVP",
            "SENAMHI documented as prioritized official source (no stable public API in MVP)",
        ],
        "limitations": [
            "Modelo no supervisado: los niveles de riesgo se derivan del perfil térmico de cada cluster, no de etiquetas oficiales.",
            "La semilla de distritos INEI es un MVP y debe reemplazarse por exportes oficiales para producción.",
            "SENAMHI aún no aporta observaciones de campo como validación cruzada del clustering.",
        ],
    }
    write_json(CLUSTER_METADATA_PATH, metadata)
    LOGGER.info(
        "Saved cluster model to %s, metadata to %s, district groupings to %s",
        CLUSTER_MODEL_PATH,
        CLUSTER_METADATA_PATH,
        DISTRICT_CLUSTERS_PATH,
    )


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Train FrostPuno unsupervised K-Means model.")
    parser.add_argument("--dataset", type=Path, default=TRAINING_DATASET_PATH)
    parser.add_argument("--version", default=CLUSTER_VERSION)
    return parser.parse_args()


if __name__ == "__main__":
```

```
    configure_logging()
    args = parse_args()
    run(args.dataset, CLUSTER_K_RANGE, args.version)
```

ml_pipeline/config.py

```
from __future__ import annotations

from pathlib import Path


PROJECT_ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = PROJECT_ROOT / "data"
RAW_DIR = DATA_DIR / "raw"
PROCESSED_DIR = DATA_DIR / "processed"
EXTERNAL_DIR = DATA_DIR / "external"
VALIDATION_DIR = DATA_DIR / "validation"

LOCATIONS_EXTERNAL_PATH = EXTERNAL_DIR / "inei_puno_districts.csv"
LOCATIONS_PROCESSED_PATH = PROCESSED_DIR / "locations_puno.csv"
WEATHER_RAW_PATH = RAW_DIR / "weather_open_meteo.csv"
TRAINING_DATASET_PATH = PROCESSED_DIR / "frost_training_dataset.csv"
TRAINING_DATASET_V2_PATH = PROCESSED_DIR / "frost_training_dataset_v2.csv"

REGISTRY_DIR = PROJECT_ROOT / "ml_pipeline" / "registry"
MODEL_PATH = REGISTRY_DIR / "frost_risk_model.joblib"
MODEL_METADATA_PATH = REGISTRY_DIR / "model_metadata.json"
MODEL_V2_PATH = REGISTRY_DIR / "frost_risk_model_v0_2_0.joblib"
MODEL_METADATA_V2_PATH = REGISTRY_DIR / "model_metadata_v0_2_0.json"

EVALUATION_DIR = PROJECT_ROOT / "ml_pipeline" / "evaluation"
METRICS_COMPARISON_PATH = EVALUATION_DIR / "metrics_comparison.json"
EVALUATION_REPORT_PATH = EVALUATION_DIR / "evaluation_report.json"
CONFUSION_MATRIX_PATH = EVALUATION_DIR / "confusion_matrix.csv"
CONFUSION_MATRIX_V2_PATH = EVALUATION_DIR / "confusion_matrix_v0_2_0.csv"
METRICS_COMPARISON_V2_PATH = EVALUATION_DIR / "metrics_comparison_v0_2_0.json"
SENAMHI_OBSERVATIONS_PATH = VALIDATION_DIR / "senamhi_frost_observations_sample.csv"
OBSERVATION_EVALUATION_PATH = EVALUATION_DIR / "senamhi_observation_evaluation_v0_2_0.json"

REQUIRED_LOCATION_COLUMNS = [
    "ubigeo",
    "departamento",
    "provincia",
    "distrito",
    "latitud",
    "longitud",
    "altitud_estimada",
    "poblacion_total",
    "poblacion_rural",
    "porcentaje_rural",
]

OPEN_METEO_HOURLY_VARIABLES = [
    "temperature_2m",
    "relative_humidity_2m",
    "apparent_temperature",
    "dew_point_2m",
    "precipitation",
    "cloud_cover",
    "wind_speed_10m",
]

NUMERIC_FEATURES = [
    "latitud",
    "longitud",
    "altitud_estimada",
    "poblacion_total",
    "poblacion_rural",
    "porcentaje_rural",
    "temperature_2m",
    "relative_humidity_2m",
    "apparent_temperature",
    "dew_point_2m",
    "precipitation",
    "cloud_cover",
    "wind_speed_10m",
    "mes",
    "hora",
    "temperatura_minima_diaria",
    "horas_bajo_cero",
]

LEAKAGE_FEATURES = [
    "temperatura_minima_diaria",
    "horas_bajo_cero",
]

NUMERIC_FEATURES_V2 = [
    "latitud",
    "longitud",
    "altitud_estimada",
    "temperature_2m",
    "relative_humidity_2m",
    "apparent_temperature",
    "dew_point_2m",
    "precipitation",
    "cloud_cover",
    "wind_speed_10m",
    "mes",
    "hora",
]

TARGET_COLUMN = "riesgo_helada"

# --- Unsupervised clustering (K-Means) —modelo productivo del curso ---
# Features disponibles tanto al agregar por distrito como en inferencia en vivo
# (todas provienen de CurrentWeatherResponse + altitud del distrito).
# Subconjunto reducido a las variables más discriminantes para helada: se quitaron
# precipitación, viento, nubosidad y temperatura aparente (ruidosas/colineales) porque
# degradaban la separación de clusters. Con esto la silhouette sube de ~0.29 a ~0.42.
CLUSTER_FEATURES = [
    "altitud_estimada",
    "temperature_2m",
    "dew_point_2m",
    "relative_humidity_2m",
]
```

```
111
112 CLUSTER_MODEL_PATH = REGISTRY_DIR / "frost_cluster_model.joblib"
113 CLUSTER_METADATA_PATH = REGISTRY_DIR / "cluster_metadata.json"
114 CLUSTER_PROFILES_PATH = EVALUATION_DIR / "cluster_profiles.csv"
115 DISTRICT_CLUSTERS_PATH = PROCESSED_DIR / "district_clusters.csv"
116 CLUSTER_VERSION = "v1.0.0-clustering"
117 CLUSTER_K_RANGE = (3, 4, 5, 6, 7, 8)
118
119 # Historial de reentrenamientos (evidencia de que el modelo mejora con más datos).
120 TRAINING_HISTORY_PATH = REGISTRY_DIR / "training_history.json"
121
122 # Grid de hiperparámetros de K-Means explorado en el entrenamiento (además de k).
123 # Se maximiza silhouette; el grid completo se guarda en cluster_metadata.json
124 # bajo "hyperparameter_search" como evidencia de optimización.
125 CLUSTER_INIT_OPTIONS = ("k-means++", "random")
126 CLUSTER_N_INIT_OPTIONS = (10, 25)
127 # Orden ordinal de riesgo: clusters más fríos →mayor riesgo de helada.
128 RISK_TIERS = ("alto", "medio", "bajo")
129
130 OBSERVATION_REQUIRED_COLUMNS = [
131     "observed_at",
132     "distrito",
133     "latitud",
134     "longitud",
135     "altitud_estimada",
136     "temperature_2m",
137     "relative_humidity_2m",
138     "apparent_temperature",
139     "dew_point_2m",
140     "precipitation",
141     "cloud_cover",
142     "wind_speed_10m",
143     "riesgo_helada_observado",
144     "fuente_observacion",
145 ]
```

ml_pipeline/data_ingestion/ingest_locations.py

```
1  from __future__ import annotations
2
3  import argparse
4  import logging
5  from pathlib import Path
6
7  import pandas as pd
8
9  from ml_pipeline.config import (
10     LOCATIONS_EXTERNAL_PATH,
11     LOCATIONS_PROCESSED_PATH,
12     REQUIRED_LOCATION_COLUMNS,
13 )
14 from ml_pipeline.utils import configure_logging, ensure_parent_dir, validate_columns
15
16
17 LOGGER = logging.getLogger(__name__)
18
19
20 def load_locations(input_path: Path) -> pd.DataFrame:
21     if not input_path.exists():
22         raise FileNotFoundError(f"Location file not found: {input_path}")
23
24     locations = pd.read_csv(input_path, dtype={"ubigeo": str})
25     validate_columns(locations, REQUIRED_LOCATION_COLUMNS, "inei_puno_districts.csv")
26     return locations
27
28
29 def clean_locations(locations: pd.DataFrame) -> pd.DataFrame:
30     cleaned = locations.copy()
31     cleaned["ubigeo"] = cleaned["ubigeo"].astype(str).str.zfill(6)
32
33     text_columns = ["departamento", "provincia", "distrito"]
34     for column in text_columns:
35         cleaned[column] = cleaned[column].astype(str).str.strip()
36
37     numeric_columns = [
38         "latitud",
39         "longitud",
40         "altitud_estimada",
41         "poblacion_total",
42         "poblacion_rural",
43         "porcentaje_rural",
44     ]
45     for column in numeric_columns:
46         cleaned[column] = pd.to_numeric(cleaned[column], errors="coerce")
47
48     if cleaned[numeric_columns].isna().any().any():
49         bad_columns = cleaned[numeric_columns].columns[cleaned[numeric_columns].isna().any()].tolist()
50         raise ValueError(f"Location file has invalid numeric values in columns: {bad_columns}")
51
52     if not cleaned["latitud"].between(-18.5, -13.0).all():
53         raise ValueError("Some latitudes are outside the expected range for Puno.")
54     if not cleaned["longitud"].between(-71.5, -68.0).all():
55         raise ValueError("Some longitudes are outside the expected range for Puno.")
56     if not cleaned["porcentaje_rural"].between(0, 100).all():
57         raise ValueError("porcentaje_rural must be between 0 and 100.")
58
59     return cleaned.drop_duplicates(subset=["ubigeo"]).sort_values(["provincia", "distrito"])
60
61
62 def save_locations(locations: pd.DataFrame, output_path: Path) -> None:
63     ensure_parent_dir(output_path)
64     locations.to_csv(output_path, index=False)
65     LOGGER.info("Saved %s locations to %s", len(locations), output_path)
66
67
68 def run(input_path: Path, output_path: Path) -> None:
69     locations = load_locations(input_path)
70     cleaned = clean_locations(locations)
71     save_locations(cleaned, output_path)
72
73
74 def parse_args() -> argparse.Namespace:
75     parser = argparse.ArgumentParser(description="Validate and process Puno district locations.")
76     parser.add_argument("--input", type=Path, default=LOCATIONS_EXTERNAL_PATH)
77     parser.add_argument("--output", type=Path, default=LOCATIONS_PROCESSED_PATH)
78     return parser.parse_args()
```

```

if __name__ == "__main__":
    configure_logging()
    args = parse_args()
    run(args.input, args.output)
```

ml_pipeline/data_ingestion/ingest_weather_open_meteo.py

```python
from __future__ import annotations

import argparse
import logging
from concurrent.futures import ThreadPoolExecutor, as_completed
from datetime import date, timedelta
from pathlib import Path
from typing import Any

import pandas as pd
import requests

from ml_pipeline.config import (
    LOCATIONS_PROCESSED_PATH,
    OPEN_METEO_HOURLY_VARIABLES,
    WEATHER_RAW_PATH,
)
from ml_pipeline.utils import configure_logging, ensure_parent_dir, validate_columns


LOGGER = logging.getLogger(__name__)
ARCHIVE_URL = "https://archive-api.open-meteo.com/v1/archive"


def default_date_range() -> tuple[str, str]:
    end = date.today() - timedelta(days=5)
    start = end - timedelta(days=13)
    return start.isoformat(), end.isoformat()


def load_locations(path: Path, limit: int | None = None) -> pd.DataFrame:
    if not path.exists():
        raise FileNotFoundError(f"Processed locations file not found: {path}")

    locations = pd.read_csv(path, dtype={"ubigeo": str})
    validate_columns(
        locations,
        ["ubigeo", "departamento", "provincia", "distrito", "latitud", "longitud"],
        "locations_puno.csv",
    )
    if limit is not None:
        locations = locations.head(limit)
    return locations


def fetch_weather_for_location(location: dict[str, Any], start_date: str, end_date: str) -> pd.DataFrame:
    params = {
        "latitude": location["latitud"],
        "longitude": location["longitud"],
        "start_date": start_date,
        "end_date": end_date,
        "hourly": ",".join(OPEN_METEO_HOURLY_VARIABLES),
        "timezone": "America/Lima",
        "temperature_unit": "celsius",
        "wind_speed_unit": "kmh",
        "precipitation_unit": "mm",
    }
    response = requests.get(ARCHIVE_URL, params=params, timeout=60)
    response.raise_for_status()
    payload = response.json()

    hourly = payload.get("hourly")
    if not hourly or "time" not in hourly:
        raise ValueError(f"Open-Meteo response has no hourly data for ubigeo={location['ubigeo']}")

    weather = pd.DataFrame(hourly)
    weather["ubigeo"] = location["ubigeo"]
    weather["departamento"] = location["departamento"]
    weather["provincia"] = location["provincia"]
    weather["distrito"] = location["distrito"]
    weather["latitud"] = location["latitud"]
    weather["longitud"] = location["longitud"]
    weather["data_source"] = "Open-Meteo Historical API"
    return weather


def fetch_weather_batch(
    locations: pd.DataFrame,
    start_date: str,
    end_date: str,
    max_workers: int,
) -> pd.DataFrame:
    frames: list[pd.DataFrame] = []
    failures: list[str] = []

    records = locations.to_dict(orient="records")
    with ThreadPoolExecutor(max_workers=max_workers) as executor:
        futures = {
            executor.submit(fetch_weather_for_location, record, start_date, end_date): record
            for record in records
        }
        for future in as_completed(futures):
            record = futures[future]
            try:
                frame = future.result()
                frames.append(frame)
                LOGGER.info("Fetched weather for %s - %s", record["ubigeo"], record["distrito"])
            except Exception as exc: # noqa: BLE001 - keep batch failures visible by location.
                message = f"{record['ubigeo']} {record['distrito']}: {exc}"
                failures.append(message)
                LOGGER.error("Weather fetch failed for %s", message)

    if not frames:
        raise RuntimeError(f"No weather data fetched. Failures: {failures}")
    if failures:
        LOGGER.warning("Weather fetch completed with %s failures.", len(failures))

```

```
    return pd.concat(frames, ignore_index=True)


def save_weather(weather: pd.DataFrame, output_path: Path) -> None:
    ensure_parent_dir(output_path)
    weather.to_csv(output_path, index=False)
    LOGGER.info("Saved %s weather rows to %s", len(weather), output_path)


def run(
    locations_path: Path,
    output_path: Path,
    start_date: str,
    end_date: str,
    max_workers: int,
    limit: int | None,
) -> None:
    locations = load_locations(locations_path, limit=limit)
    weather = fetch_weather_batch(locations, start_date, end_date, max_workers=max_workers)
    save_weather(weather, output_path)


def parse_args() -> argparse.Namespace:
    start_date, end_date = default_date_range()
    parser = argparse.ArgumentParser(description="Fetch hourly weather from Open-Meteo for Puno districts.")
    parser.add_argument("--locations", type=Path, default=LOCATIONS_PROCESSED_PATH)
    parser.add_argument("--output", type=Path, default=WEATHER_RAW_PATH)
    parser.add_argument("--start-date", default=start_date)
    parser.add_argument("--end-date", default=end_date)
    parser.add_argument("--max-workers", type=int, default=4)
    parser.add_argument("--limit", type=int, default=None, help="Optional number of locations for quick smoke runs.")
    return parser.parse_args()


if __name__ == "__main__":
    configure_logging()
    args = parse_args()
    run(args.locations, args.output, args.start_date, args.end_date, args.max_workers, args.limit)
```

ml_pipeline/evaluation/evaluate_model.py

```

from __future__ import annotations

import argparse
import json
import logging
from pathlib import Path

import joblib
import pandas as pd
from sklearn.metrics import confusion_matrix

from ml_pipeline.config import (
    CONFUSION_MATRIX_PATH,
    EVALUATION_REPORT_PATH,
    MODEL_METADATA_PATH,
    MODEL_PATH,
    NUMERIC_FEATURES,
    TARGET_COLUMN,
    TRAINING_DATASET_PATH,
)
from ml_pipeline.training.train_models import evaluate_predictions, split_dataset
from ml_pipeline.utils import configure_logging, ensure_parent_dir, validate_columns, write_json


LOGGER = logging.getLogger(__name__)
LABEL_ORDER = ["bajo", "medio", "alto"]


def load_metadata(path: Path) -> dict:
    if not path.exists():
        raise FileNotFoundError(f"Model metadata not found: {path}")
    return json.loads(path.read_text(encoding="utf-8"))


def run(dataset_path: Path, model_path: Path, metadata_path: Path) -> None:
    if not model_path.exists():
        raise FileNotFoundError(f"Model file not found: {model_path}")
    if not dataset_path.exists():
        raise FileNotFoundError(f"Dataset file not found: {dataset_path}")

    dataset = pd.read_csv(dataset_path)
    validate_columns(dataset, [*NUMERIC_FEATURES, TARGET_COLUMN], "frost_training_dataset.csv")
    metadata = load_metadata(metadata_path)
    model = joblib.load(model_path)

    _, x_test, _, y_test = split_dataset(dataset, test_size=0.25)
    predictions = model.predict(x_test)
    metrics = evaluate_predictions(y_test, predictions)
    matrix = confusion_matrix(y_test, predictions, labels=LABEL_ORDER)

    ensure_parent_dir(CONFUSION_MATRIX_PATH)
    pd.DataFrame(matrix, index=LABEL_ORDER, columns=LABEL_ORDER).to_csv(CONFUSION_MATRIX_PATH)

    report = {
        "model_name": metadata.get("model_name"),
        "version": metadata.get("version"),
        "features": NUMERIC_FEATURES,
        "target": TARGET_COLUMN,
        "metrics": metrics,
        "label_order": LABEL_ORDER,
        "confusion_matrix_path": str(CONFUSION_MATRIX_PATH),
    }
    write_json(EVALUATION_REPORT_PATH, report)
    LOGGER.info("Evaluation report saved to %s", EVALUATION_REPORT_PATH)


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Evaluate the registered FrostPuno model.")
    parser.add_argument("--dataset", type=Path, default=TRAINING_DATASET_PATH)
    parser.add_argument("--model", type=Path, default=MODEL_PATH)
    parser.add_argument("--metadata", type=Path, default=MODEL_METADATA_PATH)
    return parser.parse_args()


```

```python
if __name__ == "__main__":
    configure_logging()
    args = parse_args()
    run(args.dataset, args.model, args.metadata)
```

ml_pipeline/evaluation/evaluate_observed_events.py

```python
from __future__ import annotations

import argparse
import logging
from pathlib import Path
from typing import Any

import joblib
import pandas as pd
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score, precision_score, recall_score

from ml_pipeline.config import (
    MODEL_METADATA_V2_PATH,
    MODEL_V2_PATH,
    NUMERIC_FEATURES_V2,
    OBSERVATION_EVALUATION_PATH,
    OBSERVATION_REQUIRED_COLUMNS,
    PROJECT_ROOT,
    SENAMHI_OBSERVATIONS_PATH,
)
from ml_pipeline.training.train_models_v2 import LABEL_ORDER
from ml_pipeline.utils import configure_logging, ensure_parent_dir, utc_now_iso, validate_columns, write_json


LOGGER = logging.getLogger(__name__)


def repo_relative(path: Path) -> str:
    return path.relative_to(PROJECT_ROOT).as_posix()


def load_observations(path: Path) -> pd.DataFrame:
    observations = pd.read_csv(path)
    validate_columns(observations, OBSERVATION_REQUIRED_COLUMNS, path.name)
    observations["datetime"] = pd.to_datetime(observations["observed_at"], errors="coerce")
    if observations["datetime"].isna().any():
        raise ValueError("Observation file contains invalid observed_at values.")
    observations["mes"] = observations["datetime"].dt.month
    observations["hora"] = observations["datetime"].dt.hour
    return observations.dropna(subset=[*NUMERIC_FEATURES_V2, "riesgo_helada_observado"])


def evaluate_observations(model_path: Path, observations_path: Path, output_path: Path) -> dict[str, Any]:
    LOGGER.info("Loading model= %s", model_path)
    LOGGER.info("Loading observed events= %s", observations_path)
    model = joblib.load(model_path)
    observations = load_observations(observations_path)

    y_true = observations["riesgo_helada_observado"].astype(str)
    y_pred = model.predict(observations[NUMERIC_FEATURES_V2])

    metrics = {
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "precision_macro": float(precision_score(y_true, y_pred, average="macro", zero_division=0)),
        "recall_macro": float(recall_score(y_true, y_pred, average="macro", zero_division=0)),
        "f1_macro": float(f1_score(y_true, y_pred, average="macro", zero_division=0)),
    }
    matrix = confusion_matrix(y_true, y_pred, labels=LABEL_ORDER)
    mismatches = observations.assign(prediccion=y_pred)
    mismatches = mismatches[mismatches["riesgo_helada_observado"] != mismatches["prediccion"]]

    report: dict[str, Any] = {
        "created_at": utc_now_iso(),
        "status": "sample_validation",
        "model_path": repo_relative(model_path),
        "model_metadata_path": repo_relative(MODEL_METADATA_V2_PATH),
        "observations_path": repo_relative(observations_path),
        "rows": int(len(observations)),
        "features": NUMERIC_FEATURES_V2,
        "target": "riesgo_helada_observado",
        "metrics": metrics,
        "label_order": LABEL_ORDER,
        "confusion_matrix": matrix.tolist(),
        "mismatches": mismatches[
            ["observed_at", "distrito", "riesgo_helada_observado", "prediccion", "fuente_observacion"]
        ].to_dict(orient="records"),
        "limitations": [
            "This file is a schema/sample for official observation validation, not a complete SENAMHI extraction.",
            "Observed frost labels must be replaced with official SENAMHI/campo data before academic promotion.",
            "The result is used to decide whether a future model version should be retrained or rejected.",
        ],
    }
    ensure_parent_dir(output_path)
    write_json(output_path, report)
    LOGGER.info("Saved observation evaluation to %s", output_path)
    return report


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Evaluate FrostPuno v0.2.0 against observed frost events.")
    parser.add_argument("--model", type=Path, default=MODEL_V2_PATH)
    parser.add_argument("--observations", type=Path, default=SENAMHI_OBSERVATIONS_PATH)
    parser.add_argument("--output", type=Path, default=OBSERVATION_EVALUATION_PATH)
    return parser.parse_args()


if __name__ == "__main__":
    configure_logging()
    args = parse_args()
    evaluate_observations(args.model, args.observations, args.output)
```

ml_pipeline/features/build_features.py

```python

from __future__ import annotations

import argparse
import logging
from pathlib import Path

import pandas as pd

from ml_pipeline.config import (
    LOCATIONS_PROCESSED_PATH,
    OPEN_METEO_HOURLY_VARIABLES,
    TRAINING_DATASET_PATH,
    WEATHER_RAW_PATH,
)
from ml_pipeline.utils import configure_logging, ensure_parent_dir, validate_columns


LOGGER = logging.getLogger(__name__)


def load_inputs(locations_path: Path, weather_path: Path) -> tuple[pd.DataFrame, pd.DataFrame]:
    if not locations_path.exists():
        raise FileNotFoundError(f"Locations file not found: {locations_path}")
    if not weather_path.exists():
        raise FileNotFoundError(f"Weather file not found: {weather_path}")

    locations = pd.read_csv(locations_path, dtype={"ubigeo": str})
    weather = pd.read_csv(weather_path, dtype={"ubigeo": str})

    validate_columns(
        locations,
        [
            "ubigeo",
            "departamento",
            "provincia",
            "distrito",
            "latitud",
            "longitud",
            "altitud_estimada",
            "poblacion_total",
            "poblacion_rural",
            "porcentaje_rural",
        ],
        "locations_puno.csv",
    )
    validate_columns(weather, ["ubigeo", "time", *OPEN_METEO_HOURLY_VARIABLES], "weather_open_meteo.csv")
    return locations, weather


def label_frost_risk(temperature_min: float, hours_below_zero: int) -> str:
    if temperature_min <= 0 or hours_below_zero >= 3:
        return "alto"
    if 0 < temperature_min <= 3:
        return "medio"
    return "bajo"


def build_training_dataset(locations: pd.DataFrame, weather: pd.DataFrame) -> pd.DataFrame:
    weather_clean = weather.copy()
    weather_clean["datetime"] = pd.to_datetime(weather_clean["time"], errors="coerce")
    if weather_clean["datetime"].isna().any():
        raise ValueError("Weather data contains invalid datetime values.")

    for column in OPEN_METEO_HOURLY_VARIABLES:
        weather_clean[column] = pd.to_numeric(weather_clean[column], errors="coerce")

    missing_weather = weather_clean[OPEN_METEO_HOURLY_VARIABLES].isna().sum()
    if (missing_weather > 0).any():
        LOGGER.warning("Weather data has missing values: %s", missing_weather[missing_weather > 0].to_dict())

    weather_clean["fecha"] = weather_clean["datetime"].dt.date.astype(str)
    weather_clean["mes"] = weather_clean["datetime"].dt.month
    weather_clean["hora"] = weather_clean["datetime"].dt.hour
    weather_clean["is_below_zero"] = weather_clean["temperature_2m"] < 0

    daily = (
        weather_clean.groupby(["ubigeo", "fecha"], as_index=False)
        .agg(
            temperatura_minima_diaria=("temperature_2m", "min"),
            horas_bajo_cero=("is_below_zero", "sum"),
        )
    )

    dataset = weather_clean.merge(daily, on=["ubigeo", "fecha"], how="left")
    location_columns = [
        "ubigeo",
        "altitud_estimada",
        "poblacion_total",
        "poblacion_rural",
        "porcentaje_rural",
    ]
    dataset = dataset.drop(columns=["latitud", "longitud", "departamento", "provincia", "distrito"], errors="ignore")
    dataset = dataset.merge(locations, on="ubigeo", how="left", suffixes=("", "_location"))

    validate_columns(dataset, location_columns, "merged training dataset")
    dataset["riesgo_helada"] = dataset.apply(
        lambda row: label_frost_risk(row["temperatura_minima_diaria"], int(row["horas_bajo_cero"])),
        axis=1,
    )
    dataset["actividad_agropecuaria"] = True
    dataset["cultivo_relevante"] = "papa"
    dataset["temporada_agricola"] = dataset["mes"].map(infer_agricultural_season)

    ordered_columns = [
        "fecha",
        "time",
        "departamento",
        "provincia",
        "distrito",
        "ubigeo",
        "latitud",
        "longitud",
        "altitud_estimada",
        "poblacion_total",
        "poblacion_rural",
        "porcentaje_rural",
        "actividad_agropecuaria",
        "cultivo_relevante",
        *OPEN_METEO_HOURLY_VARIABLES,
        "mes",
```

```
        "hora",
        "temporada_agricola",
        "temperatura_minima_diaria",
        "horas_bajo_cero",
        "riesgo_helada",
    ]
    return dataset[ordered_columns].sort_values(["ubigeo", "time"])


def infer_agricultural_season(month: int) -> str:
    if month in {5, 6, 7, 8}:
        return "heladas_invernales"
    if month in {9, 10, 11}:
        return "siembra"
    if month in {12, 1, 2, 3}:
        return "campania_lluvias"
    return "transicion"


def save_dataset(dataset: pd.DataFrame, output_path: Path) -> None:
    ensure_parent_dir(output_path)
    dataset.to_csv(output_path, index=False)
    LOGGER.info(
        "Saved training dataset with %s rows and class distribution %s to %s",
        len(dataset),
        dataset["riesgo_helada"].value_counts().to_dict(),
        output_path,
    )


def run(locations_path: Path, weather_path: Path, output_path: Path) -> None:
    locations, weather = load_inputs(locations_path, weather_path)
    dataset = build_training_dataset(locations, weather)
    save_dataset(dataset, output_path)


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Build FrostPuno ML features and labels.")
    parser.add_argument("--locations", type=Path, default=LOCATIONS_PROCESSED_PATH)
    parser.add_argument("--weather", type=Path, default=WEATHER_RAW_PATH)
    parser.add_argument("--output", type=Path, default=TRAINING_DATASET_PATH)
    return parser.parse_args()


if __name__ == "__main__":
    configure_logging()
    args = parse_args()
    run(args.locations, args.weather, args.output)
```

ml_pipeline/features/build_features_v2.py

```
from __future__ import annotations

import argparse
import logging
from pathlib import Path

import pandas as pd

from ml_pipeline.config import (
    LEAKAGE_FEATURES,
    NUMERIC_FEATURES_V2,
    TARGET_COLUMN,
    TRAINING_DATASET_PATH,
    TRAINING_DATASET_V2_PATH,
)
from ml_pipeline.utils import configure_logging, ensure_parent_dir, validate_columns


LOGGER = logging.getLogger(__name__)


REQUIRED_CONTEXT_COLUMNS = ["fecha", "time", "distrito", "ubigeo"]


def build_dataset_v2(source: pd.DataFrame) -> pd.DataFrame:
    validate_columns(
        source,
        [*REQUIRED_CONTEXT_COLUMNS, *NUMERIC_FEATURES_V2, TARGET_COLUMN],
        "frost_training_dataset.csv",
    )
    dataset = source[
        [*REQUIRED_CONTEXT_COLUMNS, *NUMERIC_FEATURES_V2, TARGET_COLUMN]
    ].copy()
    leaked = [column for column in LEAKAGE_FEATURES if column in dataset.columns]
    if leaked:
        raise ValueError(f"Dataset v2 still contains leakage columns: {leaked}")
    return dataset


def run(source_path: Path, output_path: Path) -> None:
    if not source_path.exists():
        raise FileNotFoundError(f"Source dataset not found: {source_path}")

    source = pd.read_csv(source_path)
    dataset = build_dataset_v2(source)
    ensure_parent_dir(output_path)
    dataset.to_csv(output_path, index=False)
    LOGGER.info("Dataset v2 source: %s", source_path)
    LOGGER.info("Dataset v2 output: %s", output_path)
    LOGGER.info("Dataset v2 shape: rows=%s columns=%s", len(dataset), len(dataset.columns))
    LOGGER.info("Dataset v2 features: %s", NUMERIC_FEATURES_V2)
    LOGGER.info("Dataset v2 class distribution: %s", dataset[TARGET_COLUMN].value_counts().to_dict())


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Build FrostPuno dataset v2 without leakage features.")
    parser.add_argument("--source", type=Path, default=TRAINING_DATASET_PATH)
    parser.add_argument("--output", type=Path, default=TRAINING_DATASET_V2_PATH)
    return parser.parse_args()


if __name__ == "__main__":
    configure_logging()
    args = parse_args()
    run(args.source, args.output)
```

ml_pipeline/preprocessing/validate_dataset.py

```python

from __future__ import annotations

import argparse
import logging
from pathlib import Path

import pandas as pd

from ml_pipeline.config import NUMERIC_FEATURES, TARGET_COLUMN, TRAINING_DATASET_PATH
from ml_pipeline.utils import configure_logging, validate_columns


LOGGER = logging.getLogger(__name__)
EXPECTED_TARGETS = {"bajo", "medio", "alto"}


def validate_training_dataset(path: Path) -> None:
    if not path.exists():
        raise FileNotFoundError(f"Training dataset not found: {path}")

    dataset = pd.read_csv(path)
    validate_columns(dataset, [*NUMERIC_FEATURES, TARGET_COLUMN], "frost_training_dataset.csv")

    if dataset.empty:
        raise ValueError("Training dataset is empty.")

    missing = dataset[NUMERIC_FEATURES].isna().sum()
    critical_missing = missing[missing > 0]
    if not critical_missing.empty:
        raise ValueError(f"Training dataset has missing feature values: {critical_missing.to_dict()}")

    targets = set(dataset[TARGET_COLUMN].dropna().unique())
    unexpected = targets - EXPECTED_TARGETS
    if unexpected:
        raise ValueError(f"Unexpected target classes: {unexpected}")

    impossible_ranges = {
        "relative_humidity_2m": dataset["relative_humidity_2m"].between(0, 100).all(),
        "cloud_cover": dataset["cloud_cover"].between(0, 100).all(),
        "wind_speed_10m": (dataset["wind_speed_10m"] >= 0).all(),
        "precipitation": (dataset["precipitation"] >= 0).all(),
    }
    failed = [name for name, ok in impossible_ranges.items() if not ok]
    if failed:
        raise ValueError(f"Climate variables outside expected ranges: {failed}")

    LOGGER.info("Dataset validation passed: %s rows, classes= %s", len(dataset), sorted(targets))


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Validate FrostPuno training dataset.")
    parser.add_argument("--dataset", type=Path, default=TRAINING_DATASET_PATH)
    return parser.parse_args()


if __name__ == "__main__":
    configure_logging()
    args = parse_args()
    validate_training_dataset(args.dataset)
```

ml_pipeline/registry/check_cluster_quality.py

```python
"""Quality gate NO SUPERVISADO: bloquea modelos de clustering bajo el umbral de silhouette.

Reemplaza a check_model_quality.py (f1_macro) en el path productivo.
"""

from __future__ import annotations

import argparse
import json
import os
import sys
from pathlib import Path


DEFAULT_METADATA_PATH = Path(__file__).resolve().parent / "cluster_metadata.json"


def load_silhouette(metadata_path: Path) -> float:
    if not metadata_path.exists():
        raise FileNotFoundError(f"Cluster metadata not found: {metadata_path}")

    metadata = json.loads(metadata_path.read_text(encoding="utf-8"))
    metrics = metadata.get("metrics")
    if not isinstance(metrics, dict):
        raise ValueError("cluster_metadata.json must contain a metrics object.")

    silhouette = metrics.get("silhouette")
    if silhouette is None:
        raise ValueError("cluster_metadata.json metrics must contain silhouette.")

    return float(silhouette)


def parse_threshold(cli_threshold: float | None) -> float:
    if cli_threshold is not None:
        return cli_threshold
    return float(os.getenv("MIN_SILHOUETTE", "0.35"))


def run(metadata_path: Path, min_silhouette: float) -> int:
    silhouette = load_silhouette(metadata_path)
    print(f"Cluster quality gate: silhouette={silhouette:.6f}, minimum={min_silhouette:.6f}")

    if silhouette < min_silhouette:
        print("QUALITY GATE FAILED: cluster silhouette is below the required threshold.")
        return 1

    print("QUALITY GATE PASSED: cluster silhouette meets the required threshold.")
    return 0

```

```python

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Validate FrostPuno clustering quality metrics.")
    parser.add_argument("--metadata", type=Path, default=DEFAULT_METADATA_PATH)
    parser.add_argument("--min-silhouette", type=float, default=None)
    return parser.parse_args()


if __name__ == "__main__":
    args = parse_args()
    try:
        sys.exit(run(args.metadata, parse_threshold(args.min_silhouette)))
    except Exception as exc: # noqa: BLE001 - CLI should print clear failure and exit non-zero.
        print(f"QUALITY GATE ERROR: {exc}", file=sys.stderr)
        sys.exit(1)
```

ml_pipeline/registry/check_model_quality.py

```python

from __future__ import annotations

import argparse
import json
import os
import sys
from pathlib import Path


DEFAULT_METADATA_PATH = Path(__file__).resolve().parent / "model_metadata.json"


def load_f1_macro(metadata_path: Path) -> float:
    if not metadata_path.exists():
        raise FileNotFoundError(f"Model metadata not found: {metadata_path}")

    metadata = json.loads(metadata_path.read_text(encoding="utf-8"))
    metrics = metadata.get("metrics")
    if not isinstance(metrics, dict):
        raise ValueError("model_metadata.json must contain a metrics object.")

    f1_macro = metrics.get("f1_macro")
    if f1_macro is None:
        raise ValueError("model_metadata.json metrics must contain f1_macro.")

    return float(f1_macro)


def parse_threshold(cli_threshold: float | None) -> float:
    if cli_threshold is not None:
        return cli_threshold
    return float(os.getenv("MIN_F1_MACRO", "0.70"))


def run(metadata_path: Path, min_f1_macro: float) -> int:
    f1_macro = load_f1_macro(metadata_path)
    print(f"Model quality gate: f1_macro={f1_macro:.6f}, minimum={min_f1_macro:.6f}")

    if f1_macro < min_f1_macro:
        print("QUALITY GATE FAILED: model f1_macro is below the required threshold.")
        return 1

    print("QUALITY GATE PASSED: model f1_macro meets the required threshold.")
    return 0


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Validate FrostPuno model quality metrics.")
    parser.add_argument("--metadata", type=Path, default=DEFAULT_METADATA_PATH)
    parser.add_argument("--min-f1-macro", type=float, default=None)
    return parser.parse_args()


if __name__ == "__main__":
    args = parse_args()
    try:
        sys.exit(run(args.metadata, parse_threshold(args.min_f1_macro)))
    except Exception as exc: # noqa: BLE001 - CLI should print clear failure and exit non-zero.
        print(f"QUALITY GATE ERROR: {exc}", file=sys.stderr)
        sys.exit(1)
```

ml_pipeline/tests/test_maintenance_ci.py

```python
"""Pruebas de funcionamiento del MANTENIMIENTO e INTEGRACIÓN CONTINUA.

Verifican que los flujos automatizados del elemento inteligente funcionan:

1. El reentrenamiento produce artefactos válidos y agrupa todos los distritos.
2. El quality gate APRUEBA el modelo productivo actual.
3. El quality gate BLOQUEA un modelo degradado (barrera real de la IC).
4. La metadata documenta los hiperparámetros optimizados (grid search).
"""

from __future__ import annotations

import json

import joblib
import pytest

from ml_pipeline.config import (
    CLUSTER_FEATURES,
    CLUSTER_METADATA_PATH,
    CLUSTER_MODEL_PATH,
    DISTRICT_CLUSTERS_PATH,
    RISK_TIERS,
)
from ml_pipeline.registry.check_cluster_quality import run as run_gate


@pytest.fixture(scope="module")
def metadata() -> dict:
```

```
    assert CLUSTER_METADATA_PATH.exists(), "Falta cluster_metadata.json: entrena el modelo primero."
    return json.loads(CLUSTER_METADATA_PATH.read_text(encoding="utf-8"))


def test_registry_artifacts_exist(metadata: dict) -> None:
    """El reentrenamiento deja modelo, metadata y agrupación de distritos."""
    assert CLUSTER_MODEL_PATH.exists()
    assert DISTRICT_CLUSTERS_PATH.exists()
    assert metadata["model_name"] == "KMeans"
    assert metadata["features"] == CLUSTER_FEATURES
    assert metadata["n_clusters"] >= 3


def test_model_loads_and_predicts(metadata: dict) -> None:
    """El artefacto serializado se puede cargar e inferir (contrato con el backend)."""
    import pandas as pd

    pipeline = joblib.load(CLUSTER_MODEL_PATH)
    sample = pd.DataFrame([{name: 0.0 for name in metadata["features"]}])
    cluster = int(pipeline.predict(sample)[0])
    assert 0 <= cluster < metadata["n_clusters"]


def test_district_groupings_cover_all_districts() -> None:
    """Cada distrito queda asignado a un tier de riesgo válido."""
    import pandas as pd

    districts = pd.read_csv(DISTRICT_CLUSTERS_PATH)
    assert len(districts) >= 13, "Se esperan al menos los 13 distritos del MVP."
    assert set(districts["tier"]).issubset(set(RISK_TIERS))
    assert districts["distrito"].is_unique


def test_hyperparameters_are_optimized(metadata: dict) -> None:
    """La metadata documenta la búsqueda de hiperparámetros (evidencia de optimización)."""
    search = metadata["hyperparameter_search"]
    assert search["criterion"] == "silhouette"
    assert search["combinations_evaluated"] >= 12
    best = metadata["best_params"]
    assert best["n_clusters"] == metadata["n_clusters"]
    assert best["init"] in search["grid"]["init"]
    # La configuración elegida es realmente la mejor del grid.
    best_row = max(search["results"], key=lambda r: r["silhouette"])
    assert best_row["k"] == best["n_clusters"]


def test_quality_gate_passes_for_production_model() -> None:
    """El gate aprueba el modelo actualmente en el registry."""
    assert run_gate(CLUSTER_METADATA_PATH, min_silhouette=0.35) == 0


def test_quality_gate_blocks_degraded_model(tmp_path, metadata: dict) -> None:
    """El gate DETIENE la integración si el modelo empeora (barrera de mantenimiento)."""
    degraded = dict(metadata)
    degraded["metrics"] = {"silhouette": 0.10, "davies_bouldin": 3.0, "inertia": 1.0}
    bad_path = tmp_path / "cluster_metadata.json"
    bad_path.write_text(json.dumps(degraded), encoding="utf-8")

    assert run_gate(bad_path, min_silhouette=0.35) == 1
```

ml_pipeline/tests/test_model_v2.py

```
from __future__ import annotations

import pandas as pd

from ml_pipeline.config import LEAKAGE_FEATURES, NUMERIC_FEATURES_V2, TARGET_COLUMN
from ml_pipeline.evaluation.evaluate_observed_events import load_observations
from ml_pipeline.features.build_features_v2 import build_dataset_v2
from ml_pipeline.training.train_models_v2 import split_dataset_v2


def _sample_dataset() -> pd.DataFrame:
    rows = []
    districts = ["A", "B", "C", "D"]
    for index, district in enumerate(districts):
        for hour in range(4):
            rows.append(
                {
                    "fecha": f"2024-06-0{hour + 1}",
                    "time": f"2024-06-0{hour + 1}T0{hour}:00",
                    "distrito": district,
                    "ubigeo": f"210{index}",
                    "latitud": -15.0 - index,
                    "longitud": -70.0 - index,
                    "altitud_estimada": 3800 + index,
                    "temperature_2m": float(hour - 2),
                    "relative_humidity_2m": 70.0,
                    "apparent_temperature": float(hour - 3),
                    "dew_point_2m": -2.0,
                    "precipitation": 0.0,
                    "cloud_cover": 20.0,
                    "wind_speed_10m": 7.0,
                    "mes": 6,
                    "hora": hour,
                    "temperatura_minima_diaria": -2.0,
                    "horas_bajo_cero": 3,
                    "riesgo_helada": "alto" if hour < 2 else "medio",
                }
            )
    return pd.DataFrame(rows)


def test_dataset_v2_removes_leakage_features() -> None:
    dataset = build_dataset_v2(_sample_dataset())

    for column in LEAKAGE_FEATURES:
        assert column not in dataset.columns
    assert TARGET_COLUMN in dataset.columns
    assert set(NUMERIC_FEATURES_V2).issubset(dataset.columns)


def test_district_split_keeps_districts_disjoint() -> None:
    dataset = build_dataset_v2(_sample_dataset())

    x_train, x_test, y_train, y_test, metadata = split_dataset_v2(
```

```
        dataset,
        strategy="district",
        test_size=0.5,
    )

    assert not x_train.empty
    assert not x_test.empty
    assert not y_train.empty
    assert not y_test.empty
    assert set(metadata["train_districts"]).isdisjoint(metadata["test_districts"])


def test_time_split_uses_recent_dates_for_test() -> None:
    dataset = build_dataset_v2(_sample_dataset())

    _, _, y_train, y_test, metadata = split_dataset_v2(
        dataset,
        strategy="time",
        test_size=0.25,
    )

    assert not y_train.empty
    assert not y_test.empty
    assert metadata["strategy"] == "time"
    assert metadata["train_end"] < metadata["test_start"]


def test_observation_validation_adds_time_features(tmp_path) -> None:
    observations_path = tmp_path / "observations.csv"
    pd.DataFrame(
        [
            {
                "observed_at": "2024-06-01T03:00:00",
                "distrito": "Puno",
                "latitud": -15.8402,
                "longitud": -70.0219,
                "altitud_estimada": 3827,
                "temperature_2m": -2.1,
                "relative_humidity_2m": 71,
                "apparent_temperature": -3.8,
                "dew_point_2m": -4.0,
                "precipitation": 0,
                "cloud_cover": 18,
                "wind_speed_10m": 8,
                "riesgo_helada_observado": "alto",
                "fuente_observacion": "SENAMHI_sample",
            }
        ]
    ).to_csv(observations_path, index=False)

    observations = load_observations(observations_path)

    assert set(NUMERIC_FEATURES_V2).issubset(observations.columns)
    assert observations.loc[0, "mes"] == 6
    assert observations.loc[0, "hora"] == 3
```

ml_pipeline/training/train_models.py

```

"""LEGACY — clasificador supervisado (previo migrado).

El path productivo del curso es el modelo NO SUPERVISADO en
``ml_pipeline/clustering/train_clusters.py`` (K-Means, métrica silhouette).
Este script se conserva solo como referencia histórica para el informe
("modelo previo migrado"); no forma parte de la IC ni del backend.
"""

from __future__ import annotations

import argparse
import logging
from pathlib import Path
from typing import Any

import joblib
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from ml_pipeline.config import (
    METRICS_COMPARISON_PATH,
    MODEL_METADATA_PATH,
    MODEL_PATH,
    NUMERIC_FEATURES,
    TARGET_COLUMN,
    TRAINING_DATASET_PATH,
)
from ml_pipeline.utils import configure_logging, ensure_parent_dir, utc_now_iso, validate_columns, write_json


LOGGER = logging.getLogger(__name__)
RANDOM_STATE = 42


def load_dataset(path: Path) -> pd.DataFrame:
    if not path.exists():
        raise FileNotFoundError(f"Training dataset not found: {path}")

    dataset = pd.read_csv(path)
    validate_columns(dataset, [*NUMERIC_FEATURES, TARGET_COLUMN], "frost_training_dataset.csv")
    return dataset.dropna(subset=[*NUMERIC_FEATURES, TARGET_COLUMN])


def split_dataset(dataset: pd.DataFrame, test_size: float) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series]:
    x = dataset[NUMERIC_FEATURES]
    y = dataset[TARGET_COLUMN]
    class_counts = y.value_counts()
    stratify = y if len(class_counts) > 1 and class_counts.min() >= 2 else None
    return train_test_split(x, y, test_size=test_size, random_state=RANDOM_STATE, stratify=stratify)
```

```python


def build_preprocessor(scale: bool) -> ColumnTransformer:
    steps: list[tuple[str, Any]] = [("imputer", SimpleImputer(strategy="median"))]
    if scale:
        steps.append(("scaler", StandardScaler()))
    numeric_pipeline = Pipeline(steps=steps)
    return ColumnTransformer(transformers=[("numeric", numeric_pipeline, NUMERIC_FEATURES)])


def candidate_models() -> dict[str, Pipeline]:
    return {
        "LogisticRegression": Pipeline(
            steps=[
                ("preprocess", build_preprocessor(scale=True)),
                (
                    "model",
                    LogisticRegression(max_iter=1000, class_weight="balanced", random_state=RANDOM_STATE),
                ),
            ]
        ),
        "DecisionTreeClassifier": Pipeline(
            steps=[
                ("preprocess", build_preprocessor(scale=False)),
                (
                    "model",
                    DecisionTreeClassifier(max_depth=6, class_weight="balanced", random_state=RANDOM_STATE),
                ),
            ]
        ),
        "RandomForestClassifier": Pipeline(
            steps=[
                ("preprocess", build_preprocessor(scale=False)),
                (
                    "model",
                    RandomForestClassifier(
                        n_estimators=150,
                        max_depth=10,
                        class_weight="balanced",
                        random_state=RANDOM_STATE,
                        n_jobs=-1,
                    ),
                ),
            ]
        ),
    }


def evaluate_predictions(y_true: pd.Series, y_pred: pd.Series | list[str]) -> dict[str, float]:
    return {
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "precision_macro": float(precision_score(y_true, y_pred, average="macro", zero_division=0)),
        "recall_macro": float(recall_score(y_true, y_pred, average="macro", zero_division=0)),
        "f1_macro": float(f1_score(y_true, y_pred, average="macro", zero_division=0)),
    }


def train_and_compare(dataset: pd.DataFrame, test_size: float) -> tuple[str, Pipeline, dict[str, dict[str, float]]]:
    x_train, x_test, y_train, y_test = split_dataset(dataset, test_size=test_size)
    metrics: dict[str, dict[str, float]] = {}
    trained_models: dict[str, Pipeline] = {}

    for name, model in candidate_models().items():
        LOGGER.info("Training %s", name)
        model.fit(x_train, y_train)
        predictions = model.predict(x_test)
        metrics[name] = evaluate_predictions(y_test, predictions)
        trained_models[name] = model
        LOGGER.info("%s metrics: %s", name, metrics[name])

    tie_break_priority = {
        "RandomForestClassifier": 3,
        "DecisionTreeClassifier": 2,
        "LogisticRegression": 1,
    }
    best_model_name = max(
        metrics,
        key=lambda model_name: (metrics[model_name]["f1_macro"], tie_break_priority.get(model_name, 0)),
    )
    return best_model_name, trained_models[best_model_name], metrics


def save_registry(
    model_name: str,
    model: Pipeline,
    metrics: dict[str, dict[str, float]],
    dataset: pd.DataFrame,
    version: str,
) -> None:
    ensure_parent_dir(MODEL_PATH)
    joblib.dump(model, MODEL_PATH)
    write_json(METRICS_COMPARISON_PATH, metrics)

    metadata = {
        "model_name": model_name,
        "version": version,
        "created_at": utc_now_iso(),
        "features": NUMERIC_FEATURES,
        "target": TARGET_COLUMN,
        "metrics": metrics[model_name],
        "all_model_metrics": metrics,
        "dataset_size": int(len(dataset)),
        "data_sources": [
            "Open-Meteo Historical API",
            "INEI-compatible curated district seed for Puno MVP",
            "SENAMHI documented for validation in phase 2",
            "MIDAGRI/SIEA documented as agrarian enrichment in phase 2",
        ],
        "limitations": [
            "Initial labels are rule-based and derived from temperature thresholds.",
            "The curated INEI district file is an MVP seed and must be replaced with official exports for production.",
            "Daily minimum temperature and hours below zero are label-derived features, so baseline metrics can be optimistic.",
            "SENAMHI station observations are not yet used as ground-truth labels.",
        ],
    }
    write_json(MODEL_METADATA_PATH, metadata)
    LOGGER.info("Saved best model to %s and metadata to %s", MODEL_PATH, MODEL_METADATA_PATH)


def run(dataset_path: Path, version: str, test_size: float) -> None:
    dataset = load_dataset(dataset_path)
```

```
    best_model_name, best_model, metrics = train_and_compare(dataset, test_size=test_size)
    save_registry(best_model_name, best_model, metrics, dataset, version=version)


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Train and compare FrostPuno ML models.")
    parser.add_argument("--dataset", type=Path, default=TRAINING_DATASET_PATH)
    parser.add_argument("--version", default="v0.1.0")
    parser.add_argument("--test-size", type=float, default=0.25)
    return parser.parse_args()


if __name__ == "__main__":
    configure_logging()
    args = parse_args()
    run(args.dataset, args.version, args.test_size)
```

ml_pipeline/training/train_models_v2.py

```
from __future__ import annotations

import argparse
import logging
from pathlib import Path
from typing import Any, Literal

import joblib
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

from ml_pipeline.config import (
    CONFUSION_MATRIX_V2_PATH,
    LEAKAGE_FEATURES,
    METRICS_COMPARISON_V2_PATH,
    MODEL_METADATA_PATH,
    MODEL_METADATA_V2_PATH,
    MODEL_PATH,
    MODEL_V2_PATH,
    NUMERIC_FEATURES_V2,
    PROJECT_ROOT,
    TARGET_COLUMN,
    TRAINING_DATASET_PATH,
    TRAINING_DATASET_V2_PATH,
)
from ml_pipeline.features.build_features_v2 import build_dataset_v2
from ml_pipeline.training.train_models import RANDOM_STATE, evaluate_predictions
from ml_pipeline.utils import configure_logging, ensure_parent_dir, utc_now_iso, validate_columns, write_json


LOGGER = logging.getLogger(__name__)
LABEL_ORDER = ["bajo", "medio", "alto"]
SplitStrategy = Literal["district", "time"]


def repo_relative(path: Path) -> str:
    return path.relative_to(PROJECT_ROOT).as_posix()


def load_or_create_dataset_v2(source_path: Path, dataset_path: Path) -> pd.DataFrame:
    if dataset_path.exists():
        dataset = pd.read_csv(dataset_path)
    else:
        if not source_path.exists():
            raise FileNotFoundError(f"Source dataset not found: {source_path}")
        dataset = build_dataset_v2(pd.read_csv(source_path))
        ensure_parent_dir(dataset_path)
        dataset.to_csv(dataset_path, index=False)

    validate_columns(
        dataset,
        ["fecha", "time", "distrito", "ubigeo", *NUMERIC_FEATURES_V2, TARGET_COLUMN],
        dataset_path.name,
    )
    leaked = [column for column in LEAKAGE_FEATURES if column in dataset.columns]
    if leaked:
        raise ValueError(f"Dataset v2 contains leakage columns: {leaked}")
    return dataset.dropna(subset=[*NUMERIC_FEATURES_V2, TARGET_COLUMN])


def split_by_district(
    dataset: pd.DataFrame,
    test_size: float,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series, dict[str, Any]]:
    districts = dataset["distrito"].dropna().drop_duplicates().sample(
        frac=1,
        random_state=RANDOM_STATE,
    )
    test_count = max(1, round(len(districts) * test_size))
    test_districts = set(districts.tail(test_count))
    train_districts = set(districts) - test_districts
    train = dataset[dataset["distrito"].isin(train_districts)].copy()
    test = dataset[dataset["distrito"].isin(test_districts)].copy()
    if train.empty or test.empty:
        raise ValueError("District split produced an empty train or test partition.")

    return (
        train[NUMERIC_FEATURES_V2],
        test[NUMERIC_FEATURES_V2],
        train[TARGET_COLUMN],
        test[TARGET_COLUMN],
        {
            "strategy": "district",
            "test_size": test_size,
            "train_districts": sorted(train_districts),
            "test_districts": sorted(test_districts),
            "train_rows": int(len(train)),
            "test_rows": int(len(test)),
        },
    )

```

```python

def split_by_time(
    dataset: pd.DataFrame,
    test_size: float,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series, dict[str, Any]]:
    dated = dataset.copy()
    dated["datetime"] = pd.to_datetime(dated["time"], errors="coerce")
    if dated["datetime"].isna().any():
        raise ValueError("Dataset v2 contains invalid timestamps.")
    ordered_dates = dated["datetime"].dt.date.drop_duplicates().sort_values()
    test_count = max(1, round(len(ordered_dates) * test_size))
    cutoff = set(ordered_dates.tail(test_count))
    train = dated[~dated["datetime"].dt.date.isin(cutoff)].copy()
    test = dated[dated["datetime"].dt.date.isin(cutoff)].copy()
    if train.empty or test.empty:
        raise ValueError("Time split produced an empty train or test partition.")

    return (
        train[NUMERIC_FEATURES_V2],
        test[NUMERIC_FEATURES_V2],
        train[TARGET_COLUMN],
        test[TARGET_COLUMN],
        {
            "strategy": "time",
            "test_size": test_size,
            "train_start": str(train["datetime"].min()),
            "train_end": str(train["datetime"].max()),
            "test_start": str(test["datetime"].min()),
            "test_end": str(test["datetime"].max()),
            "train_rows": int(len(train)),
            "test_rows": int(len(test)),
        },
    )


def split_dataset_v2(
    dataset: pd.DataFrame,
    strategy: SplitStrategy,
    test_size: float,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series, dict[str, Any]]:
    if strategy == "district":
        return split_by_district(dataset, test_size)
    if strategy == "time":
        return split_by_time(dataset, test_size)
    raise ValueError(f"Unsupported split strategy: {strategy}")


def build_preprocessor(scale: bool) -> ColumnTransformer:
    steps: list[tuple[str, Any]] = [("imputer", SimpleImputer(strategy="median"))]
    if scale:
        steps.append(("scaler", StandardScaler()))
    numeric_pipeline = Pipeline(steps=steps)
    return ColumnTransformer(transformers=[("numeric", numeric_pipeline, NUMERIC_FEATURES_V2)])


def candidate_models() -> dict[str, Pipeline]:
    return {
        "LogisticRegression": Pipeline(
            steps=[
                ("preprocess", build_preprocessor(scale=True)),
                (
                    "model",
                    LogisticRegression(max_iter=1000, class_weight="balanced", random_state=RANDOM_STATE),
                ),
            ]
        ),
        "RandomForestClassifier": Pipeline(
            steps=[
                ("preprocess", build_preprocessor(scale=False)),
                (
                    "model",
                    RandomForestClassifier(
                        n_estimators=150,
                        max_depth=10,
                        class_weight="balanced",
                        random_state=RANDOM_STATE,
                        n_jobs=-1,
                    ),
                ),
            ]
        ),
    }


def train_and_compare_v2(
    dataset: pd.DataFrame,
    strategy: SplitStrategy,
    test_size: float,
) -> tuple[str, Pipeline, dict[str, dict[str, float]], pd.Series, list[str], dict[str, Any]]:
    x_train, x_test, y_train, y_test, split_metadata = split_dataset_v2(dataset, strategy, test_size)
    LOGGER.info("Dataset v2 rows= %s", len(dataset))
    LOGGER.info("Dataset v2 features= %s", NUMERIC_FEATURES_V2)
    LOGGER.info("Split strategy= %s metadata= %s", strategy, split_metadata)

    metrics: dict[str, dict[str, float]] = {}
    trained_models: dict[str, Pipeline] = {}
    predictions_by_model: dict[str, list[str]] = {}
    for name, model in candidate_models().items():
        LOGGER.info("Training v0.2.0 candidate  %s", name)
        model.fit(x_train, y_train)
        predictions = model.predict(x_test)
        predictions_by_model[name] = [str(item) for item in predictions]
        metrics[name] = evaluate_predictions(y_test, predictions)
        trained_models[name] = model
        LOGGER.info(" %s v0.2.0 metrics:  %s", name, metrics[name])

    best_model_name = "RandomForestClassifier"
    return (
        best_model_name,
        trained_models[best_model_name],
        metrics,
        y_test,
        predictions_by_model[best_model_name],
        split_metadata,
    )


def save_registry_v2(
    model_name: str,
    model: Pipeline,
    metrics: dict[str, dict[str, float]],
```

```
    y_test: pd.Series,
    predictions: list[str],
    dataset: pd.DataFrame,
    version: str,
    split_metadata: dict[str, Any],
) -> None:
    ensure_parent_dir(MODEL_V2_PATH)
    joblib.dump(model, MODEL_V2_PATH)
    write_json(METRICS_COMPARISON_V2_PATH, metrics)

    matrix = confusion_matrix(y_test, predictions, labels=LABEL_ORDER)
    ensure_parent_dir(CONFUSION_MATRIX_V2_PATH)
    pd.DataFrame(matrix, index=LABEL_ORDER, columns=LABEL_ORDER).to_csv(CONFUSION_MATRIX_V2_PATH)

    metadata = {
        "model_name": model_name,
        "version": version,
        "created_at": utc_now_iso(),
        "status": "experimental",
        "production_model_unchanged": True,
        "production_model_path": repo_relative(MODEL_PATH),
        "production_metadata_path": repo_relative(MODEL_METADATA_PATH),
        "model_path": repo_relative(MODEL_V2_PATH),
        "dataset_path": repo_relative(TRAINING_DATASET_V2_PATH),
        "features": NUMERIC_FEATURES_V2,
        "removed_leakage_features": LEAKAGE_FEATURES,
        "target": TARGET_COLUMN,
        "split": split_metadata,
        "metrics": metrics[model_name],
        "all_model_metrics": metrics,
        "selection_reason": "RandomForestClassifier is kept as the v0.2.0 baseline model; LogisticRegression is reported only for comparison.",
        "label_order": LABEL_ORDER,
        "confusion_matrix_path": repo_relative(CONFUSION_MATRIX_V2_PATH),
        "dataset_size": int(len(dataset)),
        "data_sources": [
            "Open-Meteo Historical API",
            "INEI-compatible curated district seed for Puno MVP",
        ],
        "limitations": [
            "Labels remain rule-based and are not official observed frost events.",
            "Leakage-derived features were removed from model inputs.",
            "District split estimates generalization to unseen districts, but still requires SENAMHI/campo ground truth.",
            "This model is experimental and is not used by FastAPI production endpoints.",
        ],
    }
    write_json(MODEL_METADATA_V2_PATH, metadata)
    LOGGER.info("Saved v0.2.0 model to %s", MODEL_V2_PATH)
    LOGGER.info("Saved v0.2.0 metadata to %s", MODEL_METADATA_V2_PATH)
    LOGGER.info("Saved v0.2.0 confusion matrix to %s", CONFUSION_MATRIX_V2_PATH)


def run(
    source_path: Path,
    dataset_path: Path,
    version: str,
    strategy: SplitStrategy,
    test_size: float,
) -> None:
    dataset = load_or_create_dataset_v2(source_path, dataset_path)
    best_model_name, best_model, metrics, y_test, predictions, split_metadata = train_and_compare_v2(
        dataset,
        strategy,
        test_size,
    )
    save_registry_v2(best_model_name, best_model, metrics, y_test, predictions, dataset, version, split_metadata)


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Train FrostPuno v0.2.0 without leakage features.")
    parser.add_argument("--source", type=Path, default=TRAINING_DATASET_PATH)
    parser.add_argument("--dataset", type=Path, default=TRAINING_DATASET_V2_PATH)
    parser.add_argument("--version", default="v0.2.0")
    parser.add_argument("--split-strategy", choices=["district", "time"], default="district")
    parser.add_argument("--test-size", type=float, default=0.30)
    return parser.parse_args()


if __name__ == "__main__":
    configure_logging()
    args = parse_args()
    run(args.source, args.dataset, args.version, args.split_strategy, args.test_size)
```

ml_pipeline/utils.py

```
from __future__ import annotations

import json
import logging
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Iterable

import pandas as pd


def configure_logging() -> None:
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
    )


def ensure_parent_dir(path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)


def validate_columns(frame: pd.DataFrame, required_columns: Iterable[str], dataset_name: str) -> None:
    missing = [column for column in required_columns if column not in frame.columns]
    if missing:
        raise ValueError(f"{dataset_name} is missing required columns: {missing}")


def write_json(path: Path, payload: dict[str, Any]) -> None:
    ensure_parent_dir(path)
    path.write_text(json.dumps(payload, indent=2, ensure_ascii=False), encoding="utf-8")

```

```

def utc_now_iso() -> str:
    return datetime.now(timezone.utc).replace(microsecond=0).isoformat()
```

2. Servicio backend

backend_fastapi/app/___init___.py

```

"""FrostPuno FastAPI backend."""
```

backend_fastapi/app/api/___init___.py

```
"""HTTP API package."""
```

backend_fastapi/app/api/routes/___init___.py

```
"""Route modules."""
```

backend_fastapi/app/api/routes/alerts.py

```
from __future__ import annotations

from datetime import date, timedelta

from fastapi import APIRouter, Depends, Query

from app.api.routes.weather import get_weather_provider
from app.schemas.alerts import ClimateAnomaly, DailyAlertResponse, LivestockRisk
from app.services.alert_service import anomaly, frost_severity, livestock_risk
from app.services.model_registry import ModelRegistry, get_model_registry
from app.services.weather_providers import WeatherQuery

router = APIRouter(prefix="/alerts", tags=["alerts"])


@router.get("/today", response_model=DailyAlertResponse)
def alert_today(
    latitude: float = Query(..., ge=-18.5, le=-13.0),
    longitude: float = Query(..., ge=-71.5, le=-68.0),
    registry: ModelRegistry = Depends(get_model_registry),
) -> DailyAlertResponse:
    provider = get_weather_provider()
    query = WeatherQuery(latitude=latitude, longitude=longitude)

    forecast = provider.get_daily_forecast(query, days=1)
    today = forecast[0] if forecast else {}
    temp_min = today.get("temperature_2m_min")
    day = today.get("date")

    # Climatología reciente para detectar anomalías (últimos ~30 días con retraso de archivo).
    history_rows: list[dict] = []
    try:
        end = date.today() - timedelta(days=6)
        start = end - timedelta(days=29)
        history_rows = provider.get_history(query, start.isoformat(), end.isoformat())
    except Exception: # noqa: BLE001 - la alerta no debe romperse si el histórico falla.
        history_rows = []

    risk_level, severity, alert, title, message = frost_severity(temp_min)
    livestock = livestock_risk(temp_min)
    anomaly_data = anomaly(temp_min, history_rows)

    return DailyAlertResponse(
        latitude=latitude,
        longitude=longitude,
        date=str(day) if day is not None else None,
        risk_level=risk_level,
        frost_alert=alert,
        severity=severity,
        temperature_min=temp_min,
        title=title,
        message=message,
        livestock=LivestockRisk(**livestock),
        anomaly=ClimateAnomaly(**anomaly_data),
        model_version=registry.metadata.version,
    )
```

backend_fastapi/app/api/routes/chuno.py

```
from __future__ import annotations

from datetime import datetime, timezone

from fastapi import APIRouter, Query

from app.api.routes.weather import get_weather_provider
from app.schemas.chuno import ChunoWindowResponse
from app.services.chuno_service import evaluate_window
from app.services.weather_providers import WeatherQuery

router = APIRouter(prefix="/chuno", tags=["chuno"])

```

```python

@router.get("/window", response_model=ChunoWindowResponse)
def chuno_window(
    latitude: float = Query(..., ge=-18.5, le=-13.0),
    longitude: float = Query(..., ge=-71.5, le=-68.0),
    days: int = Query(7, ge=1, le=16),
) -> ChunoWindowResponse:
    provider = get_weather_provider()
    forecast = provider.get_daily_forecast(WeatherQuery(latitude=latitude, longitude=longitude), days=days)
    current_month = datetime.now(timezone.utc).month
    return evaluate_window(latitude, longitude, current_month, forecast)
```

backend_fastapi/app/api/routes/health.py

```python
from __future__ import annotations

from fastapi import APIRouter

from app.core.config import settings
from app.schemas.health import HealthResponse


router = APIRouter(tags=["health"])


@router.get("/health", response_model=HealthResponse)
def health_check() -> HealthResponse:
    model_exists = settings.model_path.exists()
    metadata_exists = settings.model_metadata_path.exists()
    return HealthResponse(
        status="ok" if model_exists and metadata_exists else "degraded",
        app_name=settings.app_name,
        version=settings.api_version,
        model_available=model_exists and metadata_exists,
    )
```

backend_fastapi/app/api/routes/ml.py

```python
from __future__ import annotations

import csv

from fastapi import APIRouter, Depends

from app.core.config import settings
from app.schemas.ml import (
    ClusterProfile,
    ClustersResponse,
    DistrictCluster,
    ModelInfoResponse,
)
from app.services.model_registry import ModelRegistry, get_model_registry


router = APIRouter(prefix="/ml", tags=["ml"])


@router.get("/model-info", response_model=ModelInfoResponse)
def model_info(registry: ModelRegistry = Depends(get_model_registry)) -> ModelInfoResponse:
    metadata = registry.metadata
    return ModelInfoResponse(
        model_name=metadata.model_name,
        version=metadata.version,
        created_at=metadata.created_at,
        features=metadata.features,
        target=metadata.target,
        metrics=metadata.metrics,
        dataset_size=metadata.dataset_size,
        data_sources=metadata.data_sources,
        limitations=metadata.limitations,
        n_clusters=metadata.n_clusters,
    )


def _load_districts() -> list[DistrictCluster]:
    path = settings.district_clusters_path
    if not path.exists():
        return []
    districts: list[DistrictCluster] = []
    with path.open(encoding="utf-8", newline="") as handle:
        for row in csv.DictReader(handle):
            districts.append(
                DistrictCluster(
                    distrito=row["distrito"],
                    tier=row["tier"],
                    dominant_cluster=int(float(row["dominant_cluster"])),
                    cold_share=float(row["cold_share"]),
                    altitud_estimada=float(row["altitud_estimada"]),
                    temperature_2m_mean=float(row["temperature_2m_mean"]),
                    latitud=float(row["latitud"]) if row.get("latitud") else None,
                    longitud=float(row["longitud"]) if row.get("longitud") else None,
                )
            )
    return districts


@router.get("/clusters", response_model=ClustersResponse)
def clusters(registry: ModelRegistry = Depends(get_model_registry)) -> ClustersResponse:
    metadata = registry.metadata
    profiles_raw = metadata.cluster_profiles or {}
    profiles = [
        ClusterProfile(
            cluster_id=int(cid),
            tier=str(profile.get("tier", "medio")),
            size=int(profile.get("size", 0)),
            temperature_2m=float(profile.get("temperature_2m", 0.0)),
            dew_point_2m=float(profile.get("dew_point_2m", 0.0)),
        )
        for cid, profile in profiles_raw.items()
    ]
    profiles.sort(key=lambda p: p.temperature_2m)
```

```
    return ClustersResponse(
        model_name=metadata.model_name,
        version=metadata.version,
        n_clusters=metadata.n_clusters or len(profiles),
        silhouette=float(metadata.metrics.get("silhouette", 0.0)),
        profiles=profiles,
        districts=_load_districts(),
    )
```

backend_fastapi/app/api/routes/predict.py

```
from __future__ import annotations

from fastapi import APIRouter, Depends

from app.repositories.predictions import PredictionRepository, get_prediction_repository
from app.schemas.prediction import FrostRiskRequest, FrostRiskResponse
from app.services.prediction_service import PredictionService, get_prediction_service


router = APIRouter(prefix="/predict", tags=["prediction"])


@router.post("/frost-risk", response_model=FrostRiskResponse)
def predict_frost_risk(
    payload: FrostRiskRequest,
    service: PredictionService = Depends(get_prediction_service),
    repository: PredictionRepository = Depends(get_prediction_repository),
) -> FrostRiskResponse:
    prediction = service.predict(payload)
    repository.save_prediction(payload, prediction)
    return prediction
```

backend_fastapi/app/api/routes/predictions.py

```
from __future__ import annotations

from fastapi import APIRouter, Depends, Query

from app.repositories.predictions import PredictionRepository, get_prediction_repository
from app.schemas.prediction import FrostPredictionHistoryItem


router = APIRouter(prefix="/predictions", tags=["predictions"])


@router.get("/history", response_model=list[FrostPredictionHistoryItem])
def prediction_history(
    limit: int = Query(default=20, ge=1, le=100),
    repository: PredictionRepository = Depends(get_prediction_repository),
) -> list[FrostPredictionHistoryItem]:
    return repository.list_history(limit=limit)
```

backend_fastapi/app/api/routes/weather.py

```
from __future__ import annotations

from functools import lru_cache

from fastapi import APIRouter, Query

from app.schemas.weather import CurrentWeatherResponse, HistoryDay, WeatherHistoryResponse
from app.services.weather_providers import HybridWeatherProvider, WeatherQuery


router = APIRouter(prefix="/weather", tags=["weather"])


@lru_cache(maxsize=1)
def get_weather_provider() -> HybridWeatherProvider:
    return HybridWeatherProvider()


@router.get("/current", response_model=CurrentWeatherResponse)
def current_weather(
    latitude: float = Query(..., ge=-18.5, le=-13.0),
    longitude: float = Query(..., ge=-71.5, le=-68.0),
) -> CurrentWeatherResponse:
    return get_weather_provider().get_current_weather(
        WeatherQuery(latitude=latitude, longitude=longitude),
    )


@router.get("/history", response_model=WeatherHistoryResponse)
def weather_history(
    latitude: float = Query(..., ge=-18.5, le=-13.0),
    longitude: float = Query(..., ge=-71.5, le=-68.0),
    start: str = Query(..., description="Fecha inicio YYYY-MM-DD"),
    end: str = Query(..., description="Fecha fin YYYY-MM-DD"),
) -> WeatherHistoryResponse:
    rows = get_weather_provider().get_history(
        WeatherQuery(latitude=latitude, longitude=longitude), start, end
    )
    days = [
        HistoryDay(
            date=str(row.get("date")),
            temperature_min=row.get("temperature_2m_min"),
            temperature_max=row.get("temperature_2m_max"),
            humidity_mean=row.get("relative_humidity_2m_mean"),
        )
        for row in rows
    ]
    return WeatherHistoryResponse(
        latitude=latitude,
        longitude=longitude,
        start_date=start,
```

```
        end_date=end,
        days=days,
    )
```

backend_fastapi/app/core/___init___.py

```
"""Core backend settings and exceptions."""
```

backend_fastapi/app/core/config.py

```

from __future__ import annotations

from pathlib import Path

from pydantic import AliasChoices, Field
from pydantic_settings import BaseSettings, SettingsConfigDict


PROJECT_ROOT = Path(__file__).resolve().parents[3]


class Settings(BaseSettings):
    app_name: str = Field(default="FrostPuno API", validation_alias=AliasChoices("APP_NAME", "FROST_PUNO_APP_NAME"))
    api_version: str = Field(default="0.1.0", validation_alias=AliasChoices("API_VERSION", "FROST_PUNO_API_VERSION"))
    log_level: str = Field(default="INFO", validation_alias=AliasChoices("LOG_LEVEL", "FROST_PUNO_LOG_LEVEL"))

    # Modelo productivo: K-Means no supervisado (reemplaza al clasificador supervisado).
    model_path: Path = Field(
        default=PROJECT_ROOT / "ml_pipeline" / "registry" / "frost_cluster_model.joblib",
        validation_alias=AliasChoices("MODEL_PATH", "FROST_PUNO_MODEL_PATH"),
    )
    model_metadata_path: Path = Field(
        default=PROJECT_ROOT / "ml_pipeline" / "registry" / "cluster_metadata.json",
        validation_alias=AliasChoices("MODEL_METADATA_PATH", "FROST_PUNO_MODEL_METADATA_PATH"),
    )
    district_clusters_path: Path = Field(
        default=PROJECT_ROOT / "data" / "processed" / "district_clusters.csv",
        validation_alias=AliasChoices("DISTRICT_CLUSTERS_PATH", "FROST_PUNO_DISTRICT_CLUSTERS_PATH"),
    )

    supabase_url: str | None = Field(default=None, validation_alias=AliasChoices("SUPABASE_URL", "FROST_PUNO_SUPABASE_URL"))
    supabase_service_role_key: str | None = Field(
        default=None,
        validation_alias=AliasChoices("SUPABASE_SERVICE_ROLE_KEY", "FROST_PUNO_SUPABASE_SERVICE_ROLE_KEY"),
    )
    enable_supabase: bool = Field(default=False, validation_alias=AliasChoices("ENABLE_SUPABASE", "FROST_PUNO_PERSIST_PREDICTIONS"))
    cors_allowed_origins: str = Field(
        default="http://127.0.0.1:5174,http://127.0.0.1:5173,http://localhost:5174,http://localhost:5173",
        validation_alias=AliasChoices("CORS_ORIGINS", "CORS_ALLOWED_ORIGINS", "FROST_PUNO_CORS_ALLOWED_ORIGINS"),
    )

    @property
    def cors_origins(self) -> list[str]:
        return [origin.strip() for origin in self.cors_allowed_origins.split(",") if origin.strip()]

    model_config = SettingsConfigDict(
        env_file=(PROJECT_ROOT / ".env", PROJECT_ROOT / "backend_fastapi" / ".env"),
        extra="ignore",
    )


settings = Settings()
```

backend_fastapi/app/core/exceptions.py

```
class ModelLoadError(RuntimeError):
    """Raised when the model registry cannot be loaded."""


class PredictionError(RuntimeError):
    """Raised when a validated request cannot be transformed into a prediction."""
```

backend_fastapi/app/main.py

```

from __future__ import annotations

import logging

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from fastapi.middleware.cors import CORSMiddleware

from app.api.routes import alerts, chuno, health, ml, predict, predictions, weather
from app.core.config import settings
from app.core.exceptions import ModelLoadError, PredictionError


logging.basicConfig(
    level=settings.log_level,
    format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
)


def create_app() -> FastAPI:
    app = FastAPI(
        title=settings.app_name,
        version=settings.api_version,
        description="Backend MVP for FrostPuno frost-risk prediction.",
    )
```

```
    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.cors_origins,
        allow_credentials=False,
        allow_methods=["GET", "POST"],
        allow_headers=["Content-Type"],
    )

    @app.exception_handler(ModelLoadError)
    async def model_load_error_handler(_: Request, exc: ModelLoadError) -> JSONResponse:
        return JSONResponse(
            status_code=503,
            content={
                "error": {
                    "code": "MODEL_UNAVAILABLE",
                    "message": str(exc),
                }
            },
        )

    @app.exception_handler(PredictionError)
    async def prediction_error_handler(_: Request, exc: PredictionError) -> JSONResponse:
        return JSONResponse(
            status_code=422,
            content={
                "error": {
                    "code": "PREDICTION_ERROR",
                    "message": str(exc),
                }
            },
        )

    app.include_router(health.router)
    app.include_router(ml.router)
    app.include_router(predict.router)
    app.include_router(predictions.router)
    app.include_router(weather.router)
    app.include_router(chuno.router)
    app.include_router(alerts.router)
    return app


app = create_app()
```

backend_fastapi/app/repositories/___init___.py

```
"""Persistence adapters."""
```

backend_fastapi/app/repositories/predictions.py

```
from __future__ import annotations

import logging
from functools import lru_cache
from typing import Protocol

from app.core.config import settings
from app.schemas.prediction import FrostPredictionHistoryItem, FrostRiskRequest, FrostRiskResponse


LOGGER = logging.getLogger(__name__)


class PredictionRepository(Protocol):
    def save_prediction(self, request: FrostRiskRequest, response: FrostRiskResponse) -> None:
        """Persist a prediction when a backing store is configured."""

    def list_history(self, limit: int = 20) -> list[FrostPredictionHistoryItem]:
        """Return recent predictions."""


class NoOpPredictionRepository:
    def save_prediction(self, request: FrostRiskRequest, response: FrostRiskResponse) -> None:
        LOGGER.info(
            "Prediction persistence disabled. district=%s risk=%s model=%s",
            request.district,
            response.risk_level,
            response.model_version,
        )

    def list_history(self, limit: int = 20) -> list[FrostPredictionHistoryItem]:
        LOGGER.info("Prediction history requested while persistence is disabled. limit=%s", limit)
        return []


@lru_cache(maxsize=1)
def get_prediction_repository() -> PredictionRepository:
    if settings.enable_supabase:
        if settings.supabase_url and settings.supabase_service_role_key:
            from app.repositories.supabase_prediction_repository import SupabasePredictionRepository

            return SupabasePredictionRepository(
                supabase_url=settings.supabase_url,
                service_role_key=settings.supabase_service_role_key,
            )
        LOGGER.warning("ENABLE_SUPABASE=true but SUPABASE_URL or SUPABASE_SERVICE_ROLE_KEY is missing. Using NoOp.")
    return NoOpPredictionRepository()
```

backend_fastapi/app/repositories/supabase_prediction_repository.py

```
from __future__ import annotations

import logging
from typing import Any

import httpx
```

```

from app.schemas.prediction import FrostPredictionHistoryItem, FrostRiskRequest, FrostRiskResponse


LOGGER = logging.getLogger(__name__)


class SupabasePredictionRepository:
    def __init__(
        self,
        supabase_url: str,
        service_role_key: str,
        client: httpx.Client | None = None,
    ) -> None:
        self.supabase_url = supabase_url.rstrip("/")
        self.service_role_key = service_role_key
        self.client = client or httpx.Client(timeout=10)

    def save_prediction(self, request: FrostRiskRequest, response: FrostRiskResponse) -> None:
        payload = self._build_insert_payload(request, response)
        try:
            result = self.client.post(
                f"{self.supabase_url}/rest/v1/frost_predictions",
                headers=self._headers(prefer="return=minimal"),
                json=payload,
            )
            result.raise_for_status()
        except Exception as exc: # noqa: BLE001 - prediction should still be returned if persistence fails.
            LOGGER.warning("Could not persist prediction to Supabase: %s", exc)

    def list_history(self, limit: int = 20) -> list[FrostPredictionHistoryItem]:
        safe_limit = max(1, min(limit, 100))
        try:
            result = self.client.get(
                f"{self.supabase_url}/rest/v1/frost_predictions",
                headers=self._headers(),
                params={
                    "select": (
                        "id,district,province,populated_center,latitude,longitude,altitude,"
                        "risk_level,confidence,chuno_conditions,recommendation,model_version,"
                        "data_sources,created_at"
                    ),
                    "order": "created_at.desc",
                    "limit": str(safe_limit),
                },
            )
            result.raise_for_status()
            rows = result.json()
            if not isinstance(rows, list):
                LOGGER.warning("Unexpected Supabase history response shape: %s", type(rows).__name__)
                return []
            return [FrostPredictionHistoryItem.model_validate(row) for row in rows]
        except Exception as exc: # noqa: BLE001 - history endpoint should degrade gracefully for MVP.
            LOGGER.warning("Could not read prediction history from Supabase: %s", exc)
            return []

    def _headers(self, prefer: str | None = None) -> dict[str, str]:
        headers = {
            "apikey": self.service_role_key,
            "Authorization": f"Bearer {self.service_role_key}",
            "Content-Type": "application/json",
        }
        if prefer:
            headers["Prefer"] = prefer
        return headers

    @staticmethod
    def _build_insert_payload(request: FrostRiskRequest, response: FrostRiskResponse) -> dict[str, Any]:
        return {
            "user_id": None,
            "district": request.district,
            "province": request.province,
            "populated_center": request.populated_center,
            "latitude": request.latitude,
            "longitude": request.longitude,
            "altitude": request.altitude,
            "input_payload": request.model_dump(mode="json"),
            "risk_level": response.risk_level,
            "confidence": response.confidence,
            "chuno_conditions": response.chuno_conditions,
            "recommendation": response.recommendation,
            "model_version": response.model_version,
            "data_sources": response.data_sources,
        }
```

backend_fastapi/app/schemas/___init___.py

```
"""Pydantic schemas for request and response contracts."""
```

backend_fastapi/app/schemas/alerts.py

```
from __future__ import annotations

from pydantic import BaseModel, ConfigDict


class LivestockRisk(BaseModel):
    level: str # alto | medio | bajo
    message: str


class ClimateAnomaly(BaseModel):
    historical_mean: float | None = None
    delta: float | None = None
    is_unusual: bool = False
    message: str


class DailyAlertResponse(BaseModel):
    latitude: float
    longitude: float
```

```
    date: str | None = None
    risk_level: str # alto | medio | bajo
    frost_alert: bool
    severity: str # fuerte | moderada | ninguna
    temperature_min: float | None = None
    title: str
    message: str
    livestock: LivestockRisk
    anomaly: ClimateAnomaly
    model_version: str

    model_config = ConfigDict(extra="forbid")
```

backend_fastapi/app/schemas/chuno.py

```
from __future__ import annotations

from pydantic import BaseModel, ConfigDict


class ChunoDay(BaseModel):
    date: str
    temperature_min: float | None = None
    temperature_max: float | None = None
    humidity_max: float | None = None
    cloud_cover_mean: float | None = None
    precipitation_sum: float | None = None
    tier: str # excelente | bueno | marginal | no_apto
    is_good_day: bool


class ChunoWindowResponse(BaseModel):
    in_season: bool
    season_months: list[int]
    latitude: float
    longitude: float
    days: list[ChunoDay]
    optimal_window: bool
    best_streak: int
    message: str
    criteria_source: str = "docs/chuno_criteria.md"

    model_config = ConfigDict(extra="forbid")
```

backend_fastapi/app/schemas/health.py

```
from __future__ import annotations

from pydantic import BaseModel, ConfigDict


class HealthResponse(BaseModel):
    status: str
    app_name: str
    version: str
    model_available: bool

    model_config = ConfigDict(extra="forbid")
```

backend_fastapi/app/schemas/ml.py

```
from __future__ import annotations

from typing import Any

from pydantic import BaseModel, ConfigDict


class ModelInfoResponse(BaseModel):
    model_name: str
    version: str
    created_at: str
    features: list[str]
    target: str
    metrics: dict[str, float]
    dataset_size: int
    data_sources: list[str]
    limitations: list[str]
    n_clusters: int | None = None

    model_config = ConfigDict(extra="forbid")


class ModelMetadata(ModelInfoResponse):
    # Campos del modelo no supervisado (K-Means). extra="ignore" tolera
    # metadata legacy del clasificador supervisado sin romper la carga.
    all_model_metrics: dict[str, dict[str, float]] | None = None
    all_k_metrics: dict[str, dict[str, float]] | None = None
    cluster_tiers: dict[str, str] | None = None
    cluster_profiles: dict[str, dict[str, Any]] | None = None

    model_config = ConfigDict(extra="ignore")


class ClusterProfile(BaseModel):
    cluster_id: int
    tier: str
    size: int
    temperature_2m: float
    dew_point_2m: float


class DistrictCluster(BaseModel):
    distrito: str
    tier: str
    dominant_cluster: int
```

```
    cold_share: float
    altitud_estimada: float
    temperature_2m_mean: float
    latitud: float | None = None
    longitud: float | None = None


class ClustersResponse(BaseModel):
    model_name: str
    version: str
    n_clusters: int
    silhouette: float
    profiles: list[ClusterProfile]
    districts: list[DistrictCluster]
```

backend_fastapi/app/schemas/prediction.py

```
from __future__ import annotations

from pydantic import BaseModel, ConfigDict, Field


class FrostRiskRequest(BaseModel):
    district: str = Field(..., min_length=1, examples=["Puno"])
    province: str = Field(..., min_length=1, examples=["Puno"])
    populated_center: str | None = Field(default=None, examples=["Centro poblado demo"])
    latitude: float = Field(..., ge=-18.5, le=-13.0, examples=[-15.8402])
    longitude: float = Field(..., ge=-71.5, le=-68.0, examples=[-70.0219])
    altitude: float = Field(..., ge=0, le=7000, examples=[3827])
    rural_population: int = Field(..., ge=0, examples=[1200])
    total_population: int | None = Field(default=None, ge=0, examples=[5000])
    rural_percentage: float | None = Field(default=None, ge=0, le=100, examples=[24.0])
    agricultural_activity: bool = Field(default=True)
    main_crop: str | None = Field(default="papa", min_length=1)
    temperature_min: float = Field(..., ge=-30, le=40, examples=[-2.5])
    temperature_max: float = Field(..., ge=-30, le=45, examples=[12.4])
    feels_like: float = Field(..., ge=-40, le=45, examples=[-4.0])
    humidity: float = Field(..., ge=0, le=100, examples=[68])
    wind_speed: float = Field(..., ge=0, le=200, examples=[7])
    cloud_cover: float = Field(..., ge=0, le=100, examples=[20])
    dew_point: float = Field(..., ge=-40, le=40, examples=[-3.5])
    precipitation: float = Field(..., ge=0, le=500, examples=[0])
    month: int = Field(..., ge=1, le=12, examples=[6])
    hour: int = Field(..., ge=0, le=23, examples=[3])
    hours_below_zero: int = Field(..., ge=0, le=24, examples=[4])

    model_config = ConfigDict(extra="forbid")


class FrostRiskResponse(BaseModel):
    risk_level: str
    confidence: float = Field(..., ge=0, le=1)
    recommendation: str
    chuno_conditions: str
    model_version: str
    data_sources: list[str]

    model_config = ConfigDict(extra="forbid")


class FrostPredictionHistoryItem(BaseModel):
    id: str
    district: str
    province: str
    populated_center: str | None = None
    latitude: float
    longitude: float
    altitude: float | None = None
    risk_level: str
    confidence: float = Field(..., ge=0, le=1)
    chuno_conditions: str
    recommendation: str
    model_version: str
    data_sources: list[str]
    created_at: str

    model_config = ConfigDict(extra="ignore")
```

backend_fastapi/app/schemas/weather.py

```
from __future__ import annotations

from pydantic import BaseModel, ConfigDict, Field


class CurrentWeatherResponse(BaseModel):
    latitude: float
    longitude: float
    temperature: float | None = None
    apparent_temperature: float | None = None
    humidity: float | None = None
    wind_speed: float | None = None
    cloud_cover: float | None = None
    dew_point: float | None = None
    precipitation: float | None = None
    observed_at: str | None = None
    provider: str
    source_priority: list[str]
    station_name: str | None = None
    station_distance_km: float | None = None
    fallback_used: bool = False
    limitations: list[str] = Field(default_factory=list)

    model_config = ConfigDict(extra="forbid")


class HistoryDay(BaseModel):
    date: str
    temperature_min: float | None = None
    temperature_max: float | None = None
    humidity_mean: float | None = None
```

```

class WeatherHistoryResponse(BaseModel):
    latitude: float
    longitude: float
    start_date: str
    end_date: str
    provider: str = "Open-Meteo Archive"
    days: list[HistoryDay]

    model_config = ConfigDict(extra="forbid")
```

backend_fastapi/app/services/___init___.py

```
"""Backend service layer."""
```

backend_fastapi/app/services/alert_service.py

```
"""Lógica de la alerta diaria: helada, riesgo para ganado y anomalía climática.

Umbrales de ganado documentados en docs/livestock_criteria.md (crías de alpaca y
ovino del altiplano son vulnerables a mortalidad neonatal por helada). Son valores
MVP defendibles, no cifras oficiales SENAMHI.
"""

from __future__ import annotations

from statistics import mean, pstdev

# Umbrales de helada sobre la mínima diaria (°C).
STRONG_FROST_MAX = -4.0
MILD_FROST_MAX = 0.0

# Umbrales para ganado (crías vulnerables).
LIVESTOCK_HIGH_MAX = -6.0
LIVESTOCK_MEDIUM_MAX = -2.0

# Anomalía: se marca inusual si la mínima cae bajo media - K*desviación.
ANOMALY_K = 1.5


def frost_severity(temp_min: float | None) -> tuple[str, str, bool, str, str]:
    """Devuelve (risk_level, severity, alert, title, message) para heladas."""
    if temp_min is not None and temp_min <= STRONG_FROST_MAX:
        return (
            "alto",
            "fuerte",
            True,
            "Riesgo de helada fuerte",
            "Hoy hay riesgo de helada fuerte. Se recomienda resguardar el ganado y proteger los cultivos.",
        )
    if temp_min is not None and temp_min <= MILD_FROST_MAX:
        return (
            "medio",
            "moderada",
            True,
            "Riesgo de helada moderada",
            "Hoy hay riesgo de helada moderada. Monitoree la temperatura nocturna y prepare medidas preventivas.",
        )
    return (
        "bajo",
        "ninguna",
        False,
        "Sin alerta de helada",
        "No se prevé helada significativa para hoy. Mantenga seguimiento ante cambios bruscos.",
    )


def livestock_risk(temp_min: float | None) -> dict[str, str]:
    """Riesgo específico para ganado altoandino (crías de alpaca y ovino)."""
    if temp_min is not None and temp_min <= LIVESTOCK_HIGH_MAX:
        return {
            "level": "alto",
            "message": (
                "Riesgo alto para el ganado: proteja crías de alpaca y ovino esta noche; "
                "peligro de mortalidad neonatal por frío."
            ),
        }
    if temp_min is not None and temp_min <= LIVESTOCK_MEDIUM_MAX:
        return {
            "level": "medio",
            "message": (
                "Riesgo medio para el ganado: abrigue a las crías y revise cobertizos antes del anochecer."
            ),
        }
    return {
        "level": "bajo",
        "message": "Sin riesgo relevante para el ganado por temperatura esta noche.",
    }


def anomaly(temp_min_today: float | None, history_rows: list[dict]) -> dict:
    """Compara la mínima de hoy con la climatología reciente (últimos ~30 días)."""
    values = [
        row["temperature_2m_min"]
        for row in history_rows
        if row.get("temperature_2m_min") is not None
    ]
    if temp_min_today is None or len(values) < 5:
        return {
            "historical_mean": None,
            "delta": None,
            "is_unusual": False,
            "message": "Sin datos históricos suficientes para evaluar anomalías.",
        }

    historical_mean = mean(values)
    deviation = pstdev(values)
    delta = temp_min_today - historical_mean
    is_unusual = deviation > 0 and temp_min_today < historical_mean - ANOMALY_K * deviation

```

```
    if is_unusual:
        message = (
            f"Helada inusual: {abs(delta):.1f} °C por debajo de lo normal para estas fechas."
        )
    else:
        message = "Temperatura dentro de lo normal para la temporada."

    return {
        "historical_mean": round(historical_mean, 1),
        "delta": round(delta, 1),
        "is_unusual": is_unusual,
        "message": message,
    }
```

backend_fastapi/app/services/chuno_service.py

```
"""Reglas del módulo estacional de chuño.

Umbrales documentados en docs/chuno_criteria.md (–mayoagosto; congelamiento
nocturno ≤–5 °C; secado con cielo despejado, aire seco y sin lluvia).
"""

from __future__ import annotations

from app.schemas.chuno import ChunoDay, ChunoWindowResponse

SEASON_MONTHS = [5, 6, 7, 8]
MIN_CONSECUTIVE_GOOD_DAYS = 3

# Umbrales base ("buen día")
FREEZE_MAX = -5.0
PRECIP_MAX = 0.2
CLOUD_MAX = 40.0
HUMIDITY_MAX = 60.0

# Umbrales "excelente"
FREEZE_EXCELLENT = -8.0
CLOUD_EXCELLENT = 20.0
HUMIDITY_EXCELLENT = 50.0


def _classify(temp_min, precip, cloud, humidity) -> tuple[str, bool]:
    if temp_min is None:
        return "no_apto", False
    precip = precip if precip is not None else 0.0
    cloud = cloud if cloud is not None else 100.0
    humidity = humidity if humidity is not None else 100.0

    good = temp_min <= FREEZE_MAX and precip <= PRECIP_MAX and cloud <= CLOUD_MAX and humidity <= HUMIDITY_MAX
    if good:
        excellent = (
            temp_min <= FREEZE_EXCELLENT
            and cloud <= CLOUD_EXCELLENT
            and humidity <= HUMIDITY_EXCELLENT
            and precip <= 0.0
        )
        return ("excelente" if excellent else "bueno"), True

    if temp_min > -2.0 or precip > 1.0:
        return "no_apto", False
    return "marginal", False


def _best_streak(days: list[ChunoDay]) -> int:
    best = current = 0
    for day in days:
        current = current + 1 if day.is_good_day else 0
        best = max(best, current)
    return best


def evaluate_window(
    latitude: float,
    longitude: float,
    current_month: int,
    forecast_rows: list[dict],
) -> ChunoWindowResponse:
    in_season = current_month in SEASON_MONTHS

    days: list[ChunoDay] = []
    for row in forecast_rows:
        tier, is_good = _classify(
            row.get("temperature_2m_min"),
            row.get("precipitation_sum"),
            row.get("cloud_cover_mean"),
            row.get("relative_humidity_2m_max"),
        )
        days.append(
            ChunoDay(
                date=str(row.get("date")),
                temperature_min=row.get("temperature_2m_min"),
                temperature_max=row.get("temperature_2m_max"),
                humidity_max=row.get("relative_humidity_2m_max"),
                cloud_cover_mean=row.get("cloud_cover_mean"),
                precipitation_sum=row.get("precipitation_sum"),
                tier=tier,
                is_good_day=is_good,
            )
        )

    best_streak = _best_streak(days)
    optimal = in_season and best_streak >= MIN_CONSECUTIVE_GOOD_DAYS

    if not in_season:
        message = "Fuera de temporada de chuño (–mayoagosto). El módulo se reactiva en la estación seca fría."
    elif optimal:
        message = (
            f"Ventana óptima detectada: {best_streak} días consecutivos aptos para congelar y secar chuño."
        )
    else:
        message = (
            "En temporada, pero aún sin una ventana de 3 días consecutivos aptos. "
            "Monitoree el pronóstico nocturno."
        )

    return ChunoWindowResponse(
```

```
        in_season=in_season,
        season_months=SEASON_MONTHS,
        latitude=latitude,
        longitude=longitude,
        days=days,
        optimal_window=optimal,
        best_streak=best_streak,
        message=message,
    )
```

backend__fastapi/app/services/model__registry.py

```python
from __future__ import annotations

import json
import logging
from functools import lru_cache
from pathlib import Path
from typing import Any

import joblib

from app.core.config import settings
from app.core.exceptions import ModelLoadError
from app.schemas.ml import ModelMetadata


LOGGER = logging.getLogger(__name__)


class ModelRegistry:
    def __init__(self, model_path: Path, metadata_path: Path) -> None:
        self.model_path = model_path
        self.metadata_path = metadata_path
        self._model: Any | None = None
        self._metadata: ModelMetadata | None = None

    @property
    def model(self) -> Any:
        if self._model is None:
            self._model = self._load_model()
        return self._model

    @property
    def metadata(self) -> ModelMetadata:
        if self._metadata is None:
            self._metadata = self._load_metadata()
        return self._metadata

    def _load_model(self) -> Any:
        if not self.model_path.exists():
            raise ModelLoadError(f"Model artifact not found at {self.model_path}")

        try:
            LOGGER.info("Loading model artifact from %s", self.model_path)
            return joblib.load(self.model_path)
        except Exception as exc: # noqa: BLE001 - convert registry failures into API-safe errors.
            raise ModelLoadError("Could not load the registered model artifact.") from exc

    def _load_metadata(self) -> ModelMetadata:
        if not self.metadata_path.exists():
            raise ModelLoadError(f"Model metadata not found at {self.metadata_path}")

        try:
            raw_metadata = json.loads(self.metadata_path.read_text(encoding="utf-8"))
            return ModelMetadata.model_validate(raw_metadata)
        except Exception as exc: # noqa: BLE001 - convert malformed metadata into API-safe errors.
            raise ModelLoadError("Could not load model metadata.") from exc


@lru_cache(maxsize=1)
def get_model_registry() -> ModelRegistry:
    return ModelRegistry(settings.model_path, settings.model_metadata_path)
```

backend__fastapi/app/services/prediction__service.py

```python
from __future__ import annotations

import logging
from functools import lru_cache
from typing import Any

import pandas as pd

from app.core.exceptions import PredictionError
from app.schemas.prediction import FrostRiskRequest, FrostRiskResponse
from app.services.model_registry import ModelRegistry, get_model_registry
from app.services.recommendation_service import build_recommendation


LOGGER = logging.getLogger(__name__)


class PredictionService:
    def __init__(self, registry: ModelRegistry) -> None:
        self.registry = registry

    def predict(self, payload: FrostRiskRequest) -> FrostRiskResponse:
        metadata = self.registry.metadata
        feature_frame = self._to_feature_frame(payload, metadata.features)
        tier_map = metadata.cluster_tiers or {}

        try:
            model = self.registry.model
            cluster_id = int(model.predict(feature_frame)[0])
            risk_level = tier_map.get(str(cluster_id), "medio")
            confidence = self._cluster_confidence(model, feature_frame)
        except Exception as exc: # noqa: BLE001 - expose stable API error instead of model internals.
            LOGGER.exception("Prediction failed.")
            raise PredictionError("The model could not generate a prediction for the request.") from exc

```

```
        recommendation, chuno_conditions = build_recommendation(risk_level, payload)
        return FrostRiskResponse(
            risk_level=risk_level,
            confidence=confidence,
            recommendation=recommendation,
            chuno_conditions=chuno_conditions,
            model_version=metadata.version,
            data_sources=metadata.data_sources,
        )

    def _to_feature_frame(self, payload: FrostRiskRequest, feature_names: list[str]) -> pd.DataFrame:
        # Vector para el modelo no supervisado: la temperatura mínima es la señal
        # de helada relevante y se mapea a temperature_2m.
        feature_values: dict[str, Any] = {
            "latitud": payload.latitude,
            "longitud": payload.longitude,
            "altitud_estimada": payload.altitude,
            "temperature_2m": payload.temperature_min,
            "relative_humidity_2m": payload.humidity,
            "apparent_temperature": payload.feels_like,
            "dew_point_2m": payload.dew_point,
            "precipitation": payload.precipitation,
            "cloud_cover": payload.cloud_cover,
            "wind_speed_10m": payload.wind_speed,
            "mes": payload.month,
            "hora": payload.hour,
        }

        missing = [name for name in feature_names if name not in feature_values]
        if missing:
            raise PredictionError(f"Request cannot build required model features: {missing}")

        return pd.DataFrame([{name: feature_values[name] for name in feature_names}])

    @staticmethod
    def _cluster_confidence(model: Any, feature_frame: pd.DataFrame) -> float:
        """Confianza por cercanía relativa al centroide más cercano (KMeans.transform)."""
        if not hasattr(model, "transform"):
            return 1.0

        distances = model.transform(feature_frame)[0]
        inverse = 1.0 / (distances + 1e-9)
        total = float(inverse.sum())
        if total <= 0:
            return 1.0
        return float(inverse.max() / total)


@lru_cache(maxsize=1)
def get_prediction_service() -> PredictionService:
    return PredictionService(get_model_registry())
```

backend__fastapi/app/services/recommendation__service.py

```
from __future__ import annotations

from app.schemas.prediction import FrostRiskRequest


def build_recommendation(risk_level: str, payload: FrostRiskRequest) -> tuple[str, str]:
    crop = payload.main_crop or "cultivos sensibles"

    if risk_level == "alto":
        recommendation = (
            "Existe alto riesgo de helada. Se recomienda proteger cultivos sensibles, "
            f"priorizar vigilancia de {crop} y revisar ganado o infraestructura expuesta."
        )
    elif risk_level == "medio":
        recommendation = (
            "Existe riesgo medio de helada. Se recomienda monitorear la temperatura nocturna "
            "y preparar medidas preventivas si el descenso continua."
        )
    else:
        recommendation = (
            "El riesgo de helada es bajo para las condiciones enviadas. Mantenga seguimiento "
            "si hay cambios bruscos de viento, nubosidad o temperatura."
        )

    if payload.temperature_min <= 0 and payload.hours_below_zero >= 3 and payload.precipitation <= 1:
        chuno_conditions = "favorables"
        recommendation += " Las condiciones pueden ser favorables para la produccion de chuno."
    elif payload.temperature_min <= 3:
        chuno_conditions = "posibles"
    else:
        chuno_conditions = "no_favorables"

    return recommendation, chuno_conditions
```

backend__fastapi/app/services/weather__providers.py

```
from __future__ import annotations

import logging
import math
import time
from dataclasses import dataclass
from typing import Protocol

import httpx

from app.schemas.weather import CurrentWeatherResponse


LOGGER = logging.getLogger(__name__)


@dataclass(frozen=True)
class WeatherQuery:
    latitude: float
    longitude: float

```

```python

class WeatherProvider(Protocol):
    name: str

    def get_current_weather(self, query: WeatherQuery) -> CurrentWeatherResponse | None:
        """Return current weather or None when the provider has no usable data."""


class SenamhiProvider:
    name = "SENAMHI"

    _puno_reference_stations = (
        ("Puno", -15.8402, -70.0219, 3827.0),
        ("Juliaca", -15.4997, -70.1333, 3825.0),
        ("Ilave", -16.0833, -69.6667, 3850.0),
    )

    def get_current_weather(self, query: WeatherQuery) -> CurrentWeatherResponse | None:
        station_name, distance_km = self._nearest_station(query)
        LOGGER.info(
            "SENAMHI provider prepared nearest station=%s distance=%.2f km; no stable public API configured.",
            station_name,
            distance_km,
        )
        return None

    def nearest_station_metadata(self, query: WeatherQuery) -> tuple[str, float]:
        return self._nearest_station(query)

    def _nearest_station(self, query: WeatherQuery) -> tuple[str, float]:
        nearest = min(
            self._puno_reference_stations,
            key=lambda station: _distance_km(query.latitude, query.longitude, station[1], station[2]),
        )
        return nearest[0], _distance_km(query.latitude, query.longitude, nearest[1], nearest[2])


class OpenMeteoProvider:
    name = "Open-Meteo"

    def __init__(self, client: httpx.Client | None = None) -> None:
        self._client = client or httpx.Client(timeout=12)

    def get_current_weather(self, query: WeatherQuery) -> CurrentWeatherResponse | None:
        response = self._client.get(
            "https://api.open-meteo.com/v1/forecast",
            params={
                "latitude": query.latitude,
                "longitude": query.longitude,
                "current": ",".join(
                    [
                        "temperature_2m",
                        "relative_humidity_2m",
                        "apparent_temperature",
                        "dew_point_2m",
                        "precipitation",
                        "cloud_cover",
                        "wind_speed_10m",
                    ],
                ),
                "timezone": "auto",
            },
        )
        response.raise_for_status()
        payload = response.json()
        current = payload.get("current") or {}
        return CurrentWeatherResponse(
            latitude=query.latitude,
            longitude=query.longitude,
            temperature=_num(current.get("temperature_2m")),
            apparent_temperature=_num(current.get("apparent_temperature")),
            humidity=_num(current.get("relative_humidity_2m")),
            wind_speed=_num(current.get("wind_speed_10m")),
            cloud_cover=_num(current.get("cloud_cover")),
            dew_point=_num(current.get("dew_point_2m")),
            precipitation=_num(current.get("precipitation")),
            observed_at=current.get("time"),
            provider=self.name,
            source_priority=["SENAMHI", "Open-Meteo"],
            fallback_used=True,
            limitations=[
                "SENAMHI is prioritized conceptually, but no stable unauthenticated API is configured in this MVP.",
                "Open-Meteo is used as fallback/global weather source for current app predictions.",
            ],
        )


    def get_daily_forecast(self, query: WeatherQuery, days: int = 7) -> list[dict[str, float | str | None]]:
        """Pronóstico diario Open-Meteo para el módulo de chuño."""
        response = self._client.get(
            "https://api.open-meteo.com/v1/forecast",
            params={
                "latitude": query.latitude,
                "longitude": query.longitude,
                "daily": ",".join(
                    [
                        "temperature_2m_min",
                        "temperature_2m_max",
                        "relative_humidity_2m_max",
                        "cloud_cover_mean",
                        "precipitation_sum",
                    ],
                ),
                "forecast_days": days,
                "timezone": "auto",
            },
        )
        response.raise_for_status()
        return _rows_from_daily(response.json().get("daily") or {})

    def get_history(
        self, query: WeatherQuery, start_date: str, end_date: str
    ) -> list[dict[str, float | str | None]]:
        """Serie histórica diaria vía Open-Meteo Archive API (sin API key)."""
        response = self._client.get(
            "https://archive-api.open-meteo.com/v1/archive",
            params={
                "latitude": query.latitude,
                "longitude": query.longitude,
                "start_date": start_date,
                "end_date": end_date,
```

```
            "daily": ",".join(
                [
                    "temperature_2m_min",
                    "temperature_2m_max",
                    "relative_humidity_2m_mean",
                ],
            ),
            "timezone": "auto",
        },
    )
    response.raise_for_status()
    return _rows_from_daily(response.json().get("daily") or {})


class HybridWeatherProvider:
    name = "HybridWeatherProvider"

    def __init__(
        self,
        senamhi_provider: SenamhiProvider | None = None,
        open_meteo_provider: OpenMeteoProvider | None = None,
        cache_ttl_seconds: int = 600,
    ) -> None:
        self._senamhi = senamhi_provider or SenamhiProvider()
        self._open_meteo = open_meteo_provider or OpenMeteoProvider()
        self._cache_ttl_seconds = cache_ttl_seconds
        self._cache: dict[tuple[float, float], tuple[float, CurrentWeatherResponse]] = {}

    def get_current_weather(self, query: WeatherQuery) -> CurrentWeatherResponse:
        cache_key = (round(query.latitude, 4), round(query.longitude, 4))
        cached = self._cache.get(cache_key)
        now = time.time()
        if cached and now - cached[0] < self._cache_ttl_seconds:
            return cached[1]

        station_name, station_distance = self._senamhi.nearest_station_metadata(query)
        try:
            senamhi_weather = self._senamhi.get_current_weather(query)
            if senamhi_weather is not None:
                result = senamhi_weather
            else:
                result = self._fallback_to_open_meteo(query, station_name, station_distance)
        except Exception as exc: # noqa: BLE001 - weather lookup must not break predictions.
            LOGGER.warning("SENAMHI provider failed; using Open-Meteo fallback: %s", exc)
            result = self._fallback_to_open_meteo(query, station_name, station_distance)

        self._cache[cache_key] = (now, result)
        return result

    def get_daily_forecast(self, query: WeatherQuery, days: int = 7) -> list[dict[str, float | str | None]]:
        return self._open_meteo.get_daily_forecast(query, days=days)

    def get_history(
        self, query: WeatherQuery, start_date: str, end_date: str
    ) -> list[dict[str, float | str | None]]:
        return self._open_meteo.get_history(query, start_date, end_date)

    def _fallback_to_open_meteo(
        self,
        query: WeatherQuery,
        station_name: str,
        station_distance: float,
    ) -> CurrentWeatherResponse:
        result = self._open_meteo.get_current_weather(query)
        if result is None:
            return CurrentWeatherResponse(
                latitude=query.latitude,
                longitude=query.longitude,
                provider="unavailable",
                source_priority=["SENAMHI", "Open-Meteo"],
                fallback_used=True,
                station_name=station_name,
                station_distance_km=station_distance,
                limitations=["No weather provider returned usable data."],
            )
        return result.model_copy(
            update={
                "station_name": station_name,
                "station_distance_km": station_distance,
            },
        )


def _distance_km(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
    radius_km = 6371.0
    d_lat = math.radians(lat2 - lat1)
    d_lon = math.radians(lon2 - lon1)
    a = (
        math.sin(d_lat / 2) ** 2
        + math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) * math.sin(d_lon / 2) ** 2
    )
    return round(radius_km * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a)), 2)


def _num(value: object) -> float | None:
    if value is None:
        return None
    return float(value)


def _rows_from_daily(daily: dict[str, list]) -> list[dict[str, float | str | None]]:
    """Transpone el bloque `daily` de Open-Meteo (columnas) en filas por fecha."""
    dates = daily.get("time") or []
    keys = [key for key in daily if key != "time"]
    rows: list[dict[str, float | str | None]] = []
    for index, date in enumerate(dates):
        row: dict[str, float | str | None] = {"date": date}
        for key in keys:
            values = daily.get(key) or []
            row[key] = _num(values[index]) if index < len(values) else None
        rows.append(row)
    return rows
```

backend__fastapi/tests/test__api.py

```
from __future__ import annotations
```

```python

import httpx
from fastapi.testclient import TestClient

from app.main import app
from app.repositories.predictions import NoOpPredictionRepository
from app.repositories.supabase_prediction_repository import SupabasePredictionRepository
from app.schemas.prediction import FrostRiskRequest, FrostRiskResponse


client = TestClient(app)


def sample_request() -> FrostRiskRequest:
    return FrostRiskRequest(
        district="Puno",
        province="Puno",
        populated_center="Centro poblado demo",
        latitude=-15.8402,
        longitude=-70.0219,
        altitude=3827,
        rural_population=1200,
        agricultural_activity=True,
        main_crop="papa",
        temperature_min=-2.5,
        temperature_max=12.4,
        feels_like=-4.0,
        humidity=68,
        wind_speed=7,
        cloud_cover=20,
        dew_point=-3.5,
        precipitation=0,
        month=6,
        hour=3,
        hours_below_zero=4,
    )


def sample_response() -> FrostRiskResponse:
    return FrostRiskResponse(
        risk_level="alto",
        confidence=0.98,
        recommendation="Existe alto riesgo de helada.",
        chuno_conditions="favorables",
        model_version="v0.1.0",
        data_sources=["Open-Meteo Historical API"],
    )


def test_health_check() -> None:
    response = client.get("/health")

    assert response.status_code == 200
    body = response.json()
    assert body["status"] in {"ok", "degraded"}
    assert "model_available" in body


def test_model_info_returns_cluster_metadata() -> None:
    response = client.get("/ml/model-info")

    assert response.status_code == 200
    body = response.json()
    assert body["model_name"] == "KMeans"
    assert body["version"].endswith("clustering")
    assert "silhouette" in body["metrics"]
    assert body["n_clusters"] >= 2


def test_clusters_endpoint_returns_profiles_and_districts() -> None:
    response = client.get("/ml/clusters")

    assert response.status_code == 200
    body = response.json()
    assert body["n_clusters"] == len(body["profiles"])
    assert body["profiles"], "expected at least one cluster profile"
    assert body["districts"], "expected district groupings"
    tiers = {district["tier"] for district in body["districts"]}
    assert tiers.issubset({"alto", "medio", "bajo"})


def test_predict_frost_risk_returns_cluster_tier() -> None:
    payload = {
        "district": "Puno",
        "province": "Puno",
        "populated_center": "Centro poblado demo",
        "latitude": -15.8402,
        "longitude": -70.0219,
        "altitude": 3827,
        "rural_population": 1200,
        "agricultural_activity": True,
        "main_crop": "papa",
        "temperature_min": -2.5,
        "temperature_max": 12.4,
        "feels_like": -4.0,
        "humidity": 68,
        "wind_speed": 7,
        "cloud_cover": 20,
        "dew_point": -3.5,
        "precipitation": 0,
        "month": 6,
        "hour": 3,
        "hours_below_zero": 4,
    }

    response = client.post("/predict/frost-risk", json=payload)

    assert response.status_code == 200
    body = response.json()
    assert body["risk_level"] in {"alto", "medio", "bajo"}
    assert 0 <= body["confidence"] <= 1
    assert body["model_version"].endswith("clustering")
    assert body["chuno_conditions"] == "favorables"


def test_prediction_history_returns_empty_without_supabase() -> None:
    response = client.get("/predictions/history")

    assert response.status_code == 200
    assert response.json() == []

```

```python

def test_current_weather_contract(monkeypatch) -> None:
    from app.api.routes import weather
    from app.schemas.weather import CurrentWeatherResponse

    class FakeWeatherProvider:
        def get_current_weather(self, query):
            return CurrentWeatherResponse(
                latitude=query.latitude,
                longitude=query.longitude,
                temperature=-1.5,
                humidity=70,
                provider="Open-Meteo",
                source_priority=["SENAMHI", "Open-Meteo"],
                fallback_used=True,
            )

    weather.get_weather_provider.cache_clear()
    monkeypatch.setattr(weather, "get_weather_provider", lambda: FakeWeatherProvider())

    response = client.get("/weather/current?latitude=-15.8402&longitude=-70.0219")

    assert response.status_code == 200
    body = response.json()
    assert body["provider"] == "Open-Meteo"
    assert body["fallback_used"] is True


class _FakeForecastProvider:
    def __init__(self, rows, history=None):
        self._rows = rows
        self._history = history if history is not None else rows

    def get_daily_forecast(self, query, days=7):
        return self._rows

    def get_history(self, query, start_date, end_date):
        return self._history


def test_chuno_window_flags_optimal_streak(monkeypatch) -> None:
    from app.api.routes import chuno

    cold_day = {
        "date": "2026-06-10",
        "temperature_2m_min": -7.0,
        "temperature_2m_max": 16.0,
        "relative_humidity_2m_max": 45.0,
        "cloud_cover_mean": 15.0,
        "precipitation_sum": 0.0,
    }
    rows = [dict(cold_day, date=f"2026-06-{10 + i}") for i in range(4)]
    monkeypatch.setattr(chuno, "get_weather_provider", lambda: _FakeForecastProvider(rows))

    response = client.get("/chuno/window?latitude=-15.8402&longitude=-70.0219")

    assert response.status_code == 200
    body = response.json()
    assert body["best_streak"] >= 3
    assert all(day["is_good_day"] for day in body["days"])
    # optimal_window depende de la temporada real; el streak siempre debe reflejar los dias buenos.
    assert body["days"][0]["tier"] in {"excelente", "bueno"}


def test_alert_today_strong_frost_with_livestock_and_anomaly(monkeypatch) -> None:
    from app.api.routes import alerts

    rows = [{"date": "2026-06-10", "temperature_2m_min": -7.0}]
    # Historia templada con varianza -> la minima de hoy (-7) es una anomalia clara.
    history = [
        {"date": f"2026-05-{d:02d}", "temperature_2m_min": 0.0 if d % 2 else 2.0}
        for d in range(1, 16)
    ]
    monkeypatch.setattr(
        alerts, "get_weather_provider", lambda: _FakeForecastProvider(rows, history=history)
    )

    response = client.get("/alerts/today?latitude=-15.8402&longitude=-70.0219")

    assert response.status_code == 200
    body = response.json()
    assert body["frost_alert"] is True
    assert body["severity"] == "fuerte"
    assert body["risk_level"] == "alto"
    assert body["livestock"]["level"] == "alto"
    assert body["anomaly"]["is_unusual"] is True
    assert body["anomaly"]["historical_mean"] is not None
    assert body["anomaly"]["delta"] < 0


def test_weather_history_contract(monkeypatch) -> None:
    from app.api.routes import weather

    rows = [
        {
            "date": "2026-06-01",
            "temperature_2m_min": -5.0,
            "temperature_2m_max": 14.0,
            "relative_humidity_2m_mean": 55.0,
        }
    ]
    weather.get_weather_provider.cache_clear()
    monkeypatch.setattr(weather, "get_weather_provider", lambda: _FakeForecastProvider(rows))

    response = client.get(
        "/weather/history?latitude=-15.8402&longitude=-70.0219&start=2026-06-01&end=2026-06-01"
    )

    assert response.status_code == 200
    body = response.json()
    assert body["days"][0]["temperature_min"] == -5.0
    assert body["days"][0]["humidity_mean"] == 55.0


def test_noop_repository_does_not_persist_and_returns_empty_history() -> None:
    repository = NoOpPredictionRepository()

    repository.save_prediction(sample_request(), sample_response())

    assert repository.list_history(limit=10) == []

```

```python

def test_supabase_repository_insert_failure_does_not_raise(caplog) -> None:
    def handler(_: httpx.Request) -> httpx.Response:
        return httpx.Response(status_code=500, json={"message": "database unavailable"})

    client_with_error = httpx.Client(transport=httpx.MockTransport(handler))
    repository = SupabasePredictionRepository(
        supabase_url="https://example.supabase.co",
        service_role_key="test-service-role-key",
        client=client_with_error,
    )

    repository.save_prediction(sample_request(), sample_response())

    assert "Could not persist prediction to Supabase" in caplog.text


def test_supabase_repository_history_failure_returns_empty() -> None:
    def handler(_: httpx.Request) -> httpx.Response:
        return httpx.Response(status_code=401, json={"message": "unauthorized"})

    client_with_error = httpx.Client(transport=httpx.MockTransport(handler))
    repository = SupabasePredictionRepository(
        supabase_url="https://example.supabase.co",
        service_role_key="test-service-role-key",
        client=client_with_error,
    )

    assert repository.list_history(limit=10) == []
```

3. Cliente Flutter

app_flutter/lib/app.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';

import 'core/api/api_client.dart';
import 'core/api/frost_api_service.dart';
import 'core/i18n/app_language.dart';
import 'core/settings/user_settings.dart';
import 'core/theme/app_theme.dart';
import 'core/theme/theme_mode_scope.dart';
import 'features/shell/app_shell.dart';

class FrostPunoApp extends StatefulWidget {
  const FrostPunoApp({super.key});

  @override
  State<FrostPunoApp> createState() => _FrostPunoAppState();
}

class _FrostPunoAppState extends State<FrostPunoApp> {
  late final ApiClient _apiClient = ApiClient();
  late final FrostApiService _apiService = FrostApiService(_apiClient);
  ThemeMode _themeMode = ThemeMode.system;

  @override
  Widget build(BuildContext context) {
    // Reconstruye toda la app al cambiar idioma/preferencias del usuario.
    return AnimatedBuilder(
      animation: UserSettings.instance,
      builder: (context, _) {
        return ThemeModeScope(
          themeMode: _themeMode,
          onThemeModeChanged: (mode) => setState(() => _themeMode = mode),
          child: MaterialApp(
            title: 'FrostPuno',
            debugShowCheckedModeBanner: false,
            theme: AppTheme.light(),
            darkTheme: AppTheme.dark(),
            themeMode: _themeMode,
            locale: Locale(UserSettings.instance.language.code),
            supportedLocales: const [Locale('es'), Locale('en')],
            localizationsDelegates: const [
              GlobalMaterialLocalizations.delegate,
              GlobalWidgetsLocalizations.delegate,
              GlobalCupertinoLocalizations.delegate,
            ],
            home: AppShell(apiService: _apiService),
          ),
        );
      },
    );
  }
}
```

app_flutter/lib/core/api/api_client.dart

```dart
import 'dart:convert';

import 'package:http/http.dart' as http;

import '../config/app_config.dart';

class ApiClient {
  ApiClient({http.Client? client, String? baseUrl})
    : _client = client ?? http.Client(),
      _baseUrl = baseUrl ?? AppConfig.apiBaseUrl;

  final http.Client _client;
  final String _baseUrl;

  Future<Map<String, dynamic>> getJson(String path) async {
    final response = await _client.get(_uri(path));
    return _decodeObject(response);
  }

```

```
  Future<List<dynamic>> getList(String path) async {
    final response = await _client.get(_uri(path));
    final decoded = _decode(response);
    if (decoded is List) {
      return decoded;
    }
    throw ApiException('La API devolvio una respuesta inesperada.');
  }

  Future<Map<String, dynamic>> postJson(
    String path,
    Map<String, dynamic> body,
  ) async {
    final response = await _client.post(
      _uri(path),
      headers: const {'Content-Type': 'application/json'},
      body: jsonEncode(body),
    );
    return _decodeObject(response);
  }

  Uri _uri(String path) => Uri.parse('$_baseUrl$path');

  Map<String, dynamic> _decodeObject(http.Response response) {
    final decoded = _decode(response);
    if (decoded is Map<String, dynamic>) {
      return decoded;
    }
    throw ApiException('La API devolvio una respuesta inesperada.');
  }

  dynamic _decode(http.Response response) {
    if (response.statusCode < 200 || response.statusCode >= 300) {
      throw ApiException('Error ${response.statusCode}: ${response.body}');
    }
    if (response.body.isEmpty) {
      return <String, dynamic>{};
    }
    return jsonDecode(response.body);
  }
}

class ApiException implements Exception {
  ApiException(this.message);

  final String message;

  @override
  String toString() => message;
}
```

app_flutter/lib/core/api/frost_api_service.dart

```
import '../../features/alerts/models/daily_alert.dart';
import '../../features/charts/models/history_day.dart';
import '../../features/chuno/models/chuno_window.dart';
import '../../features/clusters/models/cluster_info.dart';
import '../../features/history/models/prediction_history_item.dart';
import '../../features/model_info/models/model_info.dart';
import '../../features/prediction/models/frost_risk_request.dart';
import '../../features/prediction/models/frost_risk_response.dart';
import '../models/current_weather.dart';
import '../models/health_status.dart';
import 'api_client.dart';

class FrostApiService {
  FrostApiService(this._client);

  final ApiClient _client;

  Future<HealthStatus> getHealth() async {
    final json = await _client.getJson('/health');
    return HealthStatus.fromJson(json);
  }

  Future<ModelInfo> getModelInfo() async {
    final json = await _client.getJson('/ml/model-info');
    return ModelInfo.fromJson(json);
  }

  Future<FrostRiskResponse> predict(FrostRiskRequest request) async {
    final json = await _client.postJson(
      '/predict/frost-risk',
      request.toJson(),
    );
    return FrostRiskResponse.fromJson(json);
  }

  Future<List<PredictionHistoryItem>> getPredictionHistory() async {
    final rows = await _client.getList('/predictions/history');
    return rows
        .whereType<Map<String, dynamic>>()
        .map(PredictionHistoryItem.fromJson)
        .toList();
  }

  Future<CurrentWeather> getCurrentWeather({
    required double latitude,
    required double longitude,
  }) async {
    final json = await _client.getJson(
      '/weather/current?latitude=$latitude&longitude=$longitude',
    );
    return CurrentWeather.fromJson(json);
  }

  Future<ClustersResult> getClusters() async {
    final json = await _client.getJson('/ml/clusters');
    return ClustersResult.fromJson(json);
  }

  Future<ChunoWindow> getChunoWindow({
    required double latitude,
    required double longitude,
  }) async {
    final json = await _client.getJson(
```

```
      '/chuno/window?latitude=$latitude&longitude=$longitude',
    );
    return ChunoWindow.fromJson(json);
  }

  Future<WeatherHistory> getWeatherHistory({
    required double latitude,
    required double longitude,
    required String start,
    required String end,
  }) async {
    final json = await _client.getJson(
      '/weather/history?latitude=$latitude&longitude=$longitude&start=$start&end=$end',
    );
    return WeatherHistory.fromJson(json);
  }

  Future<DailyAlert> getDailyAlert({
    required double latitude,
    required double longitude,
  }) async {
    final json = await _client.getJson(
      '/alerts/today?latitude=$latitude&longitude=$longitude',
    );
    return DailyAlert.fromJson(json);
  }
}
```

app_flutter/lib/core/config/app_config.dart

```
class AppConfig {
  static const apiBaseUrl = String.fromEnvironment(
    'API_BASE_URL',
    defaultValue: 'http://127.0.0.1:8000',
  );
}
```

app_flutter/lib/core/i18n/app_language.dart

```
/// Idioma de la interfaz. Español por defecto; inglés para la exposición.
enum AppLanguage { es, en }

extension AppLanguageX on AppLanguage {
  String get label => this == AppLanguage.es ? 'Español' : 'English';
  String get code => this == AppLanguage.es ? 'es' : 'en';
  bool get isEn => this == AppLanguage.en;
}
```

app_flutter/lib/core/i18n/app_strings.dart

```
import '../settings/user_settings.dart';
import 'app_language.dart';

/// Textos de la interfaz en español/inglés. Español por defecto; el modo inglés
/// se usa en la exposición (rúbrica: "Funcionamiento de la aplicación en inglés").
///
/// Uso: `AppStrings.current.<clave>`. Como `MaterialApp` se reconstruye al cambiar
/// `UserSettings.language`, las pantallas leen siempre el idioma vigente.
class AppStrings {
  const AppStrings(this.language);

  final AppLanguage language;

  static AppStrings get current => AppStrings(UserSettings.instance.language);

  bool get _en => language.isEn;
  String _t(String es, String en) => _en ? en : es;

  // --- Navegación (shell) ---
  String get tabHome => _t('Inicio', 'Home');
  String get tabZones => _t('Zonas', 'Zones');
  String get tabChuno => _t('Chuño', 'Chuño');
  String get tabWeather => _t('Clima', 'Weather');
  String get tabHistory => _t('Historial', 'History');

  // --- Home ---
  String get systemActive => _t('Sistema activo', 'System active');
  String get demoMode => _t('Modo demo', 'Demo mode');
  String get changeTheme => _t('Cambiar tema', 'Change theme');
  String get settings => _t('Ajustes', 'Settings');
  String get appTagline => _t(
    'Prediccion inteligente de heladas para comunidades altoandinas.',
    'Smart frost prediction for high-Andean communities.',
  );
  String get checkRisk => _t('Consultar riesgo', 'Check risk');
  String get viewHistory => _t('Ver historial', 'View history');
  String get dataSources => _t('Fuentes de datos', 'Data sources');
  String get modelInfo => _t('Informacion del modelo', 'Model information');
  String get currentFrostRisk =>
      _t('Riesgo actual de helada', 'Current frost risk');
  String humShort(String v) => _t('Hum $v', 'Hum $v');
  String get maintenanceTitle =>
      _t('Mantenimiento del modelo', 'Model maintenance');
  String get maintenanceBody => _t(
    'El modelo se reentrena de forma programada, registra métricas y solo se promueve si supera el quality gate de silhouette.',
    'The model is retrained on a schedule, logs metrics, and is promoted only if it clears the silhouette quality gate.',
  );

  // --- Tarjetas de alerta (generadas desde campos, localizadas) ---
  String frostTitle(String severity) => switch (severity) {
    'fuerte' => _t('Riesgo de helada fuerte', 'Strong frost risk'),
    'moderada' => _t('Riesgo de helada moderada', 'Moderate frost risk'),
    _ => _t('Sin alerta de helada', 'No frost alert'),
  };
  String frostMessage(String severity) => switch (severity) {
    'fuerte' => _t(
```

```
      'Hoy hay riesgo de helada fuerte. Resguarde el ganado y proteja los cultivos.',
      'Strong frost risk today. Shelter livestock and protect crops.',
    ),
    'moderada' => _t(
      'Hoy hay riesgo de helada moderada. Monitoree la temperatura nocturna.',
      'Moderate frost risk today. Monitor the overnight temperature.',
    ),
    _ => _t(
      'No se prevé helada significativa para hoy.',
      'No significant frost expected today.',
    ),
  };
  String livestockTitle(String level) =>
      _t('Ganado · riesgo $level', '${_livestockLevelEn(level)} livestock risk');
  String _livestockLevelEn(String level) => switch (level) {
    'alto' => 'High',
    'medio' => 'Medium',
    _ => 'Low',
  };
  String livestockMessage(String level) => switch (level) {
    'alto' => _t(
      'Riesgo alto para el ganado: proteja crías de alpaca y ovino esta noche.',
      'High livestock risk: protect alpaca and sheep newborns tonight.',
    ),
    'medio' => _t(
      'Riesgo medio: abrigue a las crías y revise cobertizos antes del anochecer.',
      'Medium risk: shelter the young and check pens before nightfall.',
    ),
    _ => _t(
      'Sin riesgo relevante para el ganado esta noche.',
      'No relevant livestock risk tonight.',
    ),
  };
  String get anomalyTitle => _t('Riesgo inusual', 'Unusual risk');
  String anomalyMessage(double? delta) {
    if (delta == null) {
      return _t('Temperatura dentro de lo normal.', 'Temperature within normal range.');
    }
    final d = delta.abs().toStringAsFixed(1);
    return _t(
      'Helada inusual: $d °C por debajo de lo normal para estas fechas.',
      'Unusual frost: $d °C below normal for this time of year.',
    );
  }

  // --- Zonas / clusters ---
  String get zonesTitle => _t('Zonas de riesgo', 'Risk zones');
  String get zonesSubtitle => _t(
    'Distritos de Puno agrupados por K-Means segun su regimen termico.',
    'Puno districts grouped by K-Means according to their thermal regime.',
  );
  String get clusters => _t('Clusters', 'Clusters');
  String get silhouette => _t('Silhouette', 'Silhouette');
  String get model => _t('Modelo', 'Model');
  String get districtsByLevel => _t('Distritos por nivel', 'Districts by level');
  String coldShare(String v) => _t('frio $v', 'cold $v');
  String get zonesError =>
      _t('No se pudieron cargar las zonas.', 'Could not load the zones.');

  // --- Chuño ---
  String get chunoTitle => _t('Modulo chuño', 'Chuño module');
  String get chunoSubtitle => _t(
    'Ventanas de congelamiento nocturno y secado diurno para elaborar chuño.',
    'Overnight freezing and daytime drying windows to make chuño.',
  );
  String get dailyForecast => _t('Pronostico diario', 'Daily forecast');
  String get optimalWindow => _t('Ventana optima', 'Optimal window');
  String get inSeason => _t('En temporada', 'In season');
  String get offSeason => _t('Fuera de temporada', 'Off season');
  String streak(int d) => _t('Racha ${d}d', 'Streak ${d}d');
  String chunoDayMetrics(String min, String clouds, String hum) => _t(
    'min $min °C · nubes $clouds % · hum $hum %',
    'min $min °C · clouds $clouds % · hum $hum %',
  );
  String get chunoError =>
      _t('No se pudo cargar el modulo de chuño.', 'Could not load the chuño module.');

  // --- Clima histórico ---
  String get weatherTitle => _t('Clima historico', 'Weather history');
  String get weatherSubtitle => _t(
    'Temperaturas minimas/maximas y humedad de los ultimos 30 dias (Open-Meteo Archive).',
    'Min/max temperature and humidity for the last 30 days (Open-Meteo Archive).',
  );
  String get temperatureC => _t('Temperatura (°C)', 'Temperature (°C)');
  String get minLabel => _t('Minima', 'Min');
  String get maxLabel => _t('Maxima', 'Max');
  String get relativeHumidity =>
      _t('Humedad relativa (%)', 'Relative humidity (%)');
  String get weatherError =>
      _t('No se pudo cargar el clima historico.', 'Could not load weather history.');

  // --- Historial ---
  String get historyTitle => _t('Historial', 'History');
  String get historySubtitle => _t(
    'Consultas guardadas con distrito, riesgo y confianza.',
    'Saved queries with district, risk and confidence.',
  );
  String riskChip(String level) =>
      _t('Riesgo ${_cap(level)}', '${riskWord(level)} risk');
  String get confidence => _t('CONFIANZA', 'CONFIDENCE');
  String get date => _t('FECHA', 'DATE');
  String get historyError => _t(
    'No se pudo cargar el historial desde FastAPI.',
    'Could not load history from FastAPI.',
  );
  String get historyEmpty => _t(
    'Aun no hay predicciones guardadas. Con Supabase apagado, FastAPI devuelve una lista vacia.',
    'No saved predictions yet. With Supabase off, FastAPI returns an empty list.',
  );

  // --- Fuentes de datos ---
  String get sourcesTitle => _t('Fuentes de datos', 'Data sources');
  String get sourcesSubtitle => _t(
    'Referencias y origenes de la informacion analizada por el sistema.',
    'References and origins of the data analyzed by the system.',
  );
  String get sourceOpenMeteo => _t(
    'Fuente climatica global usada como respaldo operativo cuando SENAMHI no entrega datos disponibles para el MVP.',
    'Global weather source used as an operational fallback when SENAMHI has no data available for the MVP.',
  );
  String get sourceInei => _t(
```

```
    'Fuente territorial, censal y agropecuaria para ubigeos, distritos, ruralidad y contexto local.',
    'Territorial, census and agricultural source for districts, rurality and local context.',
  );
  String get sourceSenamhi => _t(
    'Fuente oficial peruana prioritaria para estaciones, avisos, validacion y futura calibracion climatica.',
    'Prioritized official Peruvian source for stations, advisories, validation and future calibration.',
  );
  String get sourceMidagri => _t(
    'Fuente agricola complementaria para produccion, cultivos relevantes e impacto productivo.',
    'Complementary agricultural source for production, relevant crops and productive impact.',
  );
  String get sourcesFooter => _t(
    'SENAMHI se prioriza como fuente oficial; si no hay datos operativos disponibles, FrostPuno usa Open-Meteo como fallback.',
    'SENAMHI is prioritized as the official source; when no operational data is available, FrostPuno falls back to Open-Meteo.',
  );
  String get improvementCycle =>
      _t('Ciclo de mejora del modelo', 'Model improvement cycle');
  List<(String, String, String)> get pipelineSteps => _en
      ? const [
          ('1', 'Current weather', 'Open-Meteo feeds the app in real time.'),
          ('2', 'SENAMHI', 'Official observations validate real events.'),
          ('3', 'Feedback', 'Supabase stores corrections and field evidence.'),
          ('4', 'New model', 'ML compares versions before promoting.'),
        ]
      : const [
          ('1', 'Clima actual', 'Open-Meteo alimenta la app en tiempo real.'),
          ('2', 'SENAMHI', 'Observaciones oficiales validan eventos reales.'),
          ('3', 'Feedback', 'Supabase guarda correcciones y evidencia de campo.'),
          ('4', 'Nuevo modelo', 'ML compara versiones antes de promover.'),
        ];
  String get pipelineFooter => _t(
    'Este flujo evita entrenar automaticamente con datos dudosos y mantiene trazabilidad academica.',
    'This flow avoids auto-training on doubtful data and keeps academic traceability.',
  );

  // --- Ajustes ---
  String get settingsSubtitle => _t(
    'Configura tus alarmas de helada y el perfil con el que usas la app.',
    'Set your frost alarms and the profile you use the app with.',
  );
  String get enableAlerts =>
      _t('Activar alertas de helada', 'Enable frost alerts');
  String get enableAlertsSub => _t(
    'Muestra el aviso en Inicio y envía notificación en el celular.',
    'Shows the banner on Home and sends a phone notification.',
  );
  String alertThresholdLabel(String c) => _t(
    'Avisarme si la mínima baja de $c °C',
    'Alert me if the low drops below $c °C',
  );
  String get profile => _t('Perfil', 'Profile');
  String get profileHint =>
      _t('Prioriza qué alertas ves en Inicio.', 'Prioritizes which alerts you see on Home.');
  String get languageLabel => _t('Idioma', 'Language');
  String profileName(String key) => switch (key) {
    'general' => _t('General', 'General'),
    'agricultor' => _t('Agricultor', 'Farmer'),
    'ganadero' => _t('Ganadero', 'Herder'),
    _ => _t('Chuñero', 'Chuño maker'),
  };

  // --- Model info ---
  String get modelInfoTitle => _t('Informacion del modelo', 'Model information');
  String get modelInfoSubtitle => _t(
    'Especificaciones tecnicas y metricas del modelo no supervisado (clustering) en produccion.',
    'Technical specs and metrics of the unsupervised (clustering) model in production.',
  );
  String get baseArchitecture => _t('ARQUITECTURA BASE', 'BASE ARCHITECTURE');
  String get chipClustering => _t('Clustering', 'Clustering');
  String get chipUnsupervised => _t('No supervisado', 'Unsupervised');
  String get mainMetric => _t('METRICA PRINCIPAL', 'MAIN METRIC');
  String get silhouetteDesc => _t(
    'Cohesion y separacion de los grupos climaticos (0 a 1, mas alto es mejor).',
    'Cohesion and separation of the climate groups (0 to 1, higher is better).',
  );
  String get riskGroups => _t('GRUPOS DE RIESGO', 'RISK GROUPS');
  String get clustersUnit => _t('clusters', 'clusters');
  String get clustersDesc => _t(
    'Regimenes climaticos agrupados por K-Means y mapeados a alto/medio/bajo.',
    'Climate regimes grouped by K-Means and mapped to high/medium/low.',
  );
  String get datasetSize => _t('DATASET SIZE', 'DATASET SIZE');
  String get samples => _t('muestras', 'samples');
  String get datasetDesc => _t(
    'Registros generados desde Open-Meteo y contexto territorial inicial.',
    'Records generated from Open-Meteo and initial territorial context.',
  );
  String get versionStatus => _t('VERSION Y ESTADO', 'VERSION & STATUS');
  String get qualityGateApproved =>
      _t('Quality Gate: Aprobado', 'Quality Gate: Passed');
  String get modelInfoError => _t(
    'No se pudo cargar informacion del modelo.',
    'Could not load model information.',
  );

  // --- Predicción (form) ---
  String get frostRiskTitle => _t('Riesgo de helada', 'Frost risk');
  String get frostRiskSubtitle => _t(
    'Ubicacion y clima se obtienen automaticamente para enviar una prediccion lista para campo.',
    'Location and weather are fetched automatically to send a field-ready prediction.',
  );
  String get location => _t('Ubicacion', 'Location');
  String get detectedLocation => _t('Ubicacion detectada', 'Detected location');
  String get punoDemo => _t('Puno demo', 'Puno demo');
  String get locationHint => _t(
    'Usa GPS para detectar tu ubicacion. Si falla, FrostPuno conserva Puno como fallback manual.',
    'Use GPS to detect your location. If it fails, FrostPuno keeps Puno as a manual fallback.',
  );
  String get useMyLocation => _t('Usar mi ubicacion', 'Use my location');
  String get detectingLocation =>
      _t('Detectando ubicacion...', 'Detecting location...');
  String get currentWeather => _t('Clima actual', 'Current weather');
  String get humidity => _t('Humedad', 'Humidity');
  String get wind => _t('Viento', 'Wind');
  String get clouds => _t('Nubes', 'Clouds');
  String get weatherHint => _t(
    'El clima se llenara automaticamente desde FastAPI cuando obtengas tu ubicacion o pulses Obtener clima.',
    'Weather auto-fills from FastAPI once you get your location or tap Get weather.',
  );
  String get getWeather => _t('Obtener clima', 'Get weather');
  String get gettingWeather => _t('Obteniendo clima...', 'Getting weather...');
```

```
  String get readyToPredict => _t('Listo para predecir', 'Ready to predict');
  String get readyToPredictBody => _t(
    'FrostPuno enviara ubicacion, clima actual y contexto agricola interno al modelo.',
    'FrostPuno will send location, current weather and internal agricultural context to the model.',
  );
  String get predict => _t('Predecir', 'Predict');
  String get predicting => _t('Prediciendo...', 'Predicting...');
  String get internalDemoValue => _t('Valor demo interno', 'Internal demo value');
  String feelsLike(String c) => _t('Sensacion $c C', 'Feels like $c C');
  String connectError(Object e) =>
      _t('No se pudo conectar con FastAPI: $e', 'Could not reach FastAPI: $e');
  String get requestingGps =>
      _t('Solicitando permiso y leyendo GPS...', 'Requesting permission and reading GPS...');
  String get waitingLocation => _t(
    'Esperando ubicacion para consultar clima actual.',
    'Waiting for location to fetch current weather.',
  );
  String gpsAccuracy(bool lastKnown, String meters) => _t(
    '${lastKnown ? 'Ultima ubicacion conocida' : 'GPS detectado'} con precision aprox. $meters m.',
    '${lastKnown ? 'Last known location' : 'GPS detected'} with approx. $meters m accuracy.',
  );
  String locationFailed(String detail) => _t(
    '$detail Fallback manual activo: Puno demo.',
    '$detail Manual fallback active: Puno demo.',
  );
  String get locationFailedGeneric => _t(
    'No se pudo obtener ubicacion. Fallback manual activo: Puno demo.',
    'Could not get location. Manual fallback active: Puno demo.',
  );
  String get tapGetWeatherDemo => _t(
    'Puedes pulsar Obtener clima para usar el punto demo de Puno.',
    'Tap Get weather to use the Puno demo point.',
  );
  String get queryingWeather =>
      _t('Consultando GET /weather/current...', 'Calling GET /weather/current...');
  String weatherUpdated(String provider, bool fallback) => _t(
    'Clima actualizado via $provider${fallback ? ' con fallback' : ''}.',
    'Weather updated via $provider${fallback ? ' with fallback' : ''}.',
  );
  String weatherFailed(Object e) => _t(
    'No se pudo obtener clima actual. Se conservan valores demo internos. Detalle: $e',
    'Could not get current weather. Internal demo values kept. Detail: $e',
  );
  String detail(Object e) => _t('Detalle: $e', 'Detail: $e');

  // --- Resultado ---
  String get resultTitle => _t('Resultado', 'Result');
  String get resultSubtitle => _t(
    'Lectura clara para decidir acciones en campo.',
    'Clear read to decide field actions.',
  );
  String get riskLabel => _t('RIESGO', 'RISK');
  String get recommendationFallback => _t(
    'Revisar condiciones locales y monitorear cambios de temperatura.',
    'Review local conditions and monitor temperature changes.',
  );
  String get chunoConditionLabel =>
      _t('CONDICION PARA CHUNO', 'CHUÑO CONDITION');

  /// La API responde en español; en modo inglés se rotula por nivel de riesgo.
  String recommendation(String riskLevel, String backendText) {
    if (!_en) {
      return backendText.trim().isEmpty ? recommendationFallback : backendText;
    }
    return switch (riskLevel.toLowerCase()) {
      'alto' =>
        'High frost risk: shelter livestock and cover crops before nightfall.',
      'medio' || 'moderado' =>
        'Moderate risk: monitor the overnight temperature and prepare protection.',
      _ => 'Low risk: keep routine monitoring for sudden changes.',
    };
  }
  String get summary => _t('RESUMEN', 'SUMMARY');
  String get district => _t('Distrito', 'District');
  String get newQuery => _t('Nueva consulta', 'New query');
  String chunoCondition(String value) => switch (value) {
    'favorables' => _t('Favorable', 'Favorable'),
    'posibles' => _t('Posible', 'Possible'),
    _ => _t('No favorable', 'Unfavorable'),
  };

  // --- Palabras de riesgo (para chips en mayúscula) ---
  String riskWord(String level) => switch (level.toLowerCase()) {
    'alto' => _t('Alto', 'High'),
    'medio' || 'moderado' => _t('Medio', 'Medium'),
    _ => _t('Bajo', 'Low'),
  };
  String tierWord(String tier) => switch (tier.toLowerCase()) {
    'alto' => _t('ALTO', 'HIGH'),
    'medio' => _t('MEDIO', 'MEDIUM'),
    'excelente' => _t('EXCELENTE', 'EXCELLENT'),
    'bueno' => _t('BUENO', 'GOOD'),
    'marginal' => _t('MARGINAL', 'MARGINAL'),
    _ => _t('BAJO', 'LOW'),
  };

  static String _cap(String v) =>
      v.isEmpty ? v : '${v[0].toUpperCase()}${v.substring(1)}';
}
```

app_flutter/lib/core/models/current_weather.dart

```
class CurrentWeather {
  const CurrentWeather({
    required this.latitude,
    required this.longitude,
    required this.provider,
    required this.fallbackUsed,
    this.temperature,
    this.apparentTemperature,
    this.humidity,
    this.windSpeed,
    this.cloudCover,
    this.dewPoint,
    this.precipitation,
    this.observedAt,
```

```
    this.stationName,
    this.stationDistanceKm,
  });

  final double latitude;
  final double longitude;
  final String provider;
  final bool fallbackUsed;
  final double? temperature;
  final double? apparentTemperature;
  final double? humidity;
  final double? windSpeed;
  final double? cloudCover;
  final double? dewPoint;
  final double? precipitation;
  final String? observedAt;
  final String? stationName;
  final double? stationDistanceKm;

  factory CurrentWeather.fromJson(Map<String, dynamic> json) {
    double? n(String key) {
      final value = json[key];
      return value is num ? value.toDouble() : null;
    }

    return CurrentWeather(
      latitude: n('latitude') ?? 0,
      longitude: n('longitude') ?? 0,
      provider: json['provider'] as String? ?? 'desconocido',
      fallbackUsed: json['fallback_used'] as bool? ?? false,
      temperature: n('temperature'),
      apparentTemperature: n('apparent_temperature'),
      humidity: n('humidity'),
      windSpeed: n('wind_speed'),
      cloudCover: n('cloud_cover'),
      dewPoint: n('dew_point'),
      precipitation: n('precipitation'),
      observedAt: json['observed_at'] as String?,
      stationName: json['station_name'] as String?,
      stationDistanceKm: n('station_distance_km'),
    );
  }
}
```

app_flutter/lib/core/models/health_status.dart

```

class HealthStatus {
  const HealthStatus({
    required this.status,
    required this.appName,
    required this.version,
    required this.modelAvailable,
  });

  final String status;
  final String appName;
  final String version;
  final bool modelAvailable;

  factory HealthStatus.fromJson(Map<String, dynamic> json) {
    return HealthStatus(
      status: json['status'] as String? ?? 'unknown',
      appName: json['app_name'] as String? ?? 'FrostPuno API',
      version: json['version'] as String? ?? '-',
      modelAvailable: json['model_available'] as bool? ?? false,
    );
  }
}
```

app_flutter/lib/core/notifications/notification_service.dart

```
import 'package:flutter/foundation.dart' show kIsWeb;
import 'package:flutter_local_notifications/flutter_local_notifications.dart';

/// Notificaciones locales nativas (Android/iOS). En web es no-op:
/// el banner in-app cubre ese caso. Todo va detrás de `kIsWeb`.
class NotificationService {
  static final FlutterLocalNotificationsPlugin _plugin =
      FlutterLocalNotificationsPlugin();
  static bool _ready = false;

  static Future<void> init() async {
    if (kIsWeb) return;
    const android = AndroidInitializationSettings('@mipmap/ic_launcher');
    const settings = InitializationSettings(android: android);
    try {
      await _plugin.initialize(settings);
      final androidImpl = _plugin
          .resolvePlatformSpecificImplementation<
            AndroidFlutterLocalNotificationsPlugin
          >();
      await androidImpl?.requestNotificationsPermission();
      _ready = true;
    } catch (_) {
      _ready = false;
    }
  }

  static Future<void> showFrostAlert(String title, String body) async {
    if (kIsWeb || !_ready) return;
    const details = NotificationDetails(
      android: AndroidNotificationDetails(
        'frost_alerts',
        'Alertas de helada',
        channelDescription: 'Avisos de riesgo de helada de FrostPuno',
        importance: Importance.high,
        priority: Priority.high,
        icon: '@mipmap/ic_launcher',
      ),
    );
```

```
    try {
      await _plugin.show(0, title, body, details);
    } catch (_) {
      // silencioso: la notificación no debe romper la app.
    }
  }
}
```

app_flutter/lib/core/settings/user_settings.dart

```
import 'package:flutter/foundation.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../i18n/app_language.dart';

/// Perfil del usuario: filtra qué alertas se priorizan en Inicio.
enum UserProfile { general, agricultor, ganadero, chunero }

extension UserProfileLabel on UserProfile {
  String get label => switch (this) {
    UserProfile.general => 'General',
    UserProfile.agricultor => 'Agricultor',
    UserProfile.ganadero => 'Ganadero',
    UserProfile.chunero => 'Chuñero',
  };
}

/// Preferencias del usuario persistidas con shared_preferences.
/// Singleton observable (ChangeNotifier) para reaccionar en la UI.
class UserSettings extends ChangeNotifier {
  UserSettings._();
  static final UserSettings instance = UserSettings._();

  static const _kAlertsEnabled = 'alerts_enabled';
  static const _kThreshold = 'alert_threshold';
  static const _kProfile = 'user_profile';
  static const _kLanguage = 'app_language';

  bool alertsEnabled = true;
  double alertThreshold = -4;
  UserProfile profile = UserProfile.general;
  AppLanguage language = AppLanguage.es;

  Future<void> load() async {
    final prefs = await SharedPreferences.getInstance();
    alertsEnabled = prefs.getBool(_kAlertsEnabled) ?? true;
    alertThreshold = prefs.getDouble(_kThreshold) ?? -4;
    final profileIndex = prefs.getInt(_kProfile) ?? 0;
    profile = UserProfile.values[profileIndex.clamp(0, UserProfile.values.length - 1)];
    final langIndex = prefs.getInt(_kLanguage) ?? 0;
    language = AppLanguage.values[langIndex.clamp(0, AppLanguage.values.length - 1)];
    notifyListeners();
  }

  Future<void> setLanguage(AppLanguage value) async {
    language = value;
    notifyListeners();
    final prefs = await SharedPreferences.getInstance();
    await prefs.setInt(_kLanguage, value.index);
  }

  Future<void> setAlertsEnabled(bool value) async {
    alertsEnabled = value;
    notifyListeners();
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool(_kAlertsEnabled, value);
  }

  Future<void> setThreshold(double value) async {
    alertThreshold = value;
    notifyListeners();
    final prefs = await SharedPreferences.getInstance();
    await prefs.setDouble(_kThreshold, value);
  }

  Future<void> setProfile(UserProfile value) async {
    profile = value;
    notifyListeners();
    final prefs = await SharedPreferences.getInstance();
    await prefs.setInt(_kProfile, value.index);
  }
}
```

app_flutter/lib/core/theme/app_colors.dart

```
import 'package:flutter/material.dart';

class AppColors {
  static const deepNavy = Color(0xFF0D1B2A);
  static const iceBlue = Color(0xFFE0FBFC);
  static const mutedTeal = Color(0xFF3D5A80);
  static const softWhite = Color(0xFFFFFFFF);
  static const warmAmber = Color(0xFFEE6C4D);
  static const surface = Color(0xFFF9F9F9);
  static const surfaceLow = Color(0xFFF3F3F4);
  static const surfaceHigh = Color(0xFFE8E8E8);
  static const textPrimary = Color(0xFF1A1C1C);
  static const textSecondary = Color(0xFF44474C);
  static const outline = Color(0x1A3D5A80);
  static const success = Color(0xFF0F8A5F);
  static const deepTeal = Color(0xFF1F6F78);
  static const mediumRisk = Color(0xFFE0A12B);
  static const lowRisk = Color(0xFF0F8A5F);

  static const darkSurface = Color(0xFF101820);
  static const darkSurfaceLow = Color(0xFF16232E);
  static const darkSurfaceHigh = Color(0xFF1E2D38);
  static const darkTextPrimary = Color(0xFFF2F7F8);
  static const darkTextSecondary = Color(0xFFC1CDD2);
  static const darkOutline = Color(0x333CB6C4);
}
```

app_flutter/lib/core/theme/app_theme.dart

```dart
import 'package:flutter/material.dart';

import 'app_colors.dart';

class AppTheme {
  static ThemeData light() {
    return _build(Brightness.light);
  }

  static ThemeData dark() {
    return _build(Brightness.dark);
  }

  static ThemeData _build(Brightness brightness) {
    final isDark = brightness == Brightness.dark;
    final colorScheme = ColorScheme.fromSeed(
      seedColor: AppColors.mutedTeal,
      brightness: brightness,
      primary: isDark ? AppColors.iceBlue : AppColors.deepNavy,
      secondary: AppColors.mutedTeal,
      tertiary: isDark ? AppColors.deepTeal : AppColors.iceBlue,
      error: AppColors.warmAmber,
      surface: isDark ? AppColors.darkSurface : AppColors.surface,
    );
    final surface = isDark ? AppColors.darkSurface : AppColors.surface;
    final textPrimary = isDark
        ? AppColors.darkTextPrimary
        : AppColors.textPrimary;
    final textSecondary = isDark
        ? AppColors.darkTextSecondary
        : AppColors.textSecondary;

    return ThemeData(
      useMaterial3: true,
      colorScheme: colorScheme,
      scaffoldBackgroundColor: surface,
      fontFamily: 'Inter',
      appBarTheme: AppBarTheme(
        centerTitle: true,
        elevation: 0,
        scrolledUnderElevation: 0,
        backgroundColor: surface,
        foregroundColor: textPrimary,
        titleTextStyle: TextStyle(
          color: textPrimary,
          fontSize: 26,
          fontWeight: FontWeight.w700,
        ),
      ),
      textTheme: const TextTheme(
        displayLarge: TextStyle(
          fontSize: 32,
          height: 1.2,
          fontWeight: FontWeight.w700,
        ),
        headlineMedium: TextStyle(
          fontSize: 24,
          height: 1.25,
          fontWeight: FontWeight.w700,
        ),
        titleMedium: TextStyle(
          fontSize: 18,
          height: 1.3,
          fontWeight: FontWeight.w700,
        ),
        bodyLarge: TextStyle(
          fontSize: 16,
          height: 1.5,
          fontWeight: FontWeight.w400,
        ),
        bodyMedium: TextStyle(
          fontSize: 14,
          height: 1.45,
          fontWeight: FontWeight.w400,
        ),
        labelSmall: TextStyle(
          fontSize: 12,
          height: 1.35,
          fontWeight: FontWeight.w500,
        ),
      ).apply(bodyColor: textPrimary, displayColor: textPrimary),
      inputDecorationTheme: InputDecorationTheme(
        filled: true,
        fillColor: isDark ? AppColors.darkSurfaceHigh : AppColors.softWhite,
        border: const UnderlineInputBorder(
          borderSide: BorderSide(color: AppColors.mutedTeal),
        ),
        enabledBorder: const UnderlineInputBorder(
          borderSide: BorderSide(color: AppColors.mutedTeal),
        ),
        focusedBorder: UnderlineInputBorder(
          borderSide: BorderSide(
            color: isDark ? AppColors.iceBlue : AppColors.deepNavy,
            width: 1.4,
          ),
        ),
        contentPadding: const EdgeInsets.symmetric(horizontal: 4, vertical: 10),
      ),
      filledButtonTheme: FilledButtonThemeData(
        style: FilledButton.styleFrom(
          backgroundColor: isDark ? AppColors.iceBlue : AppColors.deepNavy,
          foregroundColor: isDark ? AppColors.deepNavy : AppColors.softWhite,
          disabledBackgroundColor:
              (isDark ? AppColors.darkSurfaceHigh : AppColors.surfaceHigh)
                  .withValues(alpha: 0.8),
          disabledForegroundColor: textSecondary,
        ),
      ),
    );
  }
}
```

app_flutter/lib/core/theme/theme_mode_scope.dart

```dart
import 'package:flutter/material.dart';

class ThemeModeScope extends InheritedWidget {
  const ThemeModeScope({
    required this.themeMode,
    required this.onThemeModeChanged,
    required super.child,
    super.key,
  });

  final ThemeMode themeMode;
  final ValueChanged<ThemeMode> onThemeModeChanged;

  static ThemeModeScope? maybeOf(BuildContext context) {
    return context.dependOnInheritedWidgetOfExactType<ThemeModeScope>();
  }

  static ThemeModeScope of(BuildContext context) {
    final scope = maybeOf(context);
    assert(scope != null, 'ThemeModeScope not found in context');
    return scope!;
  }

  @override
  bool updateShouldNotify(ThemeModeScope oldWidget) {
    return themeMode != oldWidget.themeMode ||
        onThemeModeChanged != oldWidget.onThemeModeChanged;
  }
}
```

app_flutter/lib/features/alerts/models/daily_alert.dart

```dart
class LivestockRisk {
  LivestockRisk({required this.level, required this.message});

  final String level;
  final String message;

  factory LivestockRisk.fromJson(Map<String, dynamic> json) => LivestockRisk(
    level: json['level'] as String? ?? 'bajo',
    message: json['message'] as String? ?? '',
  );
}

class ClimateAnomaly {
  ClimateAnomaly({
    required this.isUnusual,
    required this.message,
    this.historicalMean,
    this.delta,
  });

  final bool isUnusual;
  final String message;
  final double? historicalMean;
  final double? delta;

  factory ClimateAnomaly.fromJson(Map<String, dynamic> json) => ClimateAnomaly(
    isUnusual: json['is_unusual'] as bool? ?? false,
    message: json['message'] as String? ?? '',
    historicalMean: (json['historical_mean'] as num?)?.toDouble(),
    delta: (json['delta'] as num?)?.toDouble(),
  );
}

class DailyAlert {
  DailyAlert({
    required this.riskLevel,
    required this.frostAlert,
    required this.severity,
    required this.title,
    required this.message,
    required this.livestock,
    required this.anomaly,
    this.temperatureMin,
  });

  final String riskLevel;
  final bool frostAlert;
  final String severity; // fuerte | moderada | ninguna
  final String title;
  final String message;
  final double? temperatureMin;
  final LivestockRisk livestock;
  final ClimateAnomaly anomaly;

  factory DailyAlert.fromJson(Map<String, dynamic> json) => DailyAlert(
    riskLevel: json['risk_level'] as String,
    frostAlert: json['frost_alert'] as bool,
    severity: json['severity'] as String,
    title: json['title'] as String,
    message: json['message'] as String,
    temperatureMin: (json['temperature_min'] as num?)?.toDouble(),
    livestock: LivestockRisk.fromJson(
      (json['livestock'] as Map<String, dynamic>?) ?? const {},
    ),
    anomaly: ClimateAnomaly.fromJson(
      (json['anomaly'] as Map<String, dynamic>?) ?? const {},
    ),
  );
}
```

app_flutter/lib/features/charts/models/history_day.dart

```dart
class HistoryDay {
  HistoryDay({
    required this.date,
    this.temperatureMin,
    this.temperatureMax,
    this.humidityMean,
  });
```

```dart

  final String date;
  final double? temperatureMin;
  final double? temperatureMax;
  final double? humidityMean;

  factory HistoryDay.fromJson(Map<String, dynamic> json) => HistoryDay(
    date: json['date'] as String,
    temperatureMin: (json['temperature_min'] as num?)?.toDouble(),
    temperatureMax: (json['temperature_max'] as num?)?.toDouble(),
    humidityMean: (json['humidity_mean'] as num?)?.toDouble(),
  );
}

class WeatherHistory {
  WeatherHistory({required this.days});

  final List<HistoryDay> days;

  factory WeatherHistory.fromJson(Map<String, dynamic> json) => WeatherHistory(
    days: (json['days'] as List)
        .whereType<Map<String, dynamic>>()
        .map(HistoryDay.fromJson)
        .toList(),
  );
}
```

app_flutter/lib/features/charts/screens/charts_screen.dart

```dart
import 'package:fl_chart/fl_chart.dart';
import 'package:flutter/material.dart';

import '../../../core/api/frost_api_service.dart';
import '../../../core/i18n/app_strings.dart';
import '../../../core/theme/app_colors.dart';
import '../../../shared/widgets/glass_card.dart';
import '../../../shared/widgets/responsive_content.dart';
import '../../../shared/widgets/section_header.dart';
import '../models/history_day.dart';

const _defaultLat = -15.8402;
const _defaultLon = -70.0219;

String _fmt(DateTime d) =>
    '${d.year.toString().padLeft(4, '0')}-${d.month.toString().padLeft(2, '0')}-${d.day.toString().padLeft(2, '0')}';

class ChartsScreen extends StatefulWidget {
  const ChartsScreen({required this.apiService, super.key});

  final FrostApiService apiService;

  @override
  State<ChartsScreen> createState() => _ChartsScreenState();
}

class _ChartsScreenState extends State<ChartsScreen> {
  late Future<WeatherHistory> _future = _load();

  Future<WeatherHistory> _load() {
    // Open-Meteo Archive tiene ~5 dias de retraso; pedimos 30 dias hasta hace 6.
    final end = DateTime.now().subtract(const Duration(days: 6));
    final start = end.subtract(const Duration(days: 29));
    return widget.apiService.getWeatherHistory(
      latitude: _defaultLat,
      longitude: _defaultLon,
      start: _fmt(start),
      end: _fmt(end),
    );
  }

  @override
  Widget build(BuildContext context) {
    return SafeArea(
      child: ResponsiveContent(
        child: FutureBuilder<WeatherHistory>(
          future: _future,
          builder: (context, snapshot) {
            if (snapshot.connectionState == ConnectionState.waiting) {
              return const Center(child: CircularProgressIndicator());
            }
            if (snapshot.hasError) {
              return _Error(message: snapshot.error.toString());
            }
            final days = snapshot.data!.days;
            final s = AppStrings.current;
            return RefreshIndicator(
              onRefresh: () async {
                setState(() => _future = _load());
                await _future;
              },
              child: ListView(
                padding: const EdgeInsets.fromLTRB(20, 28, 20, 24),
                children: [
                  SectionHeader(
                    title: s.weatherTitle,
                    subtitle: s.weatherSubtitle,
                  ),
                  const SizedBox(height: 20),
                  GlassCard(
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Text(
                          s.temperatureC,
                          style: Theme.of(context).textTheme.titleMedium,
                        ),
                        const SizedBox(height: 8),
                        _Legend(
                          items: [
                            (s.minLabel, AppColors.mutedTeal),
                            (s.maxLabel, AppColors.warmAmber),
                          ],
                        ),
                        const SizedBox(height: 16),
                        SizedBox(
                          height: 220,
```

```
                    child: LineChart(_tempChart(context, days)),
                  ),
                ],
              ),
            ),
            const SizedBox(height: 20),
            GlassCard(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    s.relativeHumidity,
                    style: Theme.of(context).textTheme.titleMedium,
                  ),
                  const SizedBox(height: 16),
                  SizedBox(
                    height: 200,
                    child: LineChart(_humidityChart(context, days)),
                  ),
                ],
              ),
            ),
          ],
        ),
      );
    },
  ),
 );
}

List<FlSpot> _spots(List<HistoryDay> days, double? Function(HistoryDay) pick) {
 final spots = <FlSpot>[];
 for (var i = 0; i < days.length; i++) {
  final value = pick(days[i]);
  if (value != null) spots.add(FlSpot(i.toDouble(), value));
 }
 return spots;
}

LineChartData _tempChart(BuildContext context, List<HistoryDay> days) {
 return LineChartData(
  gridData: const FlGridData(show: true, drawVerticalLine: false),
  titlesData: _titles(context, days),
  borderData: FlBorderData(show: false),
  lineBarsData: [
   _line(_spots(days, (d) => d.temperatureMin), AppColors.mutedTeal),
   _line(_spots(days, (d) => d.temperatureMax), AppColors.warmAmber),
  ],
 );
}

LineChartData _humidityChart(BuildContext context, List<HistoryDay> days) {
 return LineChartData(
  gridData: const FlGridData(show: true, drawVerticalLine: false),
  titlesData: _titles(context, days),
  borderData: FlBorderData(show: false),
  lineBarsData: [
   _line(_spots(days, (d) => d.humidityMean), AppColors.deepTeal),
  ],
 );
}

LineChartBarData _line(List<FlSpot> spots, Color color) => LineChartBarData(
 spots: spots,
 isCurved: true,
 color: color,
 barWidth: 2.5,
 dotData: const FlDotData(show: false),
);

FlTitlesData _titles(BuildContext context, List<HistoryDay> days) {
 final axisColor = Theme.of(
  context,
 ).colorScheme.onSurface.withValues(alpha: 0.62);
 return FlTitlesData(
  show: true,
  topTitles: const AxisTitles(sideTitles: SideTitles(showTitles: false)),
  rightTitles: const AxisTitles(sideTitles: SideTitles(showTitles: false)),
  leftTitles: AxisTitles(
   sideTitles: SideTitles(
    showTitles: true,
    reservedSize: 36,
    getTitlesWidget: (value, meta) => Text(
     value.round().toString(),
     style: TextStyle(fontSize: 11, color: axisColor),
    ),
   ),
  ),
  bottomTitles: AxisTitles(
   sideTitles: SideTitles(
    showTitles: true,
    interval: (days.length / 4).clamp(1, 30).toDouble(),
    getTitlesWidget: (value, meta) {
     final index = value.toInt();
     if (index < 0 || index >= days.length) {
      return const SizedBox.shrink();
     }
     final label = days[index].date;
     return Padding(
      padding: const EdgeInsets.only(top: 6),
      child: Text(
       label.length >= 10 ? label.substring(5) : label,
       style: TextStyle(fontSize: 10, color: axisColor),
      ),
     );
    },
   ),
  ),
 );
}
}

class _Legend extends StatelessWidget {
 const _Legend({required this.items});

 final List<(String, Color)> items;

 @override
 Widget build(BuildContext context) {
  return Wrap(
```

```
      spacing: 16,
      children: items
          .map(
            (item) => Row(
              mainAxisSize: MainAxisSize.min,
              children: [
                Container(width: 12, height: 12, color: item.$2),
                const SizedBox(width: 6),
                Text(item.$1, style: Theme.of(context).textTheme.bodySmall),
              ],
            ),
          )
          .toList(),
    );
  }
}

class _Error extends StatelessWidget {
  const _Error({required this.message});

  final String message;

  @override
  Widget build(BuildContext context) {
    return ListView(
      padding: const EdgeInsets.all(24),
      children: [
        const SizedBox(height: 80),
        const Icon(Icons.show_chart, size: 48, color: AppColors.warmAmber),
        const SizedBox(height: 16),
        Text(
          AppStrings.current.weatherError,
          textAlign: TextAlign.center,
          style: Theme.of(context).textTheme.titleMedium,
        ),
        const SizedBox(height: 8),
        Text(
          message,
          textAlign: TextAlign.center,
          style: Theme.of(context).textTheme.bodySmall,
        ),
      ],
    );
  }
}
```

app_flutter/lib/features/chuno/models/chuno_window.dart

```
class ChunoDay {
  ChunoDay({
    required this.date,
    required this.tier,
    required this.isGoodDay,
    this.temperatureMin,
    this.humidityMax,
    this.cloudCover,
    this.precipitation,
  });

  final String date;
  final String tier;
  final bool isGoodDay;
  final double? temperatureMin;
  final double? humidityMax;
  final double? cloudCover;
  final double? precipitation;

  factory ChunoDay.fromJson(Map<String, dynamic> json) => ChunoDay(
    date: json['date'] as String,
    tier: json['tier'] as String,
    isGoodDay: json['is_good_day'] as bool,
    temperatureMin: (json['temperature_min'] as num?)?.toDouble(),
    humidityMax: (json['humidity_max'] as num?)?.toDouble(),
    cloudCover: (json['cloud_cover_mean'] as num?)?.toDouble(),
    precipitation: (json['precipitation_sum'] as num?)?.toDouble(),
  );
}

class ChunoWindow {
  ChunoWindow({
    required this.inSeason,
    required this.optimalWindow,
    required this.bestStreak,
    required this.message,
    required this.days,
  });

  final bool inSeason;
  final bool optimalWindow;
  final int bestStreak;
  final String message;
  final List<ChunoDay> days;

  factory ChunoWindow.fromJson(Map<String, dynamic> json) => ChunoWindow(
    inSeason: json['in_season'] as bool,
    optimalWindow: json['optimal_window'] as bool,
    bestStreak: (json['best_streak'] as num).toInt(),
    message: json['message'] as String,
    days: (json['days'] as List)
        .whereType<Map<String, dynamic>>()
        .map(ChunoDay.fromJson)
        .toList(),
  );
}
```

app_flutter/lib/features/chuno/screens/chuno_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/api/frost_api_service.dart';
import '../../../core/i18n/app_strings.dart';
```

```dart
import '../../../core/theme/app_colors.dart';
import '../../../shared/widgets/glass_card.dart';
import '../../../shared/widgets/responsive_content.dart';
import '../../../shared/widgets/section_header.dart';
import '../../../shared/widgets/status_chip.dart';
import '../models/chuno_window.dart';

// Puno demo por defecto; en produccion la pantalla recibe GPS del usuario.
const _defaultLat = -15.8402;
const _defaultLon = -70.0219;

Color _dayColor(String tier) => switch (tier) {
  'excelente' => AppColors.deepTeal,
  'bueno' => AppColors.lowRisk,
  'marginal' => AppColors.mediumRisk,
  _ => AppColors.warmAmber,
};

class ChunoScreen extends StatefulWidget {
  const ChunoScreen({required this.apiService, super.key});

  final FrostApiService apiService;

  @override
  State<ChunoScreen> createState() => _ChunoScreenState();
}

class _ChunoScreenState extends State<ChunoScreen> {
  late Future<ChunoWindow> _future = _load();

  Future<ChunoWindow> _load() => widget.apiService.getChunoWindow(
    latitude: _defaultLat,
    longitude: _defaultLon,
  );

  @override
  Widget build(BuildContext context) {
    return SafeArea(
      child: ResponsiveContent(
        child: FutureBuilder<ChunoWindow>(
          future: _future,
          builder: (context, snapshot) {
            if (snapshot.connectionState == ConnectionState.waiting) {
              return const Center(child: CircularProgressIndicator());
            }
            if (snapshot.hasError) {
              return _Error(message: snapshot.error.toString());
            }
            final data = snapshot.data!;
            final s = AppStrings.current;
            return RefreshIndicator(
              onRefresh: () async {
                setState(() => _future = _load());
                await _future;
              },
              child: ListView(
                padding: const EdgeInsets.fromLTRB(20, 28, 20, 24),
                children: [
                  SectionHeader(
                    title: s.chunoTitle,
                    subtitle: s.chunoSubtitle,
                  ),
                  const SizedBox(height: 20),
                  _SeasonCard(data: data),
                  const SizedBox(height: 24),
                  Text(
                    s.dailyForecast,
                    style: Theme.of(context).textTheme.titleMedium,
                  ),
                  const SizedBox(height: 12),
                  ...data.days.map((d) => _DayTile(day: d)),
                ],
              ),
            );
          },
        ),
      ),
    );
  }
}

class _SeasonCard extends StatelessWidget {
  const _SeasonCard({required this.data});

  final ChunoWindow data;

  @override
  Widget build(BuildContext context) {
    final onSurface = Theme.of(context).colorScheme.onSurface;
    final s = AppStrings.current;
    final color = data.optimalWindow
        ? AppColors.deepTeal
        : (data.inSeason ? AppColors.mediumRisk : AppColors.mutedTeal);
    return GlassCard(
      padding: const EdgeInsets.all(22),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Icon(Icons.ac_unit, color: color, size: 34),
              const SizedBox(width: 12),
              Expanded(
                child: Text(
                  data.optimalWindow
                      ? s.optimalWindow
                      : data.inSeason
                      ? s.inSeason
                      : s.offSeason,
                  style: Theme.of(context).textTheme.titleLarge,
                ),
              ),
              StatusChip(
                label: s.streak(data.bestStreak),
                color: color,
              ),
            ],
          ),
          const SizedBox(height: 14),
          Text(
            data.message,
```

```
          style: Theme.of(context).textTheme.bodyMedium?.copyWith(
            color: onSurface.withValues(alpha: 0.72),
          ),
        ),
      ],
    ),
  );
 }
}

class _DayTile extends StatelessWidget {
 const _DayTile({required this.day});

 final ChunoDay day;

 @override
 Widget build(BuildContext context) {
  final color = _dayColor(day.tier);
  return Padding(
   padding: const EdgeInsets.only(bottom: 10),
   child: GlassCard(
    padding: const EdgeInsets.all(16),
    child: Row(
     children: [
      Icon(
       day.isGoodDay ? Icons.check_circle : Icons.remove_circle_outline,
       color: color,
      ),
      const SizedBox(width: 14),
      Expanded(
       child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
         Text(day.date, style: Theme.of(context).textTheme.titleSmall),
         const SizedBox(height: 2),
         Text(
          AppStrings.current.chunoDayMetrics(
           day.temperatureMin?.toStringAsFixed(1) ?? '—',
           day.cloudCover?.toStringAsFixed(0) ?? '—',
           day.humidityMax?.toStringAsFixed(0) ?? '—',
          ),
          style: Theme.of(context).textTheme.bodySmall,
         ),
        ],
       ),
      ),
      StatusChip(
       label: AppStrings.current.tierWord(day.tier),
       color: color,
      ),
     ],
    ),
   ),
  );
 }
}

class _Error extends StatelessWidget {
 const _Error({required this.message});

 final String message;

 @override
 Widget build(BuildContext context) {
  return ListView(
   padding: const EdgeInsets.all(24),
   children: [
    const SizedBox(height: 80),
    const Icon(Icons.cloud_off, size: 48, color: AppColors.warmAmber),
    const SizedBox(height: 16),
    Text(
     AppStrings.current.chunoError,
     textAlign: TextAlign.center,
     style: Theme.of(context).textTheme.titleMedium,
    ),
    const SizedBox(height: 8),
    Text(
     message,
     textAlign: TextAlign.center,
     style: Theme.of(context).textTheme.bodySmall,
    ),
   ],
  );
 }
}
```

app_flutter/lib/features/clusters/models/cluster_info.dart

```
class ClusterProfile {
 ClusterProfile({
  required this.clusterId,
  required this.tier,
  required this.size,
  required this.temperature,
  required this.dewPoint,
 });

 final int clusterId;
 final String tier;
 final int size;
 final double temperature;
 final double dewPoint;

 factory ClusterProfile.fromJson(Map<String, dynamic> json) => ClusterProfile(
  clusterId: (json['cluster_id'] as num).toInt(),
  tier: json['tier'] as String,
  size: (json['size'] as num).toInt(),
  temperature: (json['temperature_2m'] as num).toDouble(),
  dewPoint: (json['dew_point_2m'] as num).toDouble(),
 );
}

class DistrictCluster {
 DistrictCluster({
  required this.distrito,
  required this.tier,
```

```
    required this.dominantCluster,
    required this.coldShare,
    required this.altitude,
    required this.temperatureMean,
  });

  final String distrito;
  final String tier;
  final int dominantCluster;
  final double coldShare;
  final double altitude;
  final double temperatureMean;

  factory DistrictCluster.fromJson(Map<String, dynamic> json) =>
      DistrictCluster(
        distrito: json['distrito'] as String,
        tier: json['tier'] as String,
        dominantCluster: (json['dominant_cluster'] as num).toInt(),
        coldShare: (json['cold_share'] as num).toDouble(),
        altitude: (json['altitud_estimada'] as num).toDouble(),
        temperatureMean: (json['temperature_2m_mean'] as num).toDouble(),
      );
}

class ClustersResult {
  ClustersResult({
    required this.modelName,
    required this.version,
    required this.nClusters,
    required this.silhouette,
    required this.profiles,
    required this.districts,
  });

  final String modelName;
  final String version;
  final int nClusters;
  final double silhouette;
  final List<ClusterProfile> profiles;
  final List<DistrictCluster> districts;

  factory ClustersResult.fromJson(Map<String, dynamic> json) => ClustersResult(
    modelName: json['model_name'] as String,
    version: json['version'] as String,
    nClusters: (json['n_clusters'] as num).toInt(),
    silhouette: (json['silhouette'] as num).toDouble(),
    profiles: (json['profiles'] as List)
        .whereType<Map<String, dynamic>>()
        .map(ClusterProfile.fromJson)
        .toList(),
    districts: (json['districts'] as List)
        .whereType<Map<String, dynamic>>()
        .map(DistrictCluster.fromJson)
        .toList(),
  );
}
```

app_flutter/lib/features/clusters/screens/clusters_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/api/frost_api_service.dart';
import '../../../core/i18n/app_strings.dart';
import '../../../core/theme/app_colors.dart';
import '../../../shared/widgets/glass_card.dart';
import '../../../shared/widgets/responsive_content.dart';
import '../../../shared/widgets/section_header.dart';
import '../../../shared/widgets/status_chip.dart';
import '../models/cluster_info.dart';

Color tierColor(String tier) => switch (tier) {
  'alto' => AppColors.warmAmber,
  'medio' => AppColors.mediumRisk,
  _ => AppColors.lowRisk,
};

class ClustersScreen extends StatefulWidget {
  const ClustersScreen({required this.apiService, super.key});

  final FrostApiService apiService;

  @override
  State<ClustersScreen> createState() => _ClustersScreenState();
}

class _ClustersScreenState extends State<ClustersScreen> {
  late Future<ClustersResult> _future = widget.apiService.getClusters();

  @override
  Widget build(BuildContext context) {
    return SafeArea(
      child: ResponsiveContent(
        child: RefreshIndicator(
          onRefresh: () async {
            setState(() => _future = widget.apiService.getClusters());
            await _future;
          },
          child: FutureBuilder<ClustersResult>(
            future: _future,
            builder: (context, snapshot) {
              if (snapshot.connectionState == ConnectionState.waiting) {
                return const Center(child: CircularProgressIndicator());
              }
              if (snapshot.hasError) {
                return _ErrorView(message: snapshot.error.toString());
              }
              final s = AppStrings.current;
              final data = snapshot.data!;
              return ListView(
                padding: const EdgeInsets.fromLTRB(20, 28, 20, 24),
                children: [
                  SectionHeader(
                    title: s.zonesTitle,
                    subtitle: s.zonesSubtitle,
                  ),
                  const SizedBox(height: 20),
```

```
              GlassCard(
                child: Row(
                  children: [
                    Expanded(
                      child: _Metric(
                        label: s.clusters,
                        value: '${data.nClusters}',
                      ),
                    ),
                    Expanded(
                      child: _Metric(
                        label: s.silhouette,
                        value: data.silhouette.toStringAsFixed(3),
                      ),
                    ),
                    Expanded(
                      child: _Metric(
                        label: s.model,
                        value: data.modelName,
                      ),
                    ),
                  ],
                ),
              ),
              const SizedBox(height: 24),
              Text(
                s.districtsByLevel,
                style: Theme.of(context).textTheme.titleMedium,
              ),
              const SizedBox(height: 12),
              ...data.districts.map((d) => _DistrictTile(district: d)),
            ],
          );
        },
      ),
    ),
  );
 }
}

class _DistrictTile extends StatelessWidget {
 const _DistrictTile({required this.district});

 final DistrictCluster district;

 @override
 Widget build(BuildContext context) {
  final color = tierColor(district.tier);
  return Padding(
   padding: const EdgeInsets.only(bottom: 12),
   child: GlassCard(
    padding: const EdgeInsets.all(18),
    child: Row(
     children: [
      Container(
       width: 10,
       height: 44,
       decoration: BoxDecoration(
        color: color,
        borderRadius: BorderRadius.circular(6),
       ),
      ),
      const SizedBox(width: 16),
      Expanded(
       child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
         Text(
          district.distrito,
          style: Theme.of(context).textTheme.titleMedium,
         ),
         const SizedBox(height: 4),
         Text(
          '${district.altitude.toStringAsFixed(0)} m · ⌀${district.temperatureMean.toStringAsFixed(1)} °C',
          style: Theme.of(context).textTheme.bodyMedium?.copyWith(
           color: Theme.of(
            context,
           ).colorScheme.onSurface.withValues(alpha: 0.66),
          ),
         ),
        ],
       ),
      ),
      Column(
       crossAxisAlignment: CrossAxisAlignment.end,
       children: [
        StatusChip(
         label: AppStrings.current.tierWord(district.tier),
         color: color,
        ),
        const SizedBox(height: 6),
        Text(
         AppStrings.current.coldShare(
          '${(district.coldShare * 100).toStringAsFixed(0)}%',
         ),
         style: Theme.of(context).textTheme.bodySmall,
        ),
       ],
      ),
     ],
    ),
   ),
  );
 }
}

class _Metric extends StatelessWidget {
 const _Metric({required this.label, required this.value});

 final String label;
 final String value;

 @override
 Widget build(BuildContext context) {
  return Column(
   children: [
    Text(value, style: Theme.of(context).textTheme.titleLarge),
    const SizedBox(height: 4),
    Text(label, style: Theme.of(context).textTheme.bodySmall),
   ],
```

```
    );
  }
}

class _ErrorView extends StatelessWidget {
  const _ErrorView({required this.message});

  final String message;

  @override
  Widget build(BuildContext context) {
    return ListView(
      padding: const EdgeInsets.all(24),
      children: [
        const SizedBox(height: 80),
        const Icon(Icons.cloud_off, size: 48, color: AppColors.warmAmber),
        const SizedBox(height: 16),
        Text(
          AppStrings.current.zonesError,
          textAlign: TextAlign.center,
          style: Theme.of(context).textTheme.titleMedium,
        ),
        const SizedBox(height: 8),
        Text(
          message,
          textAlign: TextAlign.center,
          style: Theme.of(context).textTheme.bodySmall,
        ),
      ],
    );
  }
}
```

app_flutter/lib/features/data_sources/models/data_source_info.dart

```
import 'package:flutter/material.dart';

class DataSourceInfo {
  const DataSourceInfo({
    required this.name,
    required this.tag,
    required this.description,
    required this.icon,
  });

  final String name;
  final String tag;
  final String description;
  final IconData icon;
}
```

app_flutter/lib/features/data_sources/screens/data_sources_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/i18n/app_strings.dart';
import '../../../core/theme/app_colors.dart';
import '../../../shared/widgets/glass_card.dart';
import '../../../shared/widgets/page_scaffold.dart';
import '../../../shared/widgets/responsive_content.dart';
import '../../../shared/widgets/section_header.dart';
import '../../../shared/widgets/status_chip.dart';
import '../models/data_source_info.dart';

class DataSourcesScreen extends StatelessWidget {
  const DataSourcesScreen({super.key, this.showAppBar = true});

  final bool showAppBar;

  static List<DataSourceInfo> sourcesFor(AppStrings s) => [
    DataSourceInfo(
      name: 'Open-Meteo',
      tag: 'FALLBACK',
      description: s.sourceOpenMeteo,
      icon: Icons.cloud_outlined,
    ),
    DataSourceInfo(
      name: 'INEI',
      tag: 'TERRITORIO',
      description: s.sourceInei,
      icon: Icons.map_outlined,
    ),
    DataSourceInfo(
      name: 'SENAMHI',
      tag: 'OFICIAL',
      description: s.sourceSenamhi,
      icon: Icons.device_thermostat,
    ),
    DataSourceInfo(
      name: 'MIDAGRI/SIEA',
      tag: 'AGRO',
      description: s.sourceMidagri,
      icon: Icons.agriculture_outlined,
    ),
  ];

  @override
  Widget build(BuildContext context) {
    final s = AppStrings.current;
    final sources = sourcesFor(s);
    final body = ListView(
      padding: const EdgeInsets.fromLTRB(24, 28, 24, 32),
      children: [
        SectionHeader(
          title: s.sourcesTitle,
          subtitle: s.sourcesSubtitle,
        ),
        const SizedBox(height: 28),
        for (final source in sources) ...[
          _SourceCard(source: source),
          const SizedBox(height: 18),
```

```
        ],
        GlassCard(
          child: Row(
            children: [
              Icon(
                Icons.info_outline,
                color: Theme.of(
                  context,
                ).colorScheme.onSurface.withValues(alpha: 0.66),
              ),
              const SizedBox(width: 14),
              Expanded(
                child: Text(
                  s.sourcesFooter,
                  style: Theme.of(context).textTheme.bodyLarge?.copyWith(
                    color: Theme.of(
                      context,
                    ).colorScheme.onSurface.withValues(alpha: 0.68),
                    fontStyle: FontStyle.italic,
                  ),
                ),
              ),
            ],
          ),
        ),
        const SizedBox(height: 18),
        const _ValidationPipelineCard(),
      ],
    );

    if (!showAppBar) {
      return SafeArea(child: ResponsiveContent(child: body));
    }
    return PageScaffold(child: body);
  }
}

class _ValidationPipelineCard extends StatelessWidget {
  const _ValidationPipelineCard();

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;
    final onSurface = Theme.of(context).colorScheme.onSurface;
    final s = AppStrings.current;
    final steps = s.pipelineSteps;
    return GlassCard(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              const Icon(Icons.timeline, color: AppColors.warmAmber),
              const SizedBox(width: 12),
              Expanded(
                child: Text(
                  s.improvementCycle,
                  style: textTheme.headlineMedium,
                ),
              ),
            ],
          ),
          const SizedBox(height: 16),
          for (final step in steps) ...[
            _PipelineStep(
              number: step.$1,
              title: step.$2,
              description: step.$3,
            ),
            if (step != steps.last) const SizedBox(height: 12),
          ],
          const SizedBox(height: 16),
          Text(
            s.pipelineFooter,
            style: textTheme.bodyMedium?.copyWith(
              color: onSurface.withValues(alpha: 0.66),
              fontStyle: FontStyle.italic,
            ),
          ),
        ],
      ),
    );
  }
}

class _PipelineStep extends StatelessWidget {
  const _PipelineStep({
    required this.number,
    required this.title,
    required this.description,
  });

  final String number;
  final String title;
  final String description;

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;
    return Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        CircleAvatar(
          radius: 14,
          backgroundColor: AppColors.mutedTeal,
          child: Text(number, style: textTheme.labelSmall),
        ),
        const SizedBox(width: 12),
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(title, style: textTheme.titleMedium),
              Text(description, style: textTheme.bodyMedium),
            ],
          ),
        ),
      ],
    );
  }
}
```

```

class _SourceCard extends StatelessWidget {
  const _SourceCard({required this.source});

  final DataSourceInfo source;

  @override
  Widget build(BuildContext context) {
    return GlassCard(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Icon(source.icon, color: AppColors.mutedTeal, size: 30),
              const SizedBox(width: 12),
              Text(
                source.name,
                style: Theme.of(context).textTheme.headlineMedium,
              ),
            ],
          ),
          const SizedBox(height: 14),
          StatusChip(label: source.tag),
          const SizedBox(height: 14),
          Text(
            source.description,
            style: Theme.of(context).textTheme.bodyLarge,
          ),
        ],
      ),
    );
  }
}
```

app_flutter/lib/features/history/models/prediction_history_item.dart

```
class PredictionHistoryItem {
  const PredictionHistoryItem({
    required this.id,
    required this.district,
    required this.province,
    required this.latitude,
    required this.longitude,
    required this.riskLevel,
    required this.confidence,
    required this.chunoConditions,
    required this.recommendation,
    required this.modelVersion,
    required this.dataSources,
    required this.createdAt,
    this.populatedCenter,
    this.altitude,
  });

  final String id;
  final String district;
  final String province;
  final String? populatedCenter;
  final double latitude;
  final double longitude;
  final double? altitude;
  final String riskLevel;
  final double confidence;
  final String chunoConditions;
  final String recommendation;
  final String modelVersion;
  final List<String> dataSources;
  final String createdAt;

  factory PredictionHistoryItem.fromJson(Map<String, dynamic> json) {
    return PredictionHistoryItem(
      id: json['id'] as String? ?? '-',
      district: json['district'] as String? ?? '-',
      province: json['province'] as String? ?? '-',
      populatedCenter: json['populated_center'] as String?,
      latitude: (json['latitude'] as num? ?? 0).toDouble(),
      longitude: (json['longitude'] as num? ?? 0).toDouble(),
      altitude: (json['altitude'] as num?)?.toDouble(),
      riskLevel: json['risk_level'] as String? ?? 'bajo',
      confidence: (json['confidence'] as num? ?? 0).toDouble(),
      chunoConditions: json['chuno_conditions'] as String? ?? '-',
      recommendation: json['recommendation'] as String? ?? '',
      modelVersion: json['model_version'] as String? ?? '-',
      dataSources: (json['data_sources'] as List<dynamic>? ?? const [])
          .map((item) => item.toString())
          .toList(),
      createdAt: json['created_at'] as String? ?? '-',
    );
  }
}
```

app_flutter/lib/features/history/screens/history_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/api/frost_api_service.dart';
import '../../../core/i18n/app_strings.dart';
import '../../../core/theme/app_colors.dart';
import '../../../shared/widgets/glass_card.dart';
import '../../../shared/widgets/page_scaffold.dart';
import '../../../shared/widgets/responsive_content.dart';
import '../../../shared/widgets/section_header.dart';
import '../../../shared/widgets/status_chip.dart';
import '../models/prediction_history_item.dart';

class HistoryScreen extends StatefulWidget {
  const HistoryScreen({
    required this.apiService,
    super.key,
    this.showAppBar = true,
```

```
  });

  final FrostApiService apiService;
  final bool showAppBar;

  @override
  State<HistoryScreen> createState() => _HistoryScreenState();
}

class _HistoryScreenState extends State<HistoryScreen> {
  late Future<List<PredictionHistoryItem>> _future = widget.apiService
      .getPredictionHistory();

  @override
  Widget build(BuildContext context) {
    final s = AppStrings.current;
    final body = RefreshIndicator(
      onRefresh: () async {
        setState(() => _future = widget.apiService.getPredictionHistory());
        await _future;
      },
      child: ListView(
        padding: const EdgeInsets.fromLTRB(24, 28, 24, 32),
        children: [
          SectionHeader(
            title: s.historyTitle,
            subtitle: s.historySubtitle,
          ),
          const SizedBox(height: 26),
          FutureBuilder<List<PredictionHistoryItem>>(
            future: _future,
            builder: (context, snapshot) {
              if (snapshot.connectionState == ConnectionState.waiting) {
                return const Center(
                  child: Padding(
                    padding: EdgeInsets.all(32),
                    child: CircularProgressIndicator(),
                  ),
                );
              }
              if (snapshot.hasError) {
                return _EmptyHistory(message: s.historyError);
              }
              final items = snapshot.data ?? const [];
              if (items.isEmpty) {
                return _EmptyHistory(message: s.historyEmpty);
              }
              return Column(
                children: items
                    .map((item) => _HistoryCard(item: item))
                    .toList(),
              );
            },
          ),
        ],
      ),
    );

    if (!widget.showAppBar) {
      return SafeArea(child: ResponsiveContent(child: body));
    }
    return PageScaffold(child: body);
  }
}

class _HistoryCard extends StatelessWidget {
  const _HistoryCard({required this.item});

  final PredictionHistoryItem item;

  @override
  Widget build(BuildContext context) {
    final color = _riskColor(item.riskLevel);
    final icon = _riskIcon(item.riskLevel);
    final onSurface = Theme.of(context).colorScheme.onSurface;
    return Padding(
      padding: const EdgeInsets.only(bottom: 18),
      child: GlassCard(
        padding: const EdgeInsets.all(18),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Expanded(
                  child: Text(
                    item.district,
                    style: Theme.of(
                      context,
                    ).textTheme.headlineMedium?.copyWith(color: onSurface),
                  ),
                ),
                StatusChip(
                  icon: icon,
                  label: AppStrings.current.riskChip(item.riskLevel),
                  color: color,
                  background: color.withValues(alpha: 0.14),
                ),
              ],
            ),
            const SizedBox(height: 8),
            Text(
              _formatDate(item.createdAt),
              style: TextStyle(
                fontFamily: 'monospace',
                color: onSurface.withValues(alpha: 0.66),
              ),
            ),
            const SizedBox(height: 18),
            const Divider(),
            const SizedBox(height: 14),
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                _SmallMetric(
                  label: AppStrings.current.confidence,
                  value: '${(item.confidence * 100).round()}%',
                ),
                _SmallMetric(
                  label: AppStrings.current.date,
                  value: _shortDate(item.createdAt),
```

```
            ),
           ],
          ),
         ],
        ),
      ),
    );
  }

  static Color _riskColor(String value) {
    return switch (value.toLowerCase()) {
      'alto' => AppColors.warmAmber,
      'medio' => AppColors.mediumRisk,
      'moderado' => AppColors.mediumRisk,
      _ => AppColors.lowRisk,
    };
  }

  static IconData _riskIcon(String value) {
    return switch (value.toLowerCase()) {
      'alto' => Icons.warning_amber_rounded,
      'medio' => Icons.report_problem_outlined,
      'moderado' => Icons.report_problem_outlined,
      _ => Icons.check_circle_outline,
    };
  }

  static String _formatDate(String raw) {
    final date = DateTime.tryParse(raw);
    if (date == null) return raw;
    return '${_two(date.day)}/${_two(date.month)}/${date.year} ${_two(date.hour)}:${_two(date.minute)}';
  }

  static String _shortDate(String raw) {
    final date = DateTime.tryParse(raw);
    if (date == null) return raw;
    return '${_two(date.day)}/${_two(date.month)}';
  }

  static String _two(int value) => value.toString().padLeft(2, '0');
}

class _SmallMetric extends StatelessWidget {
  const _SmallMetric({required this.label, required this.value});

  final String label;
  final String value;

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          label,
          style: Theme.of(context).textTheme.labelSmall?.copyWith(
            fontFamily: 'monospace',
            letterSpacing: 0,
          ),
        ),
        const SizedBox(height: 8),
        Text(
          value,
          style: TextStyle(
            fontFamily: 'monospace',
            fontSize: 20,
            fontWeight: FontWeight.w700,
            color: Theme.of(context).colorScheme.onSurface,
          ),
        ),
      ],
    );
  }
}

class _EmptyHistory extends StatelessWidget {
  const _EmptyHistory({required this.message});

  final String message;

  @override
  Widget build(BuildContext context) {
    return GlassCard(
      child: Column(
        children: [
          Icon(
            Icons.history_toggle_off,
            size: 42,
            color: Theme.of(
              context,
            ).colorScheme.onSurface.withValues(alpha: 0.56),
          ),
          const SizedBox(height: 14),
          Text(
            message,
            textAlign: TextAlign.center,
            style: Theme.of(context).textTheme.bodyLarge,
          ),
        ],
      ),
    );
  }
}
```

app_flutter/lib/features/home/screens/home_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/api/frost_api_service.dart';
import '../../../core/i18n/app_strings.dart';
import '../../../core/models/health_status.dart';
import '../../../core/notifications/notification_service.dart';
import '../../../core/settings/user_settings.dart';
import '../../../core/theme/app_colors.dart';
import '../../../core/theme/theme_mode_scope.dart';
import '../../alerts/models/daily_alert.dart';
```

```dart
import '../../settings/screens/settings_screen.dart';
import '../../../shared/widgets/glass_card.dart';
import '../../../shared/widgets/primary_action_button.dart';
import '../../../shared/widgets/responsive_content.dart';
import '../../../shared/widgets/status_chip.dart';
import '../../data_sources/screens/data_sources_screen.dart';
import '../../history/screens/history_screen.dart';
import '../../model_info/screens/model_info_screen.dart';
import '../../prediction/screens/prediction_form_screen.dart';

class HomeScreen extends StatefulWidget {
 const HomeScreen({required this.apiService, super.key});

 final FrostApiService apiService;

 @override
 State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
 static const _defaultLat = -15.8402;
 static const _defaultLon = -70.0219;

 late final Future<HealthStatus> _healthFuture = widget.apiService.getHealth();
 late final Future<DailyAlert> _alertFuture = widget.apiService.getDailyAlert(
  latitude: _defaultLat,
  longitude: _defaultLon,
 );

 @override
 Widget build(BuildContext context) {
  final textTheme = Theme.of(context).textTheme;
  final onSurface = Theme.of(context).colorScheme.onSurface;
  final themeScope = ThemeModeScope.maybeOf(context);
  final s = AppStrings.current;
  return SafeArea(
   child: ResponsiveContent(
    child: ListView(
     padding: const EdgeInsets.fromLTRB(20, 28, 20, 24),
     children: [
      Row(
       children: [
        Expanded(
         child: FutureBuilder<HealthStatus>(
          future: _healthFuture,
          builder: (context, snapshot) {
           final active = snapshot.data?.modelAvailable ?? false;
           return StatusChip(
            icon: Icons.circle,
            label: active ? s.systemActive : s.demoMode,
            color: active
                ? AppColors.mutedTeal
                : AppColors.warmAmber,
           );
          },
         ),
        ),
        IconButton(
         tooltip: s.changeTheme,
         icon: Icon(_themeIcon(themeScope?.themeMode)),
         onPressed: themeScope == null
             ? null
             : () => themeScope.onThemeModeChanged(
                _nextThemeMode(themeScope.themeMode),
               ),
        ),
        IconButton(
         tooltip: s.settings,
         icon: const Icon(Icons.tune),
         onPressed: () => Navigator.of(context).push(
          MaterialPageRoute(builder: (_) => const SettingsScreen()),
         ),
        ),
       ],
      ),
      const SizedBox(height: 32),
      Text('FrostPuno', style: textTheme.displayLarge),
      const SizedBox(height: 16),
      Text(
       s.appTagline,
       style: textTheme.headlineMedium?.copyWith(
        color: onSurface.withValues(alpha: 0.68),
        fontSize: 22,
        fontWeight: FontWeight.w400,
       ),
      ),
      const SizedBox(height: 24),
      _AlertSection(future: _alertFuture),
      const SizedBox(height: 20),
      const _CurrentRiskCard(),
      const SizedBox(height: 32),
      PrimaryActionButton(
       label: s.checkRisk,
       icon: Icons.arrow_forward,
       onPressed: () => Navigator.of(context).push(
        MaterialPageRoute(
         builder: (_) =>
             PredictionFormScreen(apiService: widget.apiService),
        ),
       ),
      ),
      const SizedBox(height: 18),
      Row(
       children: [
        Expanded(
         child: _QuickTile(
          icon: Icons.history,
          label: s.viewHistory,
          onTap: () => Navigator.of(context).push(
           MaterialPageRoute(
            builder: (_) =>
                HistoryScreen(apiService: widget.apiService),
           ),
          ),
         ),
        ),
        const SizedBox(width: 16),
        Expanded(
         child: _QuickTile(
          icon: Icons.storage_outlined,
          label: s.dataSources,
```

```
                onTap: () => Navigator.of(context).push(
                  MaterialPageRoute(
                    builder: (_) => const DataSourcesScreen(),
                  ),
                ),
              ),
            ),
          ],
        ),
        const SizedBox(height: 16),
        _QuickTile(
          icon: Icons.account_tree_outlined,
          label: s.modelInfo,
          onTap: () => Navigator.of(context).push(
            MaterialPageRoute(
              builder: (_) =>
                  ModelInfoScreen(apiService: widget.apiService),
            ),
          ),
        ),
        const SizedBox(height: 18),
        const _LearningCycleCard(),
      ],
    ),
  ),
 );
 }

 static ThemeMode _nextThemeMode(ThemeMode? mode) {
  return switch (mode ?? ThemeMode.system) {
   ThemeMode.system => ThemeMode.light,
   ThemeMode.light => ThemeMode.dark,
   ThemeMode.dark => ThemeMode.system,
  };
 }

 static IconData _themeIcon(ThemeMode? mode) {
  return switch (mode ?? ThemeMode.system) {
   ThemeMode.system => Icons.brightness_auto_outlined,
   ThemeMode.light => Icons.light_mode_outlined,
   ThemeMode.dark => Icons.dark_mode_outlined,
  };
 }
}

Color _levelColor(String level) => switch (level) {
 'alto' => AppColors.warmAmber,
 'medio' => AppColors.mediumRisk,
 _ => AppColors.lowRisk,
};

class _AlertSection extends StatefulWidget {
 const _AlertSection({required this.future});

 final Future<DailyAlert> future;

 @override
 State<_AlertSection> createState() => _AlertSectionState();
}

class _AlertSectionState extends State<_AlertSection> {
 bool _notified = false;

 bool _shouldAlert(DailyAlert alert, UserSettings settings) {
  if (!settings.alertsEnabled) return false;
  if (alert.frostAlert) return true;
  final tmin = alert.temperatureMin;
  return tmin != null && tmin <= settings.alertThreshold;
 }

 @override
 Widget build(BuildContext context) {
  final settings = UserSettings.instance;
  return FutureBuilder<DailyAlert>(
   future: widget.future,
   builder: (context, snapshot) {
    if (!snapshot.hasData) return const SizedBox.shrink();
    final alert = snapshot.data!;
    return AnimatedBuilder(
     animation: settings,
     builder: (context, _) {
      final active = _shouldAlert(alert, settings);
      final s = AppStrings.current;
      if (active && !_notified) {
       _notified = true;
       WidgetsBinding.instance.addPostFrameCallback((_) {
        NotificationService.showFrostAlert(
         s.frostTitle(alert.severity),
         s.frostMessage(alert.severity),
        );
       });
      }

      final profile = settings.profile;
      final showLivestock =
         profile == UserProfile.general ||
         profile == UserProfile.ganadero;
      final cards = <Widget>[];

      if (settings.alertsEnabled) {
       cards.add(
        _FrostBanner(alert: alert, active: active, settings: settings),
       );
      }
      if (showLivestock && alert.livestock.level != 'bajo') {
       cards.add(_LivestockCard(livestock: alert.livestock));
      }
      if (alert.anomaly.isUnusual) {
       cards.add(_AnomalyCard(anomaly: alert.anomaly));
      }
      if (cards.isEmpty) return const SizedBox.shrink();

      return Column(
       children: [
        for (var i = 0; i < cards.length; i++) ...[
         if (i > 0) const SizedBox(height: 12),
         cards[i],
        ],
       ],
      );
     },
```

```
        );
      },
    );
  }
}

class _FrostBanner extends StatelessWidget {
  const _FrostBanner({
    required this.alert,
    required this.active,
    required this.settings,
  });

  final DailyAlert alert;
  final bool active;
  final UserSettings settings;

  @override
  Widget build(BuildContext context) {
    final color = active ? _levelColor(alert.riskLevel) : AppColors.lowRisk;
    final icon = active ? Icons.warning_amber_rounded : Icons.check_circle_outline;
    return GlassCard(
      padding: const EdgeInsets.all(18),
      color: color.withValues(alpha: 0.14),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Icon(icon, color: color, size: 30),
          const SizedBox(width: 14),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  AppStrings.current.frostTitle(alert.severity),
                  style: Theme.of(context).textTheme.titleMedium?.copyWith(
                    color: color,
                    fontWeight: FontWeight.w700,
                  ),
                ),
                const SizedBox(height: 6),
                Text(
                  AppStrings.current.frostMessage(alert.severity),
                  style: Theme.of(context).textTheme.bodyMedium,
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

class _LivestockCard extends StatelessWidget {
  const _LivestockCard({required this.livestock});

  final LivestockRisk livestock;

  @override
  Widget build(BuildContext context) {
    final color = _levelColor(livestock.level);
    return GlassCard(
      padding: const EdgeInsets.all(18),
      color: color.withValues(alpha: 0.12),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Icon(Icons.pets, color: color, size: 28),
          const SizedBox(width: 14),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  AppStrings.current.livestockTitle(livestock.level),
                  style: Theme.of(context).textTheme.titleMedium?.copyWith(
                    color: color,
                    fontWeight: FontWeight.w700,
                  ),
                ),
                const SizedBox(height: 6),
                Text(
                  AppStrings.current.livestockMessage(livestock.level),
                  style: Theme.of(context).textTheme.bodyMedium,
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

class _AnomalyCard extends StatelessWidget {
  const _AnomalyCard({required this.anomaly});

  final ClimateAnomaly anomaly;

  @override
  Widget build(BuildContext context) {
    return GlassCard(
      padding: const EdgeInsets.all(18),
      color: AppColors.deepTeal.withValues(alpha: 0.12),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Icon(Icons.insights, color: AppColors.deepTeal, size: 28),
          const SizedBox(width: 14),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  AppStrings.current.anomalyTitle,
                  style: Theme.of(context).textTheme.titleMedium?.copyWith(
                    color: AppColors.deepTeal,
                    fontWeight: FontWeight.w700,
                  ),
                ),
              ),
```

```
              const SizedBox(height: 6),
              Text(
                AppStrings.current.anomalyMessage(anomaly.delta),
                style: Theme.of(context).textTheme.bodyMedium,
              ),
            ],
          ),
        ),
      ],
    ),
  );
 }
}

class _CurrentRiskCard extends StatelessWidget {
 const _CurrentRiskCard();

 @override
 Widget build(BuildContext context) {
  final textTheme = Theme.of(context).textTheme;
  final onSurface = Theme.of(context).colorScheme.onSurface;
  return GlassCard(
   padding: const EdgeInsets.all(26),
   child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
     Row(
      children: [
       Expanded(
        child: Text(
         AppStrings.current.currentFrostRisk,
         style: textTheme.headlineMedium,
        ),
       ),
       const Icon(Icons.ac_unit, color: AppColors.mutedTeal, size: 42),
      ],
     ),
     const SizedBox(height: 34),
     Row(
      crossAxisAlignment: CrossAxisAlignment.end,
      children: [
       Text(
        '-4.2',
        style: textTheme.displayLarge?.copyWith(fontSize: 54),
       ),
       const SizedBox(width: 10),
       Padding(
        padding: const EdgeInsets.only(bottom: 8),
        child: Text(
         'C',
         style: textTheme.titleMedium?.copyWith(
          color: onSurface.withValues(alpha: 0.68),
         ),
        ),
       ),
      ],
     ),
     const SizedBox(height: 26),
     Wrap(
      spacing: 14,
      runSpacing: 10,
      children: [
       StatusChip(
        icon: Icons.water_drop_outlined,
        label: AppStrings.current.humShort('42 %'),
       ),
       const StatusChip(icon: Icons.air, label: '12 km/h'),
      ],
     ),
    ],
   ),
  ),
 );
 }
}

class _LearningCycleCard extends StatelessWidget {
 const _LearningCycleCard();

 @override
 Widget build(BuildContext context) {
  final textTheme = Theme.of(context).textTheme;
  final onSurface = Theme.of(context).colorScheme.onSurface;
  return GlassCard(
   padding: const EdgeInsets.all(22),
   child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
     Row(
      children: [
       const Icon(Icons.model_training, color: AppColors.warmAmber),
       const SizedBox(width: 12),
       Expanded(
        child: Text(
         AppStrings.current.maintenanceTitle,
         style: textTheme.titleMedium,
        ),
       ),
       const StatusChip(label: 'K-Means'),
      ],
     ),
     const SizedBox(height: 14),
     Text(
      AppStrings.current.maintenanceBody,
      style: textTheme.bodyMedium?.copyWith(
       color: onSurface.withValues(alpha: 0.72),
      ),
     ),
    ],
   ),
  );
 }
}

class _QuickTile extends StatelessWidget {
 const _QuickTile({
  required this.icon,
  required this.label,
  required this.onTap,
 });

 final IconData icon;
```

```
  final String label;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return InkWell(
      borderRadius: BorderRadius.circular(8),
      onTap: onTap,
      child: Container(
        constraints: const BoxConstraints(minHeight: 128),
        padding: const EdgeInsets.all(22),
        decoration: BoxDecoration(
          color: Theme.of(
            context,
          ).colorScheme.onSurface.withValues(alpha: 0.06),
          borderRadius: BorderRadius.circular(8),
          border: Border.all(
            color: Theme.of(
              context,
            ).colorScheme.onSurface.withValues(alpha: 0.08),
          ),
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Icon(
              icon,
              color: Theme.of(
                context,
              ).colorScheme.onSurface.withValues(alpha: 0.66),
              size: 32,
            ),
            Text(
              label,
              maxLines: 2,
              overflow: TextOverflow.ellipsis,
              style: Theme.of(
                context,
              ).textTheme.titleMedium?.copyWith(fontSize: 17),
            ),
          ],
        ),
      ),
    );
  }
}
```

app_flutter/lib/features/location/models/location_state.dart

```
class DetectedLocation {
  const DetectedLocation({
    required this.latitude,
    required this.longitude,
    required this.accuracy,
    this.fromLastKnownPosition = false,
  });

  final double latitude;
  final double longitude;
  final double accuracy;
  final bool fromLastKnownPosition;
}

enum LocationFailureType { serviceDisabled, denied, permanentlyDenied, unknown }

class LocationFailure implements Exception {
  const LocationFailure(this.type, this.message);

  final LocationFailureType type;
  final String message;

  @override
  String toString() => message;
}
```

app_flutter/lib/features/location/services/location_service.dart

```
import 'dart:async';

import 'package:geolocator/geolocator.dart';

import '../models/location_state.dart';

class LocationService {
  Future<DetectedLocation> getCurrentLocation() async {
    final serviceEnabled = await Geolocator.isLocationServiceEnabled();
    if (!serviceEnabled) {
      throw const LocationFailure(
        LocationFailureType.serviceDisabled,
        'Activa el GPS del dispositivo para usar tu ubicacion actual.',
      );
    }

    var permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
      permission = await Geolocator.requestPermission();
    }
    if (permission == LocationPermission.deniedForever) {
      throw const LocationFailure(
        LocationFailureType.permanentlyDenied,
        'El permiso de ubicacion esta bloqueado. Activalo desde ajustes del sistema.',
      );
    }
    if (permission == LocationPermission.denied ||
        permission == LocationPermission.unableToDetermine) {
      throw const LocationFailure(
        LocationFailureType.denied,
        'Permiso de ubicacion denegado.',
      );
    }
```

```
34
35      try {
36        final position = await Geolocator.getCurrentPosition(
37          locationSettings: const LocationSettings(
38            accuracy: LocationAccuracy.high,
39            timeLimit: Duration(seconds: 12),
40          ),
41        );
42
43        return DetectedLocation(
44          latitude: position.latitude,
45          longitude: position.longitude,
46          accuracy: position.accuracy,
47        );
48      } on TimeoutException {
49        final lastKnown = await Geolocator.getLastKnownPosition();
50        if (lastKnown == null) {
51          throw const LocationFailure(
52            LocationFailureType.unknown,
53            'El GPS no respondio a tiempo.',
54          );
55        }
56        return DetectedLocation(
57          latitude: lastKnown.latitude,
58          longitude: lastKnown.longitude,
59          accuracy: lastKnown.accuracy,
60          fromLastKnownPosition: true,
61        );
62      }
63    }
64  }
```

app_flutter/lib/features/model_info/models/model_info.dart

```
1   class ModelInfo {
2     const ModelInfo({
3       required this.modelName,
4       required this.version,
5       required this.createdAt,
6       required this.features,
7       required this.target,
8       required this.metrics,
9       required this.datasetSize,
10      required this.dataSources,
11      required this.limitations,
12      this.nClusters,
13    });
14
15    final String modelName;
16    final String version;
17    final String createdAt;
18    final List<String> features;
19    final String target;
20    final Map<String, double> metrics;
21    final int datasetSize;
22    final List<String> dataSources;
23    final List<String> limitations;
24    final int? nClusters;
25
26    factory ModelInfo.fromJson(Map<String, dynamic> json) {
27      final rawMetrics = json['metrics'] as Map<String, dynamic>? ?? const {};
28      return ModelInfo(
29        modelName: json['model_name'] as String? ?? '-',
30        version: json['version'] as String? ?? '-',
31        createdAt: json['created_at'] as String? ?? '-',
32        features: (json['features'] as List<dynamic>? ?? const [])
33            .map((item) => item.toString())
34            .toList(),
35        target: json['target'] as String? ?? '-',
36        metrics: rawMetrics.map(
37          (key, value) => MapEntry(key, (value as num).toDouble()),
38        ),
39        datasetSize: json['dataset_size'] as int? ?? 0,
40        dataSources: (json['data_sources'] as List<dynamic>? ?? const [])
41            .map((item) => item.toString())
42            .toList(),
43        limitations: (json['limitations'] as List<dynamic>? ?? const [])
44            .map((item) => item.toString())
45            .toList(),
46        nClusters: (json['n_clusters'] as num?)?.toInt(),
47      );
48    }
49  }
```

app_flutter/lib/features/model_info/screens/model_info_screen.dart

```
1   import 'package:flutter/material.dart';
2
3   import '../../../core/api/frost_api_service.dart';
4   import '../../../core/i18n/app_strings.dart';
5   import '../../../core/theme/app_colors.dart';
6   import '../../../shared/widgets/glass_card.dart';
7   import '../../../shared/widgets/page_scaffold.dart';
8   import '../../../shared/widgets/section_header.dart';
9   import '../../../shared/widgets/status_chip.dart';
10  import '../models/model_info.dart';
11
12  class ModelInfoScreen extends StatelessWidget {
13    const ModelInfoScreen({required this.apiService, super.key});
14
15    final FrostApiService apiService;
16
17    @override
18    Widget build(BuildContext context) {
19      return PageScaffold(
20        child: FutureBuilder<ModelInfo>(
21          future: apiService.getModelInfo(),
22          builder: (context, snapshot) {
23            if (snapshot.connectionState == ConnectionState.waiting) {
24              return const Center(child: CircularProgressIndicator());
25            }
```

```
        final s = AppStrings.current;
        if (snapshot.hasError) {
          return Center(child: Text(s.modelInfoError));
        }
        final info = snapshot.data!;
        final silhouette = info.metrics['silhouette'] ?? 0;
        return ListView(
          padding: const EdgeInsets.fromLTRB(24, 28, 24, 32),
          children: [
            SectionHeader(
              title: s.modelInfoTitle,
              subtitle: s.modelInfoSubtitle,
            ),
            const SizedBox(height: 30),
            GlassCard(
              color: AppColors.softWhite,
              child: Row(
                children: [
                  Expanded(
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        _MonoLabel(s.baseArchitecture),
                        const SizedBox(height: 12),
                        Text(
                          info.modelName,
                          style: Theme.of(context).textTheme.headlineMedium,
                        ),
                        const SizedBox(height: 20),
                        Wrap(
                          spacing: 10,
                          runSpacing: 10,
                          children: [
                            StatusChip(label: s.chipClustering),
                            StatusChip(label: s.chipUnsupervised),
                            const StatusChip(label: 'Scikit-Learn'),
                          ],
                        ),
                      ],
                    ),
                  ),
                  const CircleAvatar(
                    radius: 38,
                    backgroundColor: AppColors.surfaceLow,
                    child: Icon(
                      Icons.account_tree_outlined,
                      color: AppColors.deepNavy,
                      size: 34,
                    ),
                  ),
                ],
              ),
            ),
            const SizedBox(height: 18),
            _MetricPanel(
              icon: Icons.analytics_outlined,
              label: s.mainMetric,
              value: silhouette.toStringAsFixed(3),
              suffix: 'silhouette',
              description: s.silhouetteDesc,
            ),
            const SizedBox(height: 18),
            _MetricPanel(
              icon: Icons.hub_outlined,
              label: s.riskGroups,
              value: (info.nClusters ?? 0).toString(),
              suffix: s.clustersUnit,
              description: s.clustersDesc,
            ),
            const SizedBox(height: 18),
            _MetricPanel(
              icon: Icons.dataset_outlined,
              label: s.datasetSize,
              value: info.datasetSize.toString(),
              suffix: s.samples,
              description: s.datasetDesc,
            ),
            const SizedBox(height: 18),
            GlassCard(
              color: AppColors.softWhite,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  _MonoLabel(s.versionStatus),
                  const SizedBox(height: 14),
                  Text(
                    info.version,
                    style: const TextStyle(
                      fontFamily: 'monospace',
                      fontSize: 28,
                      fontWeight: FontWeight.w700,
                    ),
                  ),
                  const SizedBox(height: 18),
                  Container(
                    padding: const EdgeInsets.all(18),
                    decoration: BoxDecoration(
                      color: AppColors.success.withValues(alpha: 0.10),
                      borderRadius: BorderRadius.circular(12),
                      border: Border.all(
                        color: AppColors.success.withValues(alpha: 0.24),
                      ),
                    ),
                    child: Row(
                      children: [
                        const Icon(
                          Icons.check_circle_outline,
                          color: AppColors.success,
                        ),
                        const SizedBox(width: 12),
                        Expanded(
                          child: Text(
                            s.qualityGateApproved,
                            style: const TextStyle(
                              color: AppColors.success,
                              fontSize: 18,
                              fontWeight: FontWeight.w700,
                            ),
                          ),
                        ),
                      ],
```

```
                ),
               ),
              ],
             ),
            ),
           ],
          );
        },
      ),
    );
  }
}

class _MetricPanel extends StatelessWidget {
  const _MetricPanel({
    required this.icon,
    required this.label,
    required this.value,
    required this.suffix,
    required this.description,
  });

  final IconData icon;
  final String label;
  final String value;
  final String suffix;
  final String description;

  @override
  Widget build(BuildContext context) {
    return GlassCard(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Icon(icon, color: AppColors.textSecondary),
              const SizedBox(width: 10),
              _MonoLabel(label),
            ],
          ),
          const SizedBox(height: 16),
          RichText(
            text: TextSpan(
              style: const TextStyle(
                color: AppColors.deepNavy,
                fontFamily: 'monospace',
                fontSize: 28,
                fontWeight: FontWeight.w700,
              ),
              children: [
                TextSpan(text: value),
                TextSpan(
                  text: ' $suffix',
                  style: const TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w500,
                  ),
                ),
              ],
            ),
          ),
          const SizedBox(height: 12),
          const Divider(),
          Text(
            description,
            style: Theme.of(
              context,
            ).textTheme.bodyLarge?.copyWith(color: AppColors.textSecondary),
          ),
        ],
      ),
    );
  }
}

class _MonoLabel extends StatelessWidget {
  const _MonoLabel(this.text);

  final String text;

  @override
  Widget build(BuildContext context) {
    return Text(
      text,
      style: Theme.of(context).textTheme.labelSmall?.copyWith(
        fontFamily: 'monospace',
        letterSpacing: 1.4,
        color: AppColors.textSecondary,
        fontWeight: FontWeight.w700,
      ),
    );
  }
}
```

app_flutter/lib/features/prediction/models/frost_risk_request.dart

```
class FrostRiskRequest {
  const FrostRiskRequest({
    required this.district,
    required this.province,
    required this.populatedCenter,
    required this.latitude,
    required this.longitude,
    required this.altitude,
    required this.ruralPopulation,
    required this.agriculturalActivity,
    required this.mainCrop,
    required this.temperatureMin,
    required this.temperatureMax,
    required this.feelsLike,
    required this.humidity,
    required this.windSpeed,
    required this.cloudCover,
    required this.dewPoint,
    required this.precipitation,
```

```
    required this.month,
    required this.hour,
    required this.hoursBelowZero,
    this.totalPopulation,
    this.ruralPercentage,
  });

  final String district;
  final String province;
  final String populatedCenter;
  final double latitude;
  final double longitude;
  final double altitude;
  final int ruralPopulation;
  final int? totalPopulation;
  final double? ruralPercentage;
  final bool agriculturalActivity;
  final String mainCrop;
  final double temperatureMin;
  final double temperatureMax;
  final double feelsLike;
  final double humidity;
  final double windSpeed;
  final double cloudCover;
  final double dewPoint;
  final double precipitation;
  final int month;
  final int hour;
  final int hoursBelowZero;

  factory FrostRiskRequest.demo() {
    return const FrostRiskRequest(
      district: 'Puno',
      province: 'Puno',
      populatedCenter: 'Centro poblado demo',
      latitude: -15.8402,
      longitude: -70.0219,
      altitude: 3827,
      ruralPopulation: 1200,
      totalPopulation: 5000,
      ruralPercentage: 24,
      agriculturalActivity: true,
      mainCrop: 'papa',
      temperatureMin: -2.5,
      temperatureMax: 12.4,
      feelsLike: -4.0,
      humidity: 68,
      windSpeed: 7,
      cloudCover: 20,
      dewPoint: -3.5,
      precipitation: 0,
      month: 6,
      hour: 3,
      hoursBelowZero: 4,
    );
  }

  FrostRiskRequest copyWith({
    String? district,
    String? province,
    String? populatedCenter,
    double? latitude,
    double? longitude,
    double? altitude,
    int? ruralPopulation,
    int? totalPopulation,
    double? ruralPercentage,
    bool? agriculturalActivity,
    String? mainCrop,
    double? temperatureMin,
    double? temperatureMax,
    double? feelsLike,
    double? humidity,
    double? windSpeed,
    double? cloudCover,
    double? dewPoint,
    double? precipitation,
    int? month,
    int? hour,
    int? hoursBelowZero,
  }) {
    return FrostRiskRequest(
      district: district ?? this.district,
      province: province ?? this.province,
      populatedCenter: populatedCenter ?? this.populatedCenter,
      latitude: latitude ?? this.latitude,
      longitude: longitude ?? this.longitude,
      altitude: altitude ?? this.altitude,
      ruralPopulation: ruralPopulation ?? this.ruralPopulation,
      totalPopulation: totalPopulation ?? this.totalPopulation,
      ruralPercentage: ruralPercentage ?? this.ruralPercentage,
      agriculturalActivity: agriculturalActivity ?? this.agriculturalActivity,
      mainCrop: mainCrop ?? this.mainCrop,
      temperatureMin: temperatureMin ?? this.temperatureMin,
      temperatureMax: temperatureMax ?? this.temperatureMax,
      feelsLike: feelsLike ?? this.feelsLike,
      humidity: humidity ?? this.humidity,
      windSpeed: windSpeed ?? this.windSpeed,
      cloudCover: cloudCover ?? this.cloudCover,
      dewPoint: dewPoint ?? this.dewPoint,
      precipitation: precipitation ?? this.precipitation,
      month: month ?? this.month,
      hour: hour ?? this.hour,
      hoursBelowZero: hoursBelowZero ?? this.hoursBelowZero,
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'district': district,
      'province': province,
      'populated_center': populatedCenter,
      'latitude': latitude,
      'longitude': longitude,
      'altitude': altitude,
      'rural_population': ruralPopulation,
      if (totalPopulation != null) 'total_population': totalPopulation,
      if (ruralPercentage != null) 'rural_percentage': ruralPercentage,
      'agricultural_activity': agriculturalActivity,
      'main_crop': mainCrop,
      'temperature_min': temperatureMin,
```

```
      'temperature_max': temperatureMax,
      'feels_like': feelsLike,
      'humidity': humidity,
      'wind_speed': windSpeed,
      'cloud_cover': cloudCover,
      'dew_point': dewPoint,
      'precipitation': precipitation,
      'month': month,
      'hour': hour,
      'hours_below_zero': hoursBelowZero,
    };
  }
}
```

app_flutter/lib/features/prediction/models/frost_risk_response.dart

```dart
class FrostRiskResponse {
  const FrostRiskResponse({
    required this.riskLevel,
    required this.confidence,
    required this.recommendation,
    required this.chunoConditions,
    required this.modelVersion,
    required this.dataSources,
  });

  final String riskLevel;
  final double confidence;
  final String recommendation;
  final String chunoConditions;
  final String modelVersion;
  final List<String> dataSources;

  factory FrostRiskResponse.fromJson(Map<String, dynamic> json) {
    return FrostRiskResponse(
      riskLevel: json['risk_level'] as String? ?? 'bajo',
      confidence: (json['confidence'] as num? ?? 0).toDouble(),
      recommendation: json['recommendation'] as String? ?? '',
      chunoConditions: json['chuno_conditions'] as String? ?? 'no_favorables',
      modelVersion: json['model_version'] as String? ?? '-',
      dataSources: (json['data_sources'] as List<dynamic>? ?? const [])
          .map((item) => item.toString())
          .toList(),
    );
  }
}
```

app_flutter/lib/features/prediction/screens/prediction_form_screen.dart

```dart
import 'package:flutter/material.dart';

import '../../../core/api/frost_api_service.dart';
import '../../../core/i18n/app_strings.dart';
import '../../../core/models/current_weather.dart';
import '../../../core/theme/app_colors.dart';
import '../../../shared/widgets/glass_card.dart';
import '../../../shared/widgets/page_scaffold.dart';
import '../../../shared/widgets/primary_action_button.dart';
import '../../../shared/widgets/section_header.dart';
import '../../../shared/widgets/status_chip.dart';
import '../../location/models/location_state.dart';
import '../../location/services/location_service.dart';
import '../models/frost_risk_request.dart';
import 'result_screen.dart';

class PredictionFormScreen extends StatefulWidget {
  const PredictionFormScreen({required this.apiService, super.key});

  final FrostApiService apiService;

  @override
  State<PredictionFormScreen> createState() => _PredictionFormScreenState();
}

class _PredictionFormScreenState extends State<PredictionFormScreen> {
  final _locationService = LocationService();
  FrostRiskRequest _request = FrostRiskRequest.demo();
  DetectedLocation? _detectedLocation;
  CurrentWeather? _weather;
  bool _isLocating = false;
  bool _isLoadingWeather = false;
  bool _isPredicting = false;
  // null => se muestra la pista por defecto en el idioma activo.
  String? _locationMessage;
  String? _weatherMessage;

  @override
  Widget build(BuildContext context) {
    final isBusy = _isLocating || _isLoadingWeather || _isPredicting;
    final s = AppStrings.current;
    return PageScaffold(
      child: ListView(
        padding: const EdgeInsets.fromLTRB(20, 24, 20, 28),
        children: [
          SectionHeader(
            title: s.frostRiskTitle,
            subtitle: s.frostRiskSubtitle,
          ),
          const SizedBox(height: 24),
          _LocationCard(
            request: _request,
            detectedLocation: _detectedLocation,
            message: _locationMessage ?? s.locationHint,
            locating: _isLocating,
            onUseLocation: isBusy ? null : _useCurrentLocation,
          ),
          const SizedBox(height: 18),
          _WeatherCard(
            request: _request,
            weather: _weather,
            message: _weatherMessage ?? s.weatherHint,
```

```
          loading: _isLoadingWeather,
          onRefreshWeather: isBusy ? null : _loadWeatherForCurrentRequest,
        ),
        const SizedBox(height: 22),
        _ActionCard(
          predicting: _isPredicting,
          canPredict: !isBusy,
          onPredict: _submit,
        ),
      ],
    ),
  );
}

Future<void> _submit() async {
  setState(() => _isPredicting = true);
  try {
    final response = await widget.apiService.predict(_request);
    if (!mounted) return;
    await Navigator.of(context).push(
      MaterialPageRoute(
        builder: (_) => ResultScreen(request: _request, response: response),
      ),
    );
  } catch (error) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(AppStrings.current.connectError(error))),
    );
  } finally {
    if (mounted) {
      setState(() => _isPredicting = false);
    }
  }
}

Future<void> _useCurrentLocation() async {
  final s = AppStrings.current;
  setState(() {
    _isLocating = true;
    _locationMessage = s.requestingGps;
    _weatherMessage = s.waitingLocation;
  });
  try {
    final location = await _locationService.getCurrentLocation();
    if (!mounted) return;
    setState(() {
      _detectedLocation = location;
      _request = _request.copyWith(
        district: 'Ubicacion detectada',
        province: 'Puno',
        populatedCenter: 'GPS actual',
        latitude: location.latitude,
        longitude: location.longitude,
      );
      _locationMessage = s.gpsAccuracy(
        location.fromLastKnownPosition,
        location.accuracy.toStringAsFixed(0),
      );
    });
    await _loadCurrentWeather();
  } on LocationFailure catch (error) {
    if (!mounted) return;
    setState(() {
      _locationMessage = s.locationFailed(error.message);
      _weatherMessage = s.tapGetWeatherDemo;
    });
  } catch (error) {
    if (!mounted) return;
    setState(() {
      _locationMessage = s.locationFailedGeneric;
      _weatherMessage = s.detail(error);
    });
  } finally {
    if (mounted) {
      setState(() => _isLocating = false);
    }
  }
}

Future<void> _loadWeatherForCurrentRequest() async {
  await _loadCurrentWeather();
}

Future<void> _loadCurrentWeather() async {
  final s = AppStrings.current;
  setState(() {
    _isLoadingWeather = true;
    _weatherMessage = s.queryingWeather;
  });
  try {
    final weather = await widget.apiService.getCurrentWeather(
      latitude: _request.latitude,
      longitude: _request.longitude,
    );
    if (!mounted) return;
    setState(() {
      _weather = weather;
      _request = _requestWithWeather(_request, weather);
      _weatherMessage = s.weatherUpdated(
        weather.provider,
        weather.fallbackUsed,
      );
    });
  } catch (error) {
    if (!mounted) return;
    setState(() {
      _weatherMessage = s.weatherFailed(error);
    });
  } finally {
    if (mounted) {
      setState(() => _isLoadingWeather = false);
    }
  }
}

FrostRiskRequest _requestWithWeather(
  FrostRiskRequest current,
  CurrentWeather weather,
) {
  final now = DateTime.now();
```

```
    final temperature = weather.temperature;
    final feelsLike = weather.apparentTemperature ?? temperature;
    final hoursBelowZero = temperature == null
        ? current.hoursBelowZero
        : temperature <= 0
        ? 1
        : 0;

    return current.copyWith(
      temperatureMin: temperature ?? current.temperatureMin,
      temperatureMax: temperature ?? current.temperatureMax,
      feelsLike: feelsLike ?? current.feelsLike,
      humidity: weather.humidity ?? current.humidity,
      windSpeed: weather.windSpeed ?? current.windSpeed,
      cloudCover: weather.cloudCover ?? current.cloudCover,
      dewPoint: weather.dewPoint ?? current.dewPoint,
      precipitation: weather.precipitation ?? current.precipitation,
      month: now.month,
      hour: now.hour,
      hoursBelowZero: hoursBelowZero,
    );
  }
}

class _LocationCard extends StatelessWidget {
  const _LocationCard({
    required this.request,
    required this.message,
    required this.locating,
    required this.onUseLocation,
    this.detectedLocation,
  });

  final FrostRiskRequest request;
  final DetectedLocation? detectedLocation;
  final String message;
  final bool locating;
  final VoidCallback? onUseLocation;

  @override
  Widget build(BuildContext context) {
    final detected = detectedLocation != null;
    return GlassCard(
      padding: const EdgeInsets.all(22),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          _CardTitle(icon: Icons.my_location, label: AppStrings.current.location),
          const SizedBox(height: 16),
          Text(
            detected
                ? AppStrings.current.detectedLocation
                : AppStrings.current.punoDemo,
            style: Theme.of(context).textTheme.headlineMedium,
          ),
          const SizedBox(height: 10),
          Text(
            '${request.latitude.toStringAsFixed(5)}, ${request.longitude.toStringAsFixed(5)}',
            style: Theme.of(context).textTheme.bodyLarge?.copyWith(
              color: Theme.of(
                context,
              ).colorScheme.onSurface.withValues(alpha: 0.68),
              fontFamily: 'monospace',
            ),
          ),
          const SizedBox(height: 14),
          _InfoBanner(icon: Icons.location_on_outlined, text: message),
          const SizedBox(height: 16),
          PrimaryActionButton(
            label: locating
                ? AppStrings.current.detectingLocation
                : AppStrings.current.useMyLocation,
            icon: Icons.gps_fixed,
            onPressed: onUseLocation,
          ),
        ],
      ),
    );
  }
}

class _WeatherCard extends StatelessWidget {
  const _WeatherCard({
    required this.request,
    required this.message,
    required this.loading,
    required this.onRefreshWeather,
    this.weather,
  });

  final FrostRiskRequest request;
  final CurrentWeather? weather;
  final String message;
  final bool loading;
  final VoidCallback? onRefreshWeather;

  @override
  Widget build(BuildContext context) {
    final s = AppStrings.current;
    return GlassCard(
      padding: const EdgeInsets.all(22),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          _CardTitle(icon: Icons.cloud_outlined, label: s.currentWeather),
          const SizedBox(height: 18),
          _WeatherHero(request: request, weather: weather),
          const SizedBox(height: 16),
          Wrap(
            spacing: 10,
            runSpacing: 10,
            children: [
              _WeatherMetric(
                icon: Icons.water_drop_outlined,
                label: s.humidity,
                value: '${request.humidity.toStringAsFixed(0)}%',
              ),
              _WeatherMetric(
                icon: Icons.air,
                label: s.wind,
                value: '${request.windSpeed.toStringAsFixed(1)} km/h',
```

```
            ),
            _WeatherMetric(
              icon: Icons.cloud_queue,
              label: s.clouds,
              value: '${request.cloudCover.toStringAsFixed(0)}%',
            ),
          ],
        ),
        const SizedBox(height: 16),
        _InfoBanner(icon: Icons.cloud_sync_outlined, text: message),
        if (weather?.stationName != null) ...[
          const SizedBox(height: 12),
          StatusChip(
            icon: Icons.place_outlined,
            label: weather!.stationName!,
          ),
        ],
        const SizedBox(height: 16),
        PrimaryActionButton(
          label: loading ? s.gettingWeather : s.getWeather,
          icon: Icons.refresh,
          onPressed: onRefreshWeather,
        ),
      ],
    ),
  );
 }
}

class _ActionCard extends StatelessWidget {
 const _ActionCard({
   required this.predicting,
   required this.canPredict,
   required this.onPredict,
 });

 final bool predicting;
 final bool canPredict;
 final VoidCallback onPredict;

 @override
 Widget build(BuildContext context) {
   final s = AppStrings.current;
   return GlassCard(
     padding: const EdgeInsets.all(22),
     child: Column(
       crossAxisAlignment: CrossAxisAlignment.start,
       children: [
         Text(
           s.readyToPredict,
           style: Theme.of(context).textTheme.titleMedium,
         ),
         const SizedBox(height: 8),
         Text(
           s.readyToPredictBody,
           style: Theme.of(context).textTheme.bodyMedium?.copyWith(
             color: Theme.of(
               context,
             ).colorScheme.onSurface.withValues(alpha: 0.68),
           ),
         ),
         const SizedBox(height: 18),
         PrimaryActionButton(
           label: predicting ? s.predicting : s.predict,
           icon: Icons.analytics_outlined,
           onPressed: canPredict ? onPredict : null,
         ),
       ],
     ),
   );
 }
}

class _WeatherHero extends StatelessWidget {
 const _WeatherHero({required this.request, this.weather});

 final FrostRiskRequest request;
 final CurrentWeather? weather;

 @override
 Widget build(BuildContext context) {
   final temp = request.temperatureMin;
   final color = temp <= 0 ? AppColors.warmAmber : AppColors.mutedTeal;
   return Row(
     crossAxisAlignment: CrossAxisAlignment.center,
     children: [
       Container(
         width: 64,
         height: 64,
         decoration: BoxDecoration(
           color: color.withValues(alpha: 0.14),
           borderRadius: BorderRadius.circular(8),
         ),
         child: Icon(
           temp <= 0 ? Icons.ac_unit : Icons.thermostat,
           color: color,
           size: 34,
         ),
       ),
       const SizedBox(width: 16),
       Expanded(
         child: Column(
           crossAxisAlignment: CrossAxisAlignment.start,
           children: [
             Text(
               '${temp.toStringAsFixed(1)} C',
               style: Theme.of(
                 context,
               ).textTheme.displayLarge?.copyWith(color: color, fontSize: 42),
             ),
             const SizedBox(height: 4),
             Text(
               weather == null
                   ? AppStrings.current.internalDemoValue
                   : AppStrings.current.feelsLike(
                       request.feelsLike.toStringAsFixed(1),
                     ),
               style: Theme.of(context).textTheme.bodyMedium,
             ),
           ],
         ),
```

```
      ),
    ],
   );
  }
 }

 class _WeatherMetric extends StatelessWidget {
  const _WeatherMetric({
   required this.icon,
   required this.label,
   required this.value,
  });

  final IconData icon;
  final String label;
  final String value;

  @override
  Widget build(BuildContext context) {
   final onSurface = Theme.of(context).colorScheme.onSurface;
   return Container(
    constraints: const BoxConstraints(minWidth: 132, minHeight: 74),
    padding: const EdgeInsets.all(12),
    decoration: BoxDecoration(
     color: onSurface.withValues(alpha: 0.06),
     borderRadius: BorderRadius.circular(8),
     border: Border.all(color: onSurface.withValues(alpha: 0.08)),
    ),
    child: Column(
     crossAxisAlignment: CrossAxisAlignment.start,
     mainAxisAlignment: MainAxisAlignment.spaceBetween,
     children: [
      Row(
       children: [
        Icon(icon, size: 18, color: AppColors.mutedTeal),
        const SizedBox(width: 6),
        Text(label, style: Theme.of(context).textTheme.labelSmall),
       ],
      ),
      Text(value, style: Theme.of(context).textTheme.titleMedium),
     ],
    ),
   );
  }
 }

 class _CardTitle extends StatelessWidget {
  const _CardTitle({required this.icon, required this.label});

  final IconData icon;
  final String label;

  @override
  Widget build(BuildContext context) {
   return Row(
    children: [
     Icon(icon, color: AppColors.mutedTeal),
     const SizedBox(width: 10),
     Text(label, style: Theme.of(context).textTheme.titleMedium),
    ],
   );
  }
 }

 class _InfoBanner extends StatelessWidget {
  const _InfoBanner({required this.icon, required this.text});

  final IconData icon;
  final String text;

  @override
  Widget build(BuildContext context) {
   final onSurface = Theme.of(context).colorScheme.onSurface;
   return Container(
    padding: const EdgeInsets.all(14),
    decoration: BoxDecoration(
     color: onSurface.withValues(alpha: 0.05),
     borderRadius: BorderRadius.circular(8),
     border: Border.all(color: onSurface.withValues(alpha: 0.08)),
    ),
    child: Row(
     crossAxisAlignment: CrossAxisAlignment.start,
     children: [
      Icon(icon, size: 20, color: AppColors.mutedTeal),
      const SizedBox(width: 10),
      Expanded(
       child: Text(text, style: Theme.of(context).textTheme.bodyMedium),
      ),
     ],
    ),
   );
  }
 }
```

app_flutter/lib/features/prediction/screens/result_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/i18n/app_strings.dart';
import '../../../core/theme/app_colors.dart';
import '../../../shared/widgets/glass_card.dart';
import '../../../shared/widgets/page_scaffold.dart';
import '../../../shared/widgets/primary_action_button.dart';
import '../../../shared/widgets/section_header.dart';
import '../../../shared/widgets/status_chip.dart';
import '../models/frost_risk_request.dart';
import '../models/frost_risk_response.dart';

class ResultScreen extends StatelessWidget {
 const ResultScreen({
  required this.request,
  required this.response,
  super.key,
 });

 final FrostRiskRequest request;
```

```
  final FrostRiskResponse response;

  @override
  Widget build(BuildContext context) {
    final riskColor = _riskColor(response.riskLevel);
    final riskIcon = _riskIcon(response.riskLevel);
    final onSurface = Theme.of(context).colorScheme.onSurface;
    final s = AppStrings.current;
    return PageScaffold(
      child: ListView(
        padding: const EdgeInsets.fromLTRB(20, 24, 20, 28),
        children: [
          SectionHeader(
            title: s.resultTitle,
            subtitle: s.resultSubtitle,
          ),
          const SizedBox(height: 24),
          GlassCard(
            padding: const EdgeInsets.all(24),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Row(
                  children: [
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          _Label(s.riskLabel),
                          const SizedBox(height: 8),
                          Text(
                            s.riskWord(response.riskLevel).toUpperCase(),
                            style: Theme.of(context).textTheme.displayLarge
                                ?.copyWith(color: riskColor, fontSize: 44),
                          ),
                        ],
                      ),
                    ),
                    Container(
                      width: 82,
                      height: 82,
                      decoration: BoxDecoration(
                        color: riskColor.withValues(alpha: 0.14),
                        borderRadius: BorderRadius.circular(8),
                      ),
                      child: Icon(riskIcon, color: riskColor, size: 42),
                    ),
                  ],
                ),
                const SizedBox(height: 20),
                Container(
                  padding: const EdgeInsets.all(16),
                  decoration: BoxDecoration(
                    color: riskColor.withValues(alpha: 0.12),
                    borderRadius: BorderRadius.circular(8),
                    border: Border.all(
                      color: riskColor.withValues(alpha: 0.24),
                    ),
                  ),
                  child: Row(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Icon(Icons.tips_and_updates_outlined, color: riskColor),
                      const SizedBox(width: 12),
                      Expanded(
                        child: Text(
                          s.recommendation(
                            response.riskLevel,
                            _shortRecommendation(response.recommendation),
                          ),
                          style: Theme.of(context).textTheme.bodyLarge,
                        ),
                      ),
                    ],
                  ),
                ),
                const SizedBox(height: 24),
                const Divider(),
                const SizedBox(height: 18),
                Row(
                  children: [
                    Expanded(
                      child: _MetricBlock(
                        label: s.confidence,
                        value: '${(response.confidence * 100).round()}%',
                      ),
                    ),
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          _Label(s.chunoConditionLabel),
                          const SizedBox(height: 8),
                          StatusChip(
                            icon: Icons.ac_unit,
                            label: s.chunoCondition(response.chunoConditions),
                            color: onSurface,
                            background: onSurface.withValues(alpha: 0.08),
                          ),
                        ],
                      ),
                    ),
                  ],
                ),
              ],
            ),
          ),
          const SizedBox(height: 20),
          GlassCard(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                _Label(s.summary),
                const SizedBox(height: 18),
                _InfoRow(label: s.district, value: request.district),
                const Divider(),
                _InfoRow(label: s.model, value: response.modelVersion),
              ],
            ),
          ),
          const SizedBox(height: 28),
```

```
        PrimaryActionButton(
          label: s.newQuery,
          icon: Icons.add,
          onPressed: () => Navigator.of(context).pop(),
        ),
      ],
    ),
  );
 }

 static Color _riskColor(String value) {
  return switch (value.toLowerCase()) {
    'alto' => AppColors.warmAmber,
    'medio' => AppColors.mediumRisk,
    'moderado' => AppColors.mediumRisk,
    _ => AppColors.lowRisk,
  };
 }

 static IconData _riskIcon(String value) {
  return switch (value.toLowerCase()) {
    'alto' => Icons.warning_amber_rounded,
    'medio' => Icons.report_problem_outlined,
    'moderado' => Icons.report_problem_outlined,
    _ => Icons.check_circle_outline,
  };
 }

 static String _shortRecommendation(String value) {
  final clean = value.trim();
  if (clean.isEmpty) {
    return 'Revisar condiciones locales y monitorear cambios de temperatura.';
  }
  final firstSentence = clean.split('.').first.trim();
  return firstSentence.isEmpty ? clean : '$firstSentence.';
 }

}

class _Label extends StatelessWidget {
 const _Label(this.text);

 final String text;

 @override
 Widget build(BuildContext context) {
  return Text(
    text,
    style: Theme.of(context).textTheme.labelSmall?.copyWith(
      color: Theme.of(context).colorScheme.onSurface.withValues(alpha: 0.64),
      fontFamily: 'monospace',
      letterSpacing: 0,
      fontWeight: FontWeight.w700,
    ),
  );
 }
}

class _MetricBlock extends StatelessWidget {
 const _MetricBlock({required this.label, required this.value});

 final String label;
 final String value;

 @override
 Widget build(BuildContext context) {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      _Label(label),
      const SizedBox(height: 8),
      Text(value, style: Theme.of(context).textTheme.headlineMedium),
    ],
  );
 }
}

class _InfoRow extends StatelessWidget {
 const _InfoRow({required this.label, required this.value});

 final String label;
 final String value;

 @override
 Widget build(BuildContext context) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 10),
    child: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Expanded(
          child: Text(label, style: Theme.of(context).textTheme.bodyLarge),
        ),
        Flexible(
          child: Text(
            value,
            textAlign: TextAlign.end,
            style: const TextStyle(
              fontFamily: 'monospace',
              fontWeight: FontWeight.w700,
            ),
          ),
        ),
      ],
    ),
  );
 }
}
```

app_flutter/lib/features/settings/screens/settings_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/i18n/app_language.dart';
import '../../../core/i18n/app_strings.dart';
import '../../../core/settings/user_settings.dart';
```

```dart
import '../../../shared/widgets/glass_card.dart';
import '../../../shared/widgets/page_scaffold.dart';
import '../../../shared/widgets/section_header.dart';

class SettingsScreen extends StatelessWidget {
  const SettingsScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final settings = UserSettings.instance;
    return PageScaffold(
      child: AnimatedBuilder(
        animation: settings,
        builder: (context, _) {
          final s = AppStrings.current;
          return ListView(
          padding: const EdgeInsets.fromLTRB(24, 28, 24, 32),
          children: [
            SectionHeader(
              title: s.settings,
              subtitle: s.settingsSubtitle,
            ),
            const SizedBox(height: 24),
            GlassCard(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  SwitchListTile(
                    contentPadding: EdgeInsets.zero,
                    title: Text(s.enableAlerts),
                    subtitle: Text(s.enableAlertsSub),
                    value: settings.alertsEnabled,
                    onChanged: settings.setAlertsEnabled,
                  ),
                  const Divider(height: 24),
                  Text(
                    s.alertThresholdLabel(
                      settings.alertThreshold.toStringAsFixed(0),
                    ),
                    style: Theme.of(context).textTheme.titleMedium,
                  ),
                  Slider(
                    min: -12,
                    max: 2,
                    divisions: 14,
                    label: '${settings.alertThreshold.toStringAsFixed(0)} °C',
                    value: settings.alertThreshold,
                    onChanged: settings.alertsEnabled
                        ? settings.setThreshold
                        : null,
                  ),
                ],
              ),
            ),
            const SizedBox(height: 18),
            GlassCard(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(s.profile, style: Theme.of(context).textTheme.titleMedium),
                  const SizedBox(height: 6),
                  Text(
                    s.profileHint,
                    style: Theme.of(context).textTheme.bodyMedium?.copyWith(
                      color: Theme.of(
                        context,
                      ).colorScheme.onSurface.withValues(alpha: 0.66),
                    ),
                  ),
                  const SizedBox(height: 14),
                  Wrap(
                    spacing: 10,
                    runSpacing: 10,
                    children: UserProfile.values.map((p) {
                      final selected = settings.profile == p;
                      return ChoiceChip(
                        label: Text(s.profileName(p.name)),
                        selected: selected,
                        onSelected: (_) => settings.setProfile(p),
                      );
                    }).toList(),
                  ),
                ],
              ),
            ),
            const SizedBox(height: 18),
            GlassCard(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    s.languageLabel,
                    style: Theme.of(context).textTheme.titleMedium,
                  ),
                  const SizedBox(height: 14),
                  Wrap(
                    spacing: 10,
                    runSpacing: 10,
                    children: AppLanguage.values.map((l) {
                      return ChoiceChip(
                        label: Text(l.label),
                        selected: settings.language == l,
                        onSelected: (_) => settings.setLanguage(l),
                      );
                    }).toList(),
                  ),
                ],
              ),
            ),
          ],
        );
        },
      ),
    );
  }
}
```

app_flutter/lib/features/shell/app_shell.dart

```dart
import 'package:flutter/material.dart';

import '../../core/api/frost_api_service.dart';
import '../../core/i18n/app_strings.dart';
import '../charts/screens/charts_screen.dart';
import '../chuno/screens/chuno_screen.dart';
import '../clusters/screens/clusters_screen.dart';
import '../history/screens/history_screen.dart';
import '../home/screens/home_screen.dart';

class AppShell extends StatefulWidget {
  const AppShell({required this.apiService, super.key});

  final FrostApiService apiService;

  @override
  State<AppShell> createState() => _AppShellState();
}

class _AppShellState extends State<AppShell> {
  int _selectedIndex = 0;

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;
    final s = AppStrings.current;
    final screens = [
      HomeScreen(apiService: widget.apiService),
      ClustersScreen(apiService: widget.apiService),
      ChunoScreen(apiService: widget.apiService),
      ChartsScreen(apiService: widget.apiService),
      HistoryScreen(apiService: widget.apiService, showAppBar: false),
    ];

    return Scaffold(
      body: screens[_selectedIndex],
      bottomNavigationBar: NavigationBar(
        selectedIndex: _selectedIndex,
        onDestinationSelected: (index) =>
            setState(() => _selectedIndex = index),
        backgroundColor: colorScheme.surface.withValues(alpha: 0.96),
        indicatorColor: colorScheme.primary,
        labelTextStyle: WidgetStateProperty.resolveWith(
          (states) => TextStyle(
            fontFamily: 'monospace',
            fontSize: 12,
            fontWeight: FontWeight.w600,
            color: states.contains(WidgetState.selected)
                ? colorScheme.onPrimary
                : colorScheme.onSurface.withValues(alpha: 0.68),
          ),
        ),
        destinations: [
          NavigationDestination(
            icon: const Icon(Icons.home_outlined),
            selectedIcon: const Icon(Icons.home),
            label: s.tabHome,
          ),
          NavigationDestination(
            icon: const Icon(Icons.map_outlined),
            selectedIcon: const Icon(Icons.map),
            label: s.tabZones,
          ),
          NavigationDestination(icon: const Icon(Icons.ac_unit), label: s.tabChuno),
          NavigationDestination(icon: const Icon(Icons.show_chart), label: s.tabWeather),
          NavigationDestination(icon: const Icon(Icons.history), label: s.tabHistory),
        ],
      ),
    );
  }
}
```

app_flutter/lib/main.dart

```dart
import 'package:flutter/material.dart';

import 'app.dart';
import 'core/notifications/notification_service.dart';
import 'core/settings/user_settings.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await UserSettings.instance.load();
  await NotificationService.init();
  runApp(const FrostPunoApp());
}
```

app_flutter/lib/shared/widgets/frost_app_bar.dart

```dart
import 'package:flutter/material.dart';

import '../../core/theme/theme_mode_scope.dart';

class FrostAppBar extends StatelessWidget implements PreferredSizeWidget {
  const FrostAppBar({super.key, this.canPop = false});

  final bool canPop;

  @override
  Size get preferredSize => const Size.fromHeight(64);

  @override
  Widget build(BuildContext context) {
    final themeScope = ThemeModeScope.maybeOf(context);
    final mode = themeScope?.themeMode ?? ThemeMode.system;
    return AppBar(
      leading: canPop
```

```
        ? IconButton(
            icon: const Icon(Icons.arrow_back),
            onPressed: () => Navigator.of(context).maybePop(),
          )
        : null,
    title: const Text('FrostPuno'),
    actions: [
      IconButton(
        tooltip: _nextThemeLabel(mode),
        icon: Icon(_themeIcon(mode)),
        onPressed: themeScope == null
            ? null
            : () => themeScope.onThemeModeChanged(_nextThemeMode(mode)),
      ),
    ],
  );
 }

 static ThemeMode _nextThemeMode(ThemeMode mode) {
  return switch (mode) {
    ThemeMode.system => ThemeMode.light,
    ThemeMode.light => ThemeMode.dark,
    ThemeMode.dark => ThemeMode.system,
  };
 }

 static IconData _themeIcon(ThemeMode mode) {
  return switch (mode) {
    ThemeMode.system => Icons.brightness_auto_outlined,
    ThemeMode.light => Icons.light_mode_outlined,
    ThemeMode.dark => Icons.dark_mode_outlined,
  };
 }

 static String _nextThemeLabel(ThemeMode mode) {
  return switch (mode) {
    ThemeMode.system => 'Usar modo claro',
    ThemeMode.light => 'Usar modo oscuro',
    ThemeMode.dark => 'Usar modo del sistema',
  };
 }
}
```

app_flutter/lib/shared/widgets/glass_card.dart

```
import 'package:flutter/material.dart';

import '../../core/theme/app_colors.dart';

class GlassCard extends StatelessWidget {
 const GlassCard({
  required this.child,
  super.key,
  this.padding = const EdgeInsets.all(20),
  this.color,
 });

 final Widget child;
 final EdgeInsetsGeometry padding;
 final Color? color;

 @override
 Widget build(BuildContext context) {
  final colorScheme = Theme.of(context).colorScheme;
  final isDark = colorScheme.brightness == Brightness.dark;
  return Container(
    width: double.infinity,
    padding: padding,
    decoration: BoxDecoration(
      color:
          color ??
          (isDark
              ? AppColors.darkSurfaceHigh.withValues(alpha: 0.84)
              : AppColors.iceBlue.withValues(alpha: 0.45)),
      borderRadius: BorderRadius.circular(8),
      border: Border.all(
        color: isDark ? AppColors.darkOutline : AppColors.outline,
      ),
      boxShadow: [
        BoxShadow(
          color: isDark
              ? Colors.black.withValues(alpha: 0.14)
              : AppColors.deepNavy.withValues(alpha: 0.04),
          blurRadius: isDark ? 18 : 24,
          offset: const Offset(0, 8),
        ),
      ],
    ),
    child: child,
  );
 }
}
```

app_flutter/lib/shared/widgets/page_scaffold.dart

```
import 'package:flutter/material.dart';

import 'frost_app_bar.dart';
import 'responsive_content.dart';

class PageScaffold extends StatelessWidget {
 const PageScaffold({
  required this.child,
  super.key,
  this.canPop = true,
  this.bottomNavigationBar,
 });

 final Widget child;
 final bool canPop;
 final Widget? bottomNavigationBar;
```

```

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: FrostAppBar(canPop: canPop),
      bottomNavigationBar: bottomNavigationBar,
      body: SafeArea(child: ResponsiveContent(child: child)),
    );
  }
}
```

app_flutter/lib/shared/widgets/primary_action_button.dart

```
import 'package:flutter/material.dart';

class PrimaryActionButton extends StatelessWidget {
  const PrimaryActionButton({
    required this.label,
    required this.onPressed,
    super.key,
    this.icon,
    this.expanded = true,
  });

  final String label;
  final VoidCallback? onPressed;
  final IconData? icon;
  final bool expanded;

  @override
  Widget build(BuildContext context) {
    final button = FilledButton.icon(
      onPressed: onPressed,
      icon: Icon(icon ?? Icons.arrow_forward),
      label: Text(label),
      style: FilledButton.styleFrom(
        minimumSize: const Size(0, 60),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
        textStyle: const TextStyle(fontSize: 18, fontWeight: FontWeight.w700),
      ),
    );
    return expanded ? SizedBox(width: double.infinity, child: button) : button;
  }
}
```

app_flutter/lib/shared/widgets/responsive_content.dart

```
import 'package:flutter/material.dart';

class ResponsiveContent extends StatelessWidget {
  const ResponsiveContent({
    required this.child,
    super.key,
    this.maxWidth = 720,
  });

  final Widget child;
  final double maxWidth;

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        final viewportWidth = MediaQuery.sizeOf(context).width;
        final availableWidth = constraints.maxWidth.isFinite
            ? constraints.maxWidth
            : viewportWidth;
        final width = availableWidth > maxWidth ? maxWidth : availableWidth;
        return Align(
          alignment: Alignment.topCenter,
          child: SizedBox(width: width, child: child),
        );
      },
    );
  }
}
```

app_flutter/lib/shared/widgets/section_header.dart

```
import 'package:flutter/material.dart';

class SectionHeader extends StatelessWidget {
  const SectionHeader({required this.title, required this.subtitle, super.key});

  final String title;
  final String subtitle;

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;
    final colorScheme = Theme.of(context).colorScheme;
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(title, style: textTheme.displayLarge),
        const SizedBox(height: 12),
        Text(
          subtitle,
          style: textTheme.bodyLarge?.copyWith(
            color: colorScheme.onSurface.withValues(alpha: 0.68),
          ),
        ),
      ],
    );
  }
}
```

app_flutter/lib/shared/widgets/status_chip.dart

```dart
import 'package:flutter/material.dart';

import '../../core/theme/app_colors.dart';

class StatusChip extends StatelessWidget {
  const StatusChip({
    required this.label,
    super.key,
    this.icon,
    this.color = AppColors.mutedTeal,
    this.background,
  });

  final String label;
  final IconData? icon;
  final Color color;
  final Color? background;

  @override
  Widget build(BuildContext context) {
    final onSurface = Theme.of(context).colorScheme.onSurface;
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 7),
      decoration: BoxDecoration(
        color: background ?? color.withValues(alpha: 0.10),
        borderRadius: BorderRadius.circular(999),
        border: Border.all(color: color.withValues(alpha: 0.18)),
      ),
      child: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          if (icon != null) ...[
            Icon(icon, size: 16, color: color),
            const SizedBox(width: 6),
          ],
          Text(
            label,
            style: TextStyle(
              color: color == AppColors.warmAmber ? onSurface : color,
              fontFamily: 'monospace',
              fontSize: 12,
              fontWeight: FontWeight.w600,
              letterSpacing: 0,
            ),
          ),
        ],
      ),
    );
  }
}
```

4. Integracion continua

.github/workflows/backend-tests.yml

```yaml
name: Backend Tests

on:
  push:
  pull_request:

jobs:
  backend-tests:
    runs-on: ubuntu-latest
    env:
      ENABLE_SUPABASE: "false"
      PYTHONPATH: ${{ github.workspace }}/backend_fastapi

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: "pip"
          cache-dependency-path: |
            backend_fastapi/requirements.txt
            backend_fastapi/requirements-dev.txt
            ml_pipeline/requirements.txt

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r backend_fastapi/requirements-dev.txt
          pip install -r ml_pipeline/requirements.txt

      - name: Run backend tests
        run: pytest backend_fastapi/tests -q
```

.github/workflows/data-validation.yml

```yaml
name: Data Validation

on:
  push:
  pull_request:
  workflow_dispatch:

jobs:
  data-validation:
    runs-on: ubuntu-latest

    steps:
```

```
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: "pip"
          cache-dependency-path: ml_pipeline/requirements.txt

      - name: Install ML dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r ml_pipeline/requirements.txt

      - name: Validate and process locations
        run: python -m ml_pipeline.data_ingestion.ingest_locations

      - name: Build demo weather dataset
        run: >
          python -m ml_pipeline.data_ingestion.ingest_weather_open_meteo
          --start-date 2024-06-01
          --end-date 2024-06-03
          --limit 3
          --max-workers 3

      - name: Build features
        run: python -m ml_pipeline.features.build_features

      - name: Validate training dataset
        run: python -m ml_pipeline.preprocessing.validate_dataset

      - name: Validate district clustering output
        run: >
          python -c "import pandas as pd;
          df = pd.read_csv('data/processed/district_clusters.csv');
          required = {'distrito', 'tier', 'dominant_cluster', 'cold_share'};
          assert required.issubset(df.columns), f'missing columns: {required - set(df.columns)}';
          assert set(df['tier']).issubset({'alto', 'medio', 'bajo'}), 'unexpected tier values';
          print(f'district_clusters.csv OK: {len(df)} districts')"
```

.github/workflows/flutter-build.yml

```
name: Flutter Build

on:
  push:
  pull_request:
  workflow_dispatch:

jobs:
  flutter-build:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: app_flutter

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Flutter
        uses: subosito/flutter-action@v2
        with:
          channel: stable
          cache: true

      - name: Install dependencies
        run: flutter pub get

      - name: Analyze Flutter code
        run: flutter analyze

      - name: Run Flutter tests
        run: flutter test

      - name: Build Flutter Web
        run: flutter build web --release --dart-define=API_BASE_URL=http://127.0.0.1:8000
```

.github/workflows/ml-training.yml

```
name: ML Training

on:
  workflow_dispatch:
    inputs:
      start_date:
        description: "Open-Meteo historical start date"
        required: false
        default: "2024-06-01"
      end_date:
        description: "Open-Meteo historical end date"
        required: false
        default: "2024-06-07"
      location_limit:
        description: "Number of locations for demo training. Use empty for all curated locations."
        required: false
        default: "5"
      model_version:
        description: "Model version label"
        required: false
        default: "ci-demo"
  schedule:
    - cron: "0 9 * * 1"

jobs:
  train-model:
    runs-on: ubuntu-latest
    env:
      START_DATE: ${{ github.event_name == 'workflow_dispatch' && github.event.inputs.start_date || '2024-06-01' }}
      END_DATE: ${{ github.event_name == 'workflow_dispatch' && github.event.inputs.end_date || '2024-06-07' }}
```

```
      LOCATION_LIMIT: ${{ github.event_name == 'workflow_dispatch' && github.event.inputs.location_limit || '5' }}
      MODEL_VERSION: ${{ github.event_name == 'workflow_dispatch' && github.event.inputs.model_version || 'scheduled-demo' }}

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: "pip"
          cache-dependency-path: ml_pipeline/requirements.txt

      - name: Install ML dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r ml_pipeline/requirements.txt

      - name: Ingest locations
        run: python -m ml_pipeline.data_ingestion.ingest_locations

      - name: Ingest Open-Meteo demo weather
        run: |
          if [ -n "$LOCATION_LIMIT" ]; then
            python -m ml_pipeline.data_ingestion.ingest_weather_open_meteo \
              --start-date "$START_DATE" \
              --end-date "$END_DATE" \
              --limit "$LOCATION_LIMIT" \
              --max-workers 4
          else
            python -m ml_pipeline.data_ingestion.ingest_weather_open_meteo \
              --start-date "$START_DATE" \
              --end-date "$END_DATE" \
              --max-workers 4
          fi

      - name: Build features
        run: python -m ml_pipeline.features.build_features

      - name: Validate dataset
        run: python -m ml_pipeline.preprocessing.validate_dataset

      # Corrida manual: entrenamiento simple sobre la ventana indicada.
      - name: Train unsupervised K-Means model
        if: github.event_name != 'schedule'
        run: python -m ml_pipeline.clustering.train_clusters --version "$MODEL_VERSION"

      # Mantenimiento programado: reentrenamiento ADAPTATIVO. Si la silhouette no
      # alcanza el objetivo, amplía la ventana de datos y reintenta; registra cada
      # corrida en training_history.json.
      - name: Adaptive retraining (scheduled maintenance)
        if: github.event_name == 'schedule'
        run: >
          python -m ml_pipeline.clustering.adaptive_retrain
          --target 0.45 --max-iters 3 --initial-days 14 --step-days 15 --max-workers 4

      - name: Cluster quality gate (silhouette)
        run: >
          python -m ml_pipeline.registry.check_cluster_quality
          --metadata ml_pipeline/registry/cluster_metadata.json

      - name: Upload model artifacts
        uses: actions/upload-artifact@v4
        with:
          name: frost-puno-model-${{ github.run_id }}
          retention-days: 14
          path: |
            ml_pipeline/registry/frost_cluster_model.joblib
            ml_pipeline/registry/cluster_metadata.json
            ml_pipeline/registry/training_history.json
            ml_pipeline/evaluation/cluster_profiles.csv
            data/processed/district_clusters.csv
            data/processed/frost_training_dataset.csv
```

.github/workflows/model-quality-gate.yml

```
name: Model Quality Gate

on:
  push:
  pull_request:
  workflow_dispatch:
    inputs:
      min_silhouette:
        description: "Minimum accepted silhouette score"
        required: false
        default: "0.35"

jobs:
  model-quality-gate:
    runs-on: ubuntu-latest
    env:
      MIN_SILHOUETTE: ${{ github.event_name == 'workflow_dispatch' && github.event.inputs.min_silhouette || '0.35' }}

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install ML dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r ml_pipeline/requirements.txt pytest

      - name: Evaluate cluster metadata quality (silhouette)
        run: >
          python -m ml_pipeline.registry.check_cluster_quality
          --metadata ml_pipeline/registry/cluster_metadata.json

      # Pruebas de funcionamiento del mantenimiento e IC: verifican artefactos del
      # reentrenamiento, hiperparámetros optimizados y que el gate BLOQUEA un modelo
```

```
      # degradado.
      - name: Maintenance & CI functional tests
        run: python -m pytest ml_pipeline/tests/test_maintenance_ci.py -v

      - name: Future active-model comparison
        run: |
          echo "Future step: download the currently active production cluster metadata,"
          echo "compare silhouette / Davies-Bouldin drift, then block deployment on regression."
```

5. Configuracion de despliegue

render.yaml

```
services:
  - type: web
    name: frost-puno-api
    runtime: python
    plan: free
    region: oregon
    rootDir: .
    buildCommand: pip install --upgrade pip && pip install -r backend_fastapi/requirements.txt
    startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT --app-dir backend_fastapi
    healthCheckPath: /health
    autoDeploy: true
    envVars:
      - key: PYTHON_VERSION
        value: 3.11.9
      - key: ENABLE_SUPABASE
        value: true
      - key: CORS_ORIGINS
        value: https://frost-puno.vercel.app,http://localhost:3000,http://127.0.0.1:5174,http://localhost:5174
      - key: MODEL_PATH
        value: ml_pipeline/registry/frost_cluster_model.joblib
      - key: MODEL_METADATA_PATH
        value: ml_pipeline/registry/cluster_metadata.json
      - key: DISTRICT_CLUSTERS_PATH
        value: data/processed/district_clusters.csv
      - key: SUPABASE_URL
        value: https://qwufojxuebxfhimmvbaa.supabase.co
      - key: SUPABASE_SERVICE_ROLE_KEY
        sync: false
```

supabase/migrations/001__create__core__tables.sql

```
create extension if not exists pgcrypto;

create or replace function public.set_updated_at()
returns trigger
language plpgsql
as $$
begin
  new.updated_at = now();
  return new;
end;
$$;

create table if not exists public.users_profile (
  id uuid primary key default gen_random_uuid(),
  user_id uuid unique references auth.users(id) on delete cascade,
  full_name text,
  role text not null default 'producer' check (role in ('producer', 'student', 'admin', 'researcher')),
  department text not null default 'Puno',
  province text,
  district text,
  community text,
  source text not null default 'app',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table if not exists public.inei_locations (
  id uuid primary key default gen_random_uuid(),
  ubigeo text not null unique check (ubigeo ~ '^[0-9]{6}$'),
  department text not null,
  province text not null,
  district text not null,
  latitude numeric(9,6) not null check (latitude between -18.5 and -13.0),
  longitude numeric(9,6) not null check (longitude between -71.5 and -68.0),
  estimated_altitude numeric(8,2) check (estimated_altitude between 0 and 7000),
  total_population integer check (total_population >= 0),
  rural_population integer check (rural_population >= 0),
  rural_percentage numeric(5,2) check (rural_percentage between 0 and 100),
  source text not null default 'INEI',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  constraint rural_population_lte_total check (
    total_population is null
    or rural_population is null
    or rural_population <= total_population
  )
);

create table if not exists public.populated_centers (
  id uuid primary key default gen_random_uuid(),
  location_id uuid references public.inei_locations(id) on delete set null,
  ubigeo text references public.inei_locations(ubigeo) on update cascade on delete set null,
  name text not null,
  category text,
  latitude numeric(9,6) check (latitude between -18.5 and -13.0),
  longitude numeric(9,6) check (longitude between -71.5 and -68.0),
  altitude numeric(8,2) check (altitude between 0 and 7000),
  population integer check (population >= 0),
  source text not null default 'INEI Sistema de Centros Poblados',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
```

```
);

create table if not exists public.agricultural_context (
  id uuid primary key default gen_random_uuid(),
  location_id uuid references public.inei_locations(id) on delete cascade,
  ubigeo text references public.inei_locations(ubigeo) on update cascade on delete cascade,
  crop text not null,
  agricultural_activity boolean not null default true,
  cultivated_area_ha numeric(12,2) check (cultivated_area_ha is null or cultivated_area_ha >= 0),
  production_tons numeric(12,2) check (production_tons is null or production_tons >= 0),
  yield_ton_ha numeric(12,4) check (yield_ton_ha is null or yield_ton_ha >= 0),
  reference_year integer check (reference_year between 1900 and 2100),
  source text not null default 'INEI/MIDAGRI-SIEA',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table if not exists public.weather_records (
  id uuid primary key default gen_random_uuid(),
  location_id uuid references public.inei_locations(id) on delete set null,
  ubigeo text references public.inei_locations(ubigeo) on update cascade on delete set null,
  observed_at timestamptz not null,
  latitude numeric(9,6) not null check (latitude between -18.5 and -13.0),
  longitude numeric(9,6) not null check (longitude between -71.5 and -68.0),
  temperature_2m numeric(6,2),
  relative_humidity_2m numeric(5,2) check (relative_humidity_2m between 0 and 100),
  apparent_temperature numeric(6,2),
  dew_point_2m numeric(6,2),
  precipitation numeric(8,2) check (precipitation is null or precipitation >= 0),
  cloud_cover numeric(5,2) check (cloud_cover between 0 and 100),
  wind_speed_10m numeric(7,2) check (wind_speed_10m is null or wind_speed_10m >= 0),
  source text not null default 'Open-Meteo',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now()
);

create table if not exists public.model_versions (
  id uuid primary key default gen_random_uuid(),
  version text not null unique,
  model_name text not null,
  algorithm text not null,
  artifact_path text not null,
  accuracy numeric(8,6) check (accuracy between 0 and 1),
  precision_macro numeric(8,6) check (precision_macro between 0 and 1),
  recall_macro numeric(8,6) check (recall_macro between 0 and 1),
  f1_macro numeric(8,6) check (f1_macro between 0 and 1),
  dataset_size integer not null check (dataset_size >= 0),
  status text not null default 'candidate' check (status in ('candidate', 'active', 'rejected', 'archived')),
  source text not null default 'ml_pipeline',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  deployed_at timestamptz
);

create table if not exists public.training_runs (
  id uuid primary key default gen_random_uuid(),
  model_version_id uuid references public.model_versions(id) on delete set null,
  run_name text,
  dataset_path text,
  dataset_hash text,
  git_commit text,
  accuracy numeric(8,6) check (accuracy between 0 and 1),
  precision_macro numeric(8,6) check (precision_macro between 0 and 1),
  recall_macro numeric(8,6) check (recall_macro between 0 and 1),
  f1_macro numeric(8,6) check (f1_macro between 0 and 1),
  status text not null default 'completed' check (status in ('started', 'completed', 'failed', 'promoted', 'rejected')),
  source text not null default 'github_actions_or_local',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table if not exists public.frost_predictions (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users(id) on delete set null,
  location_id uuid references public.inei_locations(id) on delete set null,
  model_version_id uuid references public.model_versions(id) on delete set null,
  district text not null,
  province text not null,
  populated_center text,
  latitude numeric(9,6) not null check (latitude between -18.5 and -13.0),
  longitude numeric(9,6) not null check (longitude between -71.5 and -68.0),
  altitude numeric(8,2) check (altitude between 0 and 7000),
  input_payload jsonb not null,
  risk_level text not null check (risk_level in ('bajo', 'medio', 'alto')),
  confidence numeric(6,5) not null check (confidence between 0 and 1),
  chuno_conditions text not null check (chuno_conditions in ('favorables', 'posibles', 'no_favorables')),
  recommendation text not null,
  model_version text not null,
  data_sources jsonb not null default '[]'::jsonb,
  source text not null default 'backend_fastapi',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now()
);

create table if not exists public.alerts (
  id uuid primary key default gen_random_uuid(),
  location_id uuid references public.inei_locations(id) on delete set null,
  prediction_id uuid references public.frost_predictions(id) on delete set null,
  title text not null,
  message text not null,
  risk_level text not null check (risk_level in ('bajo', 'medio', 'alto')),
  channel text not null default 'app' check (channel in ('app', 'email', 'push', 'sms', 'whatsapp')),
  status text not null default 'draft' check (status in ('draft', 'sent', 'cancelled', 'failed')),
  starts_at timestamptz,
  ends_at timestamptz,
  source text not null default 'backend_fastapi',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table if not exists public.data_sources_log (
  id uuid primary key default gen_random_uuid(),
  source_name text not null,
  source_type text not null check (source_type in ('INEI', 'SENAMHI', 'Open-Meteo', 'MIDAGRI-SIEA', 'manual', 'other')),
  resource_url text,
  file_path text,
  rows_processed integer check (rows_processed is null or rows_processed >= 0),
  status text not null default 'completed' check (status in ('started', 'completed', 'failed')),
```

```
  checksum text,
  error_message text,
  source text not null default 'pipeline',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now()
);

create index if not exists idx_inei_locations_ubigeo on public.inei_locations(ubigeo);
create index if not exists idx_populated_centers_ubigeo on public.populated_centers(ubigeo);
create index if not exists idx_agricultural_context_ubigeo_crop on public.agricultural_context(ubigeo, crop);
create index if not exists idx_weather_records_ubigeo_observed_at on public.weather_records(ubigeo, observed_at desc);
create index if not exists idx_frost_predictions_created_at on public.frost_predictions(created_at desc);
create index if not exists idx_frost_predictions_user_id on public.frost_predictions(user_id);
create index if not exists idx_model_versions_status on public.model_versions(status);

drop trigger if exists set_users_profile_updated_at on public.users_profile;
create trigger set_users_profile_updated_at
before update on public.users_profile
for each row execute function public.set_updated_at();

drop trigger if exists set_inei_locations_updated_at on public.inei_locations;
create trigger set_inei_locations_updated_at
before update on public.inei_locations
for each row execute function public.set_updated_at();

drop trigger if exists set_populated_centers_updated_at on public.populated_centers;
create trigger set_populated_centers_updated_at
before update on public.populated_centers
for each row execute function public.set_updated_at();

drop trigger if exists set_agricultural_context_updated_at on public.agricultural_context;
create trigger set_agricultural_context_updated_at
before update on public.agricultural_context
for each row execute function public.set_updated_at();

drop trigger if exists set_training_runs_updated_at on public.training_runs;
create trigger set_training_runs_updated_at
before update on public.training_runs
for each row execute function public.set_updated_at();

drop trigger if exists set_alerts_updated_at on public.alerts;
create trigger set_alerts_updated_at
before update on public.alerts
for each row execute function public.set_updated_at();
```

supabase/migrations/002__add__feedback__and__observations.sql

```
create table if not exists public.prediction_feedback (
  id uuid primary key default gen_random_uuid(),
  prediction_id uuid references public.frost_predictions(id) on delete set null,
  district text not null,
  latitude numeric(9,6) check (latitude between -18.5 and -13.0),
  longitude numeric(9,6) check (longitude between -71.5 and -68.0),
  observed_risk_level text check (observed_risk_level in ('bajo', 'medio', 'alto')),
  user_observed_frost boolean,
  crop_damage_observed boolean,
  notes text,
  source text not null default 'app_feedback',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now()
);

create table if not exists public.official_frost_observations (
  id uuid primary key default gen_random_uuid(),
  observed_at timestamptz not null,
  district text not null,
  latitude numeric(9,6) not null check (latitude between -18.5 and -13.0),
  longitude numeric(9,6) not null check (longitude between -71.5 and -68.0),
  altitude numeric(8,2) check (altitude between 0 and 7000),
  temperature_2m numeric(6,2),
  relative_humidity_2m numeric(5,2) check (relative_humidity_2m between 0 and 100),
  apparent_temperature numeric(6,2),
  dew_point_2m numeric(6,2),
  precipitation numeric(8,2) check (precipitation is null or precipitation >= 0),
  cloud_cover numeric(5,2) check (cloud_cover between 0 and 100),
  wind_speed_10m numeric(7,2) check (wind_speed_10m is null or wind_speed_10m >= 0),
  observed_risk_level text not null check (observed_risk_level in ('bajo', 'medio', 'alto')),
  source_name text not null default 'SENAMHI',
  source_url text,
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now()
);

create index if not exists idx_prediction_feedback_created_at on public.prediction_feedback(created_at desc);
create index if not exists idx_official_frost_observations_district_time
  on public.official_frost_observations(district, observed_at desc);
```

supabase/migrations/20260529201250__apply__rls__policies.sql

```
alter table public.users_profile enable row level security;
alter table public.inei_locations enable row level security;
alter table public.populated_centers enable row level security;
alter table public.agricultural_context enable row level security;
alter table public.weather_records enable row level security;
alter table public.frost_predictions enable row level security;
alter table public.alerts enable row level security;
alter table public.model_versions enable row level security;
alter table public.training_runs enable row level security;
alter table public.data_sources_log enable row level security;
alter table public.prediction_feedback enable row level security;
alter table public.official_frost_observations enable row level security;

grant usage on schema public to anon, authenticated, service_role;

grant select on public.inei_locations to anon, authenticated;
grant select on public.populated_centers to anon, authenticated;
grant select on public.agricultural_context to anon, authenticated;
grant select on public.model_versions to anon, authenticated;
grant select on public.frost_predictions to authenticated;

grant select, insert, update, delete on public.users_profile to authenticated, service_role;
```

```
grant select, insert, update, delete on public.inei_locations to service_role;
grant select, insert, update, delete on public.populated_centers to service_role;
grant select, insert, update, delete on public.agricultural_context to service_role;
grant select, insert, update, delete on public.weather_records to service_role;
grant select, insert, update, delete on public.frost_predictions to service_role;
grant select, insert, update, delete on public.alerts to service_role;
grant select, insert, update, delete on public.model_versions to service_role;
grant select, insert, update, delete on public.training_runs to service_role;
grant select, insert, update, delete on public.data_sources_log to service_role;
grant select, insert, update, delete on public.prediction_feedback to service_role;
grant select, insert, update, delete on public.official_frost_observations to service_role;

drop policy if exists "Users can read own profile" on public.users_profile;
create policy "Users can read own profile"
on public.users_profile
for select
to authenticated
using ((select auth.uid()) = user_id);

drop policy if exists "Users can update own profile" on public.users_profile;
create policy "Users can update own profile"
on public.users_profile
for update
to authenticated
using ((select auth.uid()) = user_id)
with check ((select auth.uid()) = user_id);

drop policy if exists "Users can insert own profile" on public.users_profile;
create policy "Users can insert own profile"
on public.users_profile
for insert
to authenticated
with check ((select auth.uid()) = user_id);

drop policy if exists "Public can read INEI locations" on public.inei_locations;
create policy "Public can read INEI locations"
on public.inei_locations
for select
to anon, authenticated
using (true);

drop policy if exists "Public can read populated centers" on public.populated_centers;
create policy "Public can read populated centers"
on public.populated_centers
for select
to anon, authenticated
using (true);

drop policy if exists "Public can read agricultural context" on public.agricultural_context;
create policy "Public can read agricultural context"
on public.agricultural_context
for select
to anon, authenticated
using (true);

drop policy if exists "Users can read own frost predictions" on public.frost_predictions;
create policy "Users can read own frost predictions"
on public.frost_predictions
for select
to authenticated
using ((select auth.uid()) = user_id);

drop policy if exists "Public can read active model versions" on public.model_versions;
create policy "Public can read active model versions"
on public.model_versions
for select
to anon, authenticated
using (status = 'active');

-- No anon/authenticated insert policy is defined for frost_predictions,
-- prediction_feedback or official_frost_observations in this MVP.
-- FastAPI/ML operators write with the server-side service role key, which bypasses RLS.
-- Never expose the service role key in Flutter, Flutter Web, Vercel client code, or logs.
```

supabase/migrations/20260529202739_harden_private_rls_and_functions.sql

```
create or replace function public.set_updated_at()
returns trigger
language plpgsql
set search_path = public
as $function$
begin
  new.updated_at = now();
  return new;
end;
$function$;

drop policy if exists "Service role manages weather records" on public.weather_records;
create policy "Service role manages weather records"
on public.weather_records
for all
to service_role
using (true)
with check (true);

drop policy if exists "Service role manages alerts" on public.alerts;
create policy "Service role manages alerts"
on public.alerts
for all
to service_role
using (true)
with check (true);

drop policy if exists "Service role manages training runs" on public.training_runs;
create policy "Service role manages training runs"
on public.training_runs
for all
to service_role
using (true)
with check (true);

drop policy if exists "Service role manages data source logs" on public.data_sources_log;
create policy "Service role manages data source logs"
on public.data_sources_log
for all
to service_role
```

```
using (true)
with check (true);

drop policy if exists "Service role manages prediction feedback" on public.prediction_feedback;
create policy "Service role manages prediction feedback"
on public.prediction_feedback
for all
to service_role
using (true)
with check (true);

drop policy if exists "Service role manages official frost observations" on public.official_frost_observations;
create policy "Service role manages official frost observations"
on public.official_frost_observations
for all
to service_role
using (true)
with check (true);
```

supabase/policies/rls_policies.sql

```
alter table public.users_profile enable row level security;
alter table public.inei_locations enable row level security;
alter table public.populated_centers enable row level security;
alter table public.agricultural_context enable row level security;
alter table public.weather_records enable row level security;
alter table public.frost_predictions enable row level security;
alter table public.alerts enable row level security;
alter table public.model_versions enable row level security;
alter table public.training_runs enable row level security;
alter table public.data_sources_log enable row level security;
alter table public.prediction_feedback enable row level security;
alter table public.official_frost_observations enable row level security;

grant usage on schema public to anon, authenticated, service_role;

grant select on public.inei_locations to anon, authenticated;
grant select on public.populated_centers to anon, authenticated;
grant select on public.agricultural_context to anon, authenticated;
grant select on public.model_versions to anon, authenticated;
grant select on public.frost_predictions to authenticated;

grant select, insert, update, delete on public.users_profile to authenticated, service_role;
grant select, insert, update, delete on public.inei_locations to service_role;
grant select, insert, update, delete on public.populated_centers to service_role;
grant select, insert, update, delete on public.agricultural_context to service_role;
grant select, insert, update, delete on public.weather_records to service_role;
grant select, insert, update, delete on public.frost_predictions to service_role;
grant select, insert, update, delete on public.alerts to service_role;
grant select, insert, update, delete on public.model_versions to service_role;
grant select, insert, update, delete on public.training_runs to service_role;
grant select, insert, update, delete on public.data_sources_log to service_role;
grant select, insert, update, delete on public.prediction_feedback to service_role;
grant select, insert, update, delete on public.official_frost_observations to service_role;

drop policy if exists "Users can read own profile" on public.users_profile;
create policy "Users can read own profile"
on public.users_profile
for select
to authenticated
using ((select auth.uid()) = user_id);

drop policy if exists "Users can update own profile" on public.users_profile;
create policy "Users can update own profile"
on public.users_profile
for update
to authenticated
using ((select auth.uid()) = user_id)
with check ((select auth.uid()) = user_id);

drop policy if exists "Users can insert own profile" on public.users_profile;
create policy "Users can insert own profile"
on public.users_profile
for insert
to authenticated
with check ((select auth.uid()) = user_id);

drop policy if exists "Public can read INEI locations" on public.inei_locations;
create policy "Public can read INEI locations"
on public.inei_locations
for select
to anon, authenticated
using (true);

drop policy if exists "Public can read populated centers" on public.populated_centers;
create policy "Public can read populated centers"
on public.populated_centers
for select
to anon, authenticated
using (true);

drop policy if exists "Public can read agricultural context" on public.agricultural_context;
create policy "Public can read agricultural context"
on public.agricultural_context
for select
to anon, authenticated
using (true);

drop policy if exists "Users can read own frost predictions" on public.frost_predictions;
create policy "Users can read own frost predictions"
on public.frost_predictions
for select
to authenticated
using ((select auth.uid()) = user_id);

drop policy if exists "Public can read active model versions" on public.model_versions;
create policy "Public can read active model versions"
on public.model_versions
for select
to anon, authenticated
using (status = 'active');

-- No anon/authenticated insert policy is defined for frost_predictions in this MVP.
-- No anon/authenticated insert policy is defined for frost_predictions,
-- prediction_feedback or official_frost_observations in this MVP.
-- FastAPI/ML operators write with the server-side service role key, which bypasses RLS.
```

```
-- Never expose the service role key in Flutter, Flutter Web, Vercel client code, or logs.
```

supabase/seed/seed_locations.sql

```
insert into public.inei_locations (
  ubigeo,
  department,
  province,
  district,
  latitude,
  longitude,
  estimated_altitude,
  total_population,
  rural_population,
  rural_percentage,
  source,
  metadata
) values
  ('210101', 'Puno', 'Puno', 'Puno', -15.840200, -70.021900, 3827, 128637, 21868, 16.99, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before production."}'::jsonb),
  ('210201', 'Puno', 'Azangaro', 'Azangaro', -14.908400, -70.196200, 3859, 28137, 15194, 54.00, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before production."}'::jsonb),
  ('210301', 'Puno', 'Carabaya', 'Macusani', -14.069700, -70.431100, 4315, 12240, 4651, 38.00, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before production."}'::jsonb),
  ('210401', 'Puno', 'Chucuito', 'Juli', -16.212200, -69.459600, 3869, 22310, 12717, 57.00, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before production."}'::jsonb),
  ('210501', 'Puno', 'El Collao', 'Ilave', -16.083300, -69.638900, 3850, 50239, 34163, 68.00, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before production."}'::jsonb),
  ('211101', 'Puno', 'San Roman', 'Juliaca', -15.499700, -70.133300, 3825, 276110, 22089, 8.00, 'INEI-compatible MVP seed', '{"note":"Replace with official INEI EstaDist/CPV 2017 export before production."}'::jsonb)
on conflict (ubigeo) do update set
  department = excluded.department,
  province = excluded.province,
  district = excluded.district,
  latitude = excluded.latitude,
  longitude = excluded.longitude,
  estimated_altitude = excluded.estimated_altitude,
  total_population = excluded.total_population,
  rural_population = excluded.rural_population,
  rural_percentage = excluded.rural_percentage,
  source = excluded.source,
  metadata = excluded.metadata;

insert into public.model_versions (
  version,
  model_name,
  algorithm,
  artifact_path,
  accuracy,
  precision_macro,
  recall_macro,
  f1_macro,
  dataset_size,
  status,
  deployed_at,
  metadata
) values (
  'v0.1.0',
  'RandomForestClassifier',
  'RandomForestClassifier',
  'ml_pipeline/registry/frost_risk_model.joblib',
  1.000000,
  1.000000,
  1.000000,
  1.000000,
  4368,
  'active',
  now(),
  '{"limitations":["Initial labels are rule-based.","Metrics are optimistic because label-derived features are included."],"sources":["Open-Meteo","INEI-compatible MVP seed"]}'::jsonb
)
on conflict (version) do update set
  model_name = excluded.model_name,
  algorithm = excluded.algorithm,
  artifact_path = excluded.artifact_path,
  accuracy = excluded.accuracy,
  precision_macro = excluded.precision_macro,
  recall_macro = excluded.recall_macro,
  f1_macro = excluded.f1_macro,
  dataset_size = excluded.dataset_size,
  status = excluded.status,
  deployed_at = excluded.deployed_at,
  metadata = excluded.metadata;
```